Proyecto Final

Análisis de algoritmos en el contexto de la bibliometría

Cristhian Camilo López Grajales - 1007408599

Mariana Pineda Montenegro - 1095550335

Camila Hernández Peña - 1104544039



Docente: Sergio Augusto Cardona Torres

Universidad del Quindío

Facultad de Ingeniería de Sistemas y Computación

Análisis de algoritmos

Armenia, Quindío, 4/11/2026



Contenido

1. Introducción

- 1.1 Contexto y motivación
- 1.2 Objetivos
- 1.3 Alcance, corpus y restricciones del enunciado
- 1.4 Qué entrega este proyecto
- 1.5 Cómo está organizado este documento
- 1.6 Vista rápida del cumplimiento del enunciado

2. Conceptos básicos explicados con sencillez

- 2.1 De un texto a números
- 2.2 Similitud y distancia
- 2.3 Programación dinámica
- 2.4 Embeddings
- 2.5 Agrupar (clustering) y dendrograma
- 2.6 Buscar un patrón y la complejidad (Big O)

3. Arquitectura y diseño

- 3.1 Visión general
- 3.2 Flujo de datos
- 3.3 Capas del sistema y responsabilidades
- 3.4 Decisiones de diseño y sus razones
- 3.5 Tecnologías utilizadas
- 3.6 Estructura del repositorio
- 3.7 Cómo se relacionan los módulos entre sí

4. Corpus y preprocesamiento

- 4.1 Qué se pide y cómo se cumple
- 4.2 Extracción desde los PDFs
- 4.3 Reporte de calidad
- 4.4 Preprocesamiento escrito a mano
- 4.5 Qué texto recibe cada algoritmo
- 4.6 Relación entre cada concepto y el código

5. Requerimiento 1: similitud textual

5.1 Qué pide el enunciado y cómo se cumple
5.2 Panorama: seis algoritmos y qué mira cada uno
5.3 Algoritmos clásicos: idea, matemática y código
5.3.1 Levenshtein (distancia de edición)
5.3.2 Needleman-Wunsch (alineamiento global)
5.3.3 Coseno con TF-IDF
5.3.4 Coeficiente de Jaccard
5.4 Algoritmos con IA: embeddings
5.5 Relación entre las fórmulas y el código
5.6 Caso de estudio: abstracts #2 y #6
5.6.1 Cómo se eligió el caso
5.6.2 Los dos abstracts
5.6.3 Levenshtein paso a paso
5.6.4 Needleman-Wunsch paso a paso
5.6.5 Coseno con TF-IDF paso a paso
5.6.6 Jaccard paso a paso
5.6.7 Embeddings paso a paso
5.7 Análisis crítico: algoritmos clásicos frente a IA
5.8 Complejidad y tiempos
5.9 Para qué sirve
5.10 Cumplimiento del requerimiento 1

6. Requerimiento 2: clustering jerárquico

6.1 Qué pide el enunciado y cómo se cumple
6.2 Cómo funciona, paso a paso
6.3 Los tres criterios de enlace
6.4 Métricas internas: cómo saber cuál agrupa mejor
6.4.1 Correlación cofenética (la métrica principal)
6.4.2 Coeficiente de silueta (la métrica de apoyo)
6.4.3 De la fórmula al código
6.5 Matriz de distancias y pasos de fusión del corpus
6.6 Los dendrogramas
6.7 Evaluación y conclusión

6.8 Verificación contra scipy

6.9 Para qué sirve

6.10 Cumplimiento del requerimiento 2

7. Requerimiento 3: búsqueda de patrones

7.1 Qué pide el enunciado y cómo se cumple

7.2 Cómo funciona la búsqueda en la aplicación

7.3 Knuth-Morris-Pratt (KMP)

7.4 Boyer-Moore

7.5 Rabin-Karp

7.6 Relación entre las fórmulas y el código

7.7 Caso de estudio: el patrón «perspectives» en el abstract #9

7.7.1 KMP: tabla LPS

7.7.2 Boyer-Moore: tablas heurísticas

7.7.3 Rabin-Karp: cálculo numérico del hash

7.7.4 Resumen del caso

7.8 Análisis crítico: Big O frente a tiempos reales

7.8.1 Sobre el corpus completo

7.8.2 Escalabilidad

7.8.3 Casos límite

7.8.4 Lectura de los resultados

7.9 Para qué sirve

7.10 Cumplimiento del requerimiento 3

8. Requerimiento 4: despliegue y documentación

8.1 Qué pide el enunciado y cómo se cumple

8.2 La aplicación web y su API

8.3 Despliegue en Render

8.4 Documentación entregada

8.5 Pruebas y calidad

8.6 Uso de IA generativa

8.7 Cumplimiento del requerimiento 4

9. Guía del código: carpeta por carpeta, archivo por archivo

9.1 Mapa general del repositorio

9.2 Carpeta raíz

9.3 Carpeta src/: la lógica del proyecto

9.3.1 src/paths.py (57 líneas): rutas y carpetas

9.3.2 src/estructuras_datos.py (649 líneas): estructuras de datos propias

9.3.3 src/preprocesamiento.py (225 líneas): limpieza de texto

9.3.4 src/corpus.py (60 líneas): carga y cálculo base

9.3.5 src/similitud_clasica.py (319 líneas): los cuatro algoritmos clásicos

9.3.6 src/similitud_embeddings.py (75 líneas): similitud con embeddings

9.3.7 src/comparacion.py (131 líneas): comparar varios artículos

9.3.8 src/clustering.py (313 líneas): clustering jerárquico

9.3.9 src/patrones.py (331 líneas): KMP, Boyer-Moore y Rabin-Karp

9.3.10 src/benchmark_patrones.py (62 líneas): mediciones de tiempo

9.3.11 src/visualizacion.py (188 líneas): gráficas

9.3.12 src/docs_algoritmos.py y src/api.py

9.4 Carpeta scripts/: tareas fuera del servidor

9.5 Carpeta data/: los datos

9.6 Carpeta static/: la interfaz web

9.7 Carpeta tests/ y carpetas docs/ y modules/

9.8 Recorrido de una petición: qué funciones se ejecutan y en qué orden

10. Resultados consolidados y conclusiones

10.1 Resumen de resultados

10.2 Conclusiones

10.3 Limitaciones

10.4 Trabajo futuro

11. Matriz final de cumplimiento del enunciado

12. Declaración de uso de IA generativa

13. Referencias

Anexo A. Términos TF-IDF del caso de estudio (#2 y #6)

Anexo B. Matriz de distancias entre los 20 abstracts

Anexo C. Historial completo de fusiones

Anexo D. Los 5 pares más parecidos por algoritmo

Anexo E. Glosario

Anexo F. Guía de ejecución

F.1 Ejecutar la aplicación en local (Windows)

F.2 Regenerar los datos y los documentos (opcional)

F.3 Desplegar en Render

1. Introducción

1.1 Contexto y motivación

Cada año se publican miles de artículos sobre inteligencia artificial en la educación. Ninguna persona puede leerlos todos, y por eso hace falta apoyarse en el computador para comparar, agrupar y buscar entre ellos. De eso se ocupa, en buena parte, la bibliometría: el estudio de la producción científica con métodos cuantitativos.

Este proyecto toma un caso concreto y controlado: 20 artículos científicos sobre IA en educación, entregados en PDF. Con ellos respondemos tres preguntas muy prácticas: ¿qué tan parecidos son dos resúmenes?, ¿cómo se organizan los 20 artículos en familias? y ¿en qué parte exacta de los textos aparece una palabra o una frase?

Además, el proyecto tiene una intención formativa: entender qué pasa dentro de cada técnica. Por eso todos los algoritmos se programaron desde cero, con listas y bucles, en lugar de llamar a librerías que los resuelven en una sola línea. Así, cada número que muestra la aplicación se puede explicar paso a paso, con su fórmula y con el fragmento de código que la calcula.

1.2 Objetivos

Objetivo general. Implementar y analizar algoritmos clásicos y modelos de inteligencia artificial para el análisis computacional de resúmenes científicos, comparando ambos enfoques con rigor matemático, y ponerlos a disposición en una aplicación web desplegada y documentada.

Objetivos específicos.

- Extraer y validar título, autores y abstract de 20 artículos en PDF, y preprocesar el texto con herramientas escritas a mano.
- Medir la similitud entre abstracts con cuatro algoritmos clásicos (Levenshtein, Needleman-Wunsch, coseno con TF-IDF y Jaccard) y con dos modelos de embeddings, y contrastar los resultados en un caso de estudio.
- Agrupar los 20 abstracts con tres criterios de clustering jerárquico, dibujar sus dendrogramas y decidir con una métrica matemática cuál agrupa mejor.
- Buscar patrones de texto con KMP, Boyer-Moore y Rabin-Karp, y contrastar la complejidad teórica (Big O) con tiempos reales medidos.
- Desplegar la aplicación en Render y documentar el diseño, el código y los resultados, declarando el uso de IA generativa.

- Verificar el trabajo con pruebas automáticas y con comparaciones contra librerías de referencia, usadas únicamente en las pruebas.

1.3 Alcance, corpus y restricciones del enunciado

El enunciado oficial fija las reglas del juego. Las resumimos aquí porque condicionan todas las decisiones de diseño que verá en el resto del documento.

Tabla 1. Restricciones del enunciado y forma de cumplirlas

Tema	Lo que pide el enunciado	Cómo lo asumimos
Fuente de datos	Un corpus estático de 20 artículos en PDF entregados por el docente.	Los 20 PDFs se procesan una sola vez con scripts/extraer_corpus.py; la aplicación usa el resultado en JSON.
Texto que se analiza	Sólo el abstract. Título y autores sirven para identificar cada artículo.	Todos los algoritmos trabajan sobre el abstract. Título y autores se usan en las tablas y en la interfaz.
Algoritmos	Deben implementarse de forma explícita, sin funciones de alto nivel que los resuelvan.	Listas y bucles. Sin scipy, sklearn, numpy, pandas, difflib, find(), index(), in ni expresiones regulares en la lógica.
Librerías permitidas	Solo como apoyo (leer PDF, graficar, servir la web, obtener embeddings).	pypdf, matplotlib, Flask, gunicorn y dos modelos preentrenados de embeddings. scipy aparece solo en pruebas, para verificar.
Inteligencia artificial	Dos enfoques de similitud con embeddings.	GloVe y fastText de 300 dimensiones; los vectores se calculan una vez fuera del servidor.
Documentación	Documento técnico con arquitectura, casos de estudio y declaración del uso de IA generativa.	Este documento y la documentación técnica del repositorio.
Despliegue	Aplicación desplegada con enlace público.	Render, desde GitHub: https://bibliometria-algoritmos-gp0j.onrender.com

1.4 Qué entrega este proyecto

Tabla 2. Productos del proyecto

Producto	Descripción	Dónde está
Aplicación web	Cinco páginas: corpus, similitud, clustering, patrones y documentación.	https://bibliometria-algoritmos-gp0j.onrender.com
Código fuente	Algoritmos, API y scripts, con 94 pruebas automáticas.	https://github.com/Marp2/bibliometria
Datos procesados	Corpus en JSON, texto limpio, reporte de calidad y embeddings.	data/processed/
Documento técnico	Este documento (Word y PDF).	docs/ y entrega del curso
Evidencias	Números, tablas y figuras generados ejecutando el código.	docs/evidencias/ y docs/figuras/
Presentación	Diapositivas de apoyo para la exposición del trabajo.	Entrega del curso

1.5 Cómo está organizado este documento

El documento sigue el mismo orden en que se construyó el proyecto: primero los cimientos (datos y texto limpio), luego cada requerimiento y, al final, el despliegue, la guía del código y las conclusiones. Cada requerimiento tiene la misma estructura para que sea fácil comparar: qué se pide, cómo funciona, la matemática y su relación con el código, el caso de estudio, el análisis crítico, para qué sirve y cómo se cumple.

Tabla 3. Mapa del documento

Capítulo	Qué responde	Úselo si quiere…
2. Conceptos básicos	¿Qué ideas hay detrás de todo esto?	Entender los términos sin ser experto.
3. Arquitectura y diseño	¿Cómo está armado el sistema?	Ver el panorama general y las decisiones de diseño.
4. Corpus y preprocesamiento	¿De dónde salen los datos y cómo se limpian?	Conocer la base sobre la que trabajan los algoritmos.

Capítulo	Qué responde	Úselo si quiere…
5. Requerimiento 1	¿Qué tan parecidos son dos abstracts?	Ver los seis algoritmos de similitud y el caso de estudio.
6. Requerimiento 2	¿Cómo se agrupan los 20 artículos?	Ver el clustering, los dendrogramas y la evaluación.
7. Requerimiento 3	¿Dónde aparece un patrón de texto?	Ver KMP, Boyer-Moore, Rabin-Karp y los tiempos.
8. Requerimiento 4	¿Cómo se despliega y se prueba?	Ver la aplicación, la API, Render y las pruebas.
9. Guía del código	¿Dónde está cada cosa y qué hace?	Recorrer carpeta por carpeta y función por función.
10 a 13	Resultados, cumplimiento, uso de IA y referencias.	Evaluar el trabajo en conjunto.
Anexos	Tablas completas, glosario y guía de ejecución.	Consultar datos y repetir el trabajo.

1.6 Vista rápida del cumplimiento del enunciado

Tabla 4. Resumen del cumplimiento (el detalle está al final de cada requerimiento y en el capítulo 11)

Requisito	Dónde se explica	Estado
Corpus de 20 PDFs: título, autores y abstract validados	Capítulo 4	Cumple (20 de 20)
R1: selección dinámica de 2 o más artículos	5.2 y 8.2	Cumple (2 a 6 artículos)
R1: 4 algoritmos clásicos desde cero	5.3	Cumple
R1: 2 modelos de IA con embeddings	5.4	Cumple (GloVe y fastText)
R1: caso de estudio, paso a paso y análisis crítico	5.6 y 5.7	Cumple
R2: 3 criterios de enlace, historial y dendrogramas propios	6.3 y 6.6	Cumple

Requisito	Dónde se explica	Estado
R2: métrica interna y conclusión	6.4 y 6.7	Cumple (cofenética y silueta)
R3: KMP, Boyer-Moore y Rabin-Karp sin funciones nativas	7.3 a 7.5	Cumple
R3: caso de estudio con tablas, hash rodante y gráficas	7.7	Cumple
R3: Big O frente a tiempos reales	7.8	Cumple
R4: despliegue público y documentación	8.2 a 8.4	Cumple
Declaración de uso de IA generativa	Capítulo 12	Cumple
Pruebas y manejo de errores	8.5	Cumple (94 pruebas)

2. Conceptos básicos explicados con sencillez

Antes de entrar en los algoritmos conviene fijar unas pocas ideas. No se necesita formación previa: cada concepto se explica con un ejemplo pequeño. Quien ya los domina puede pasar directamente al capítulo 3.

2.1 De un texto a números

Un computador no entiende frases, pero sí números. El primer paso de casi cualquier análisis de texto es convertir cada texto en una lista de números que lo represente. Se hace en tres movimientos:

1. Dividir en palabras (*tokens*). «Learning with AI» se convierte en learning, with, ai.
2. Armar un vocabulario: la lista de todas las palabras distintas que aparecen en el conjunto de textos.
3. Contar: para cada texto, un número por cada palabra del vocabulario (cuántas veces aparece, o un peso que mide su importancia). Esa lista de números es el vector del texto.

Ejemplo mínimo. Texto A: learning ai. Texto B: ai learning assessment. El vocabulario es {ai, assessment, learning}. Los vectores con conteos simples son:

Tabla 5. Dos textos convertidos en vectores

Texto	ai	assessment	learning
A	1	0	1
B	1	1	1

Un vector es solo una lista de números ordenada. Si dos textos usan palabras parecidas, sus vectores se parecen; si usan palabras distintas, se alejan. Todo el proyecto descansa en esta idea.

2.2 Similitud y distancia

La similitud mide qué tan parecidos son dos textos (más alto, más parecidos). La distancia mide lo contrario: qué tan lejos están (más alto, más diferentes). Una se obtiene de la otra: en este proyecto, distancia = 1 − similitud. Todas las similitudes se normalizan para que queden entre 0 y 1, de modo que 1 signifique «idénticos» y 0 «sin nada en común».

Hay muchas maneras de medir la similitud, y cada una mira algo distinto. Algunas comparan la forma de las cadenas de letras, otras las palabras compartidas y otras el significado aproximado. Ese contraste es el corazón del requerimiento 1.

2.3 Programación dinámica

Es una forma de resolver un problema grande resolviendo antes problemas pequeños y guardando sus respuestas en una tabla, para no repetir cuentas. Para comparar dos palabras, por ejemplo, se construye una tabla donde cada casilla responde: «¿qué tan lejos están los primeros *i* caracteres de la primera palabra de los primeros *j* de la segunda?». La casilla final contiene la respuesta completa. Levenshtein y Needleman-Wunsch funcionan así.

2.4 Embeddings

Un *embedding* es un vector de números que representa el significado aproximado de una palabra. Se obtiene entrenando un modelo con millones de textos: palabras que se usan en contextos parecidos (por ejemplo, «teacher» y «student») terminan con vectores cercanos. En este proyecto usamos dos modelos ya entrenados por terceros, GloVe y fastText, de 300 números por palabra. El vector de un abstract completo se obtiene promediando los vectores de sus palabras.

2.5 Agrupar (clustering) y dendrograma

Agrupar consiste en organizar los textos en familias según su parecido, sin que nadie diga antes cuáles son las familias. En el clustering jerárquico se empieza con cada texto solo y, paso a paso, se unen los dos grupos más cercanos hasta que todo queda en uno. El registro de esas uniones se dibuja como un árbol llamado dendrograma: las hojas son los textos y la altura de cada unión indica a qué distancia se juntaron.

2.6 Buscar un patrón y la complejidad (Big O)

Buscar un patrón es encontrar todas las apariciones exactas de una cadena (por ejemplo, «assessment») dentro de un texto. La forma ingenua prueba el patrón en cada posición, pero hay algoritmos más inteligentes. Para compararlos se usa la notación Big O, que describe cómo crece el trabajo cuando el problema se hace más grande: si el texto tiene n caracteres y el patrón m, un algoritmo $O(n)$ hace trabajo proporcional a n, mientras que uno $O(n \cdot m)$ puede hacer mucho más. También se habla de mejor caso, caso promedio y peor caso.

En síntesis. Casi todo este proyecto consiste en dos movimientos: convertir texto en números y medir qué tan cerca están esos números, o recorrer el texto de forma inteligente para encontrar algo. Las fórmulas que aparecen más adelante son la versión precisa de esas dos ideas.

3. Arquitectura y diseño

3.1 Visión general

La aplicación se divide en tres capas que conviene tener presentes: los datos (calculados una sola vez), la lógica algorítmica (módulos de Python escritos desde cero) y la presentación (una API en Flask y cinco páginas web). El diagrama de componentes muestra cómo se relacionan.

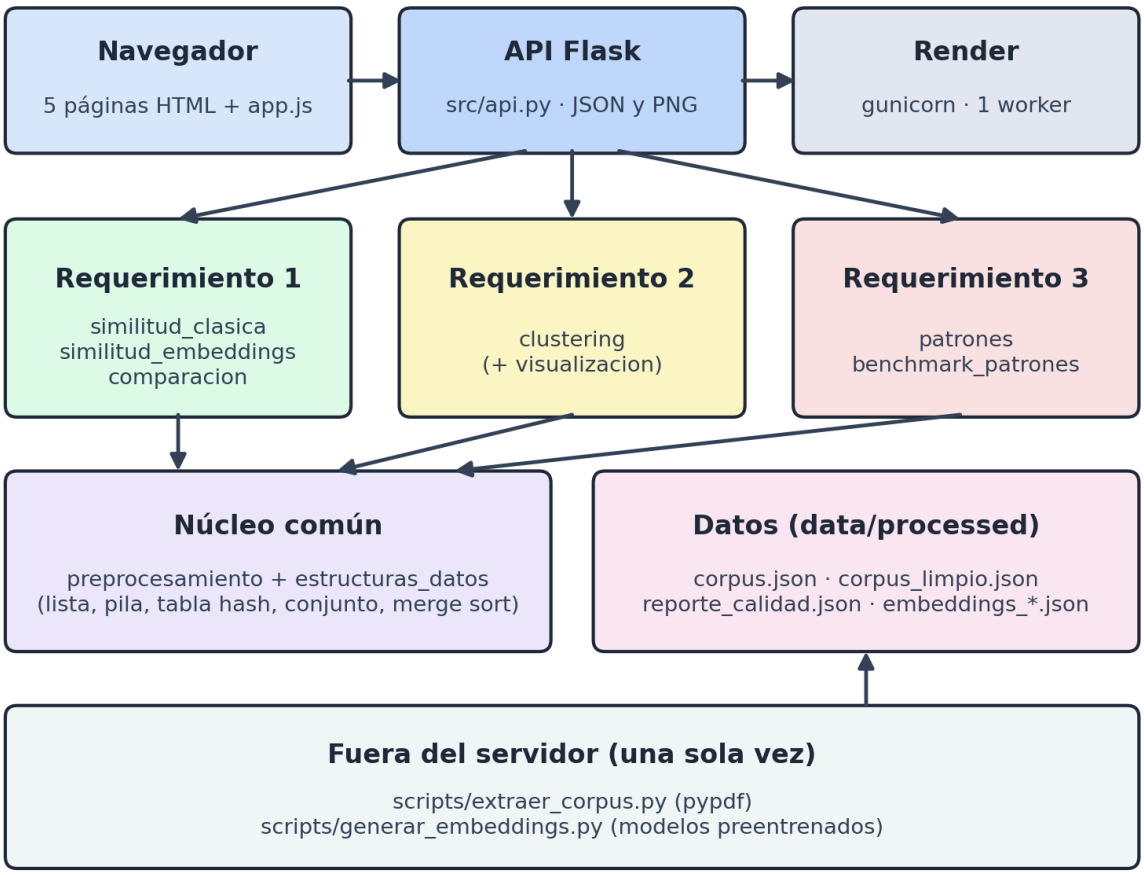


Figura 1. *Diagrama de componentes: la API conecta el navegador con los tres módulos algorítmicos, que comparten el núcleo común y los datos.*

La idea central del diseño es separar lo pesado de lo liviano. Leer PDFs y calcular embeddings requiere librerías grandes y modelos de cientos de megabytes, así que se hace fuera del servidor, una sola vez, y deja archivos JSON como resultado. El servidor solo lee esos JSON y ejecuta los algoritmos propios. Gracias a eso la aplicación cabe en el plan gratuito de Render.

3.2 Flujo de datos

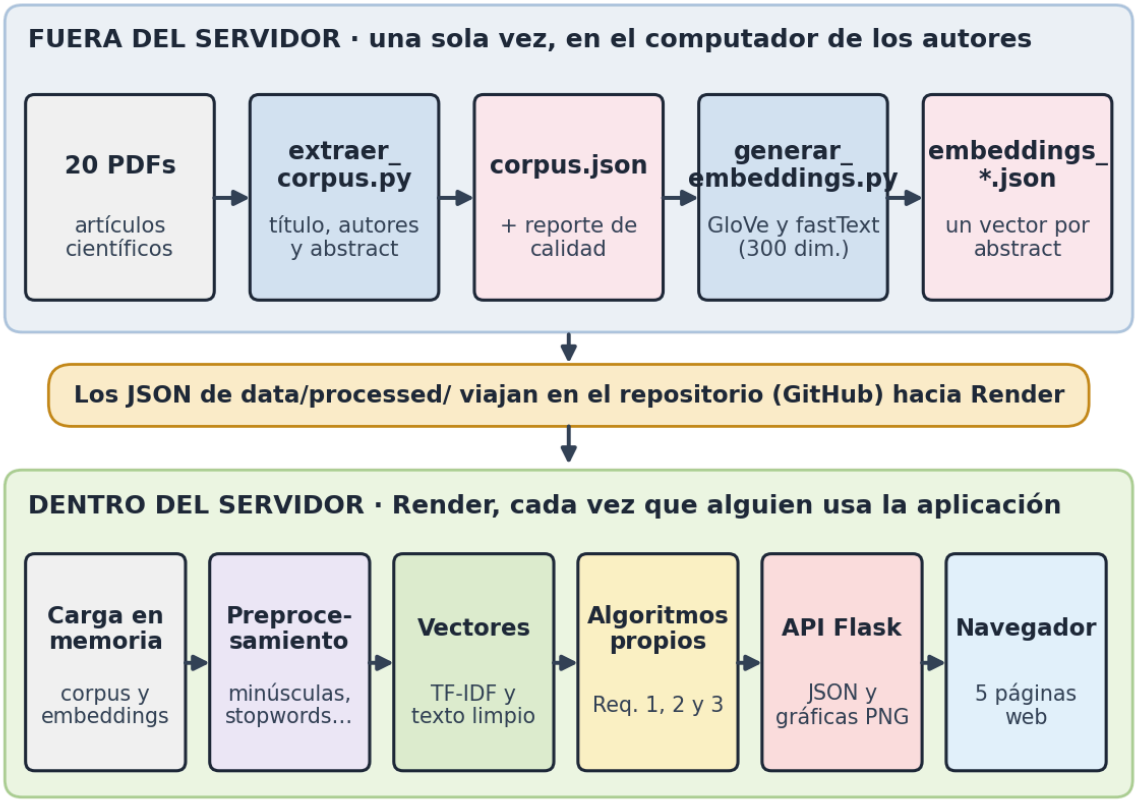


Figura 2. *Flujo de datos: lo que se hace una sola vez fuera del servidor y lo que ocurre dentro de él cada vez que alguien usa la aplicación.*

En palabras: los PDFs se convierten en corpus.json y los modelos de embeddings en embeddings_*.json. Esos archivos viajan en el repositorio hacia Render. Cuando alguien abre la aplicación, el servidor carga los JSON en memoria, limpia los textos, calcula los vectores TF-IDF y pone todo a disposición de los algoritmos. La API responde con datos (JSON) o gráficas (PNG) y el navegador las dibuja.

3.3 Capas del sistema y responsabilidades

Tabla 6. Capas del sistema

Capa	Archivos	Responsabilidad
Datos fuera del servidor	scripts/extraer_corpus.py, scripts/generar_embeddings.py	Convertir PDFs en corpus.json y reporte_calidad.json; calcular embeddings_<modelo>.json.

Capa	Archivos	Responsabilidad
Estructuras propias	src/estructuras_datos.py	Lista dinámica, pila, tabla hash, conjuntos, matrices, ordenamiento por mezcla y utilidades matemáticas.
Preprocesamiento	src/preprocesamiento.py	Minúsculas, tildes, signos, tokenización, stopwords y stemming, todo a mano.
Requerimiento 1	src/similitud_clasica.py, src/similitud_embeddings.py, src/comparacion.py	Los seis algoritmos de similitud y la comparación de varios artículos.
Requerimiento 2	src/clustering.py	Tres criterios de enlace, historial de fusiones, correlación cofenética y silueta.
Requerimiento 3	src/patrones.py, src/benchmark_patrones.py	KMP, Boyer-Moore, Rabin-Karp y mediciones de tiempo.
Presentación	src/visualizacion.py, src/api.py, static/	Gráficas PNG, endpoints JSON y las cinco páginas web.
Configuración	src/paths.py, Procfile, render.yaml	Rutas por variables de entorno (con carpeta temporal de respaldo) y arranque en Render.

3.4 Decisiones de diseño y sus razones

Tabla 7. Decisiones de diseño

Decisión	Alternativa descartada	Por qué
Calcular los embeddings fuera del servidor	Cargar el modelo dentro de la aplicación	Los modelos pesan cientos de MB o GB y no caben en el plan gratuito de Render.
Dos modelos distintos: GloVe y fastText (300 dimensiones)	Un solo modelo	El enunciado pide dos, y se entrenan con métodos diferentes (co-ocurrencia global frente a un modelo tipo word2vec con subpalabras).

Decisión	Alternativa descartada	Por qué
Levenshtein y Needleman-Wunsch sobre caracteres del texto limpio	Comparar palabra por palabra	Es la forma clásica de la programación dinámica, y con unos 185 ms por par es viable en la web.
IDF suavizado: ln((1+N)/(1+df)) + 1, con N = 20	IDF = ln(N/df)	Evita que una palabra presente en todos los abstracts tenga peso cero.
Distancia de clustering = 1 − coseno TF-IDF	Distancia euclidiana	El coseno ignora cuánto texto tiene cada abstract y mira solo hacia dónde «apunta» su vocabulario.
Criterios Single, Complete y Average (UPGMA)	Ward	El enunciado pide tres; Ward requiere distancias euclidianas y aquí la distancia natural es 1 − coseno.
Métrica principal: correlación cofenética; apoyo: silueta	Solo silueta	La silueta resultó baja y casi plana en este corpus; la cofenética mide si el árbol es fiel a las distancias.
Clustering directo desde la matriz original, $O(n^3)$	Fórmulas de actualización de Lance-Williams	Con 20 documentos es instantáneo y la definición de cada criterio queda a la vista en el código.
Buscar sobre el abstract original, sin distinguir mayúsculas por defecto	Buscar en el texto preprocesado	Las posiciones y el resaltado deben corresponder al texto real; pasar a minúsculas conserva la longitud.
Rabin-Karp con base 256 y módulo primo 1.000.000.007	Módulo pequeño	Casi no hay colisiones; el módulo pequeño se usa solo para demostrarlas.
Un solo worker de gunicorn y caché en memoria	Varios workers	Plan gratuito con poca memoria; los cálculos son deterministas, así que cachearlos es seguro.

3.5 Tecnologías utilizadas

Tabla 8. Tecnologías y su papel

Tecnología	Para qué se usa	¿Entra en la lógica de los algoritmos?
Python 3.12	Lenguaje de todo el proyecto.	Sí, solo estructuras básicas del lenguaje (listas, bucles, funciones).
Flask y gunicorn	Servir la API y las páginas web.	No.
matplotlib	Dibujar dendrogramas, mapas de calor y gráficas de iteraciones.	No: los dendrogramas se dibujan a partir del historial propio.
pypdf	Leer el texto de los PDFs (solo en el script offline).	No.
GloVe y fastText	Vectores de palabras preentrenados (solo en el script offline).	No: coseno y distancia euclidiana se calculan con funciones propias.
pytest y scipy	Pruebas; scipy solo para verificar resultados.	No: nunca se importan desde src/.
HTML, CSS y JavaScript	Interfaz web de cinco páginas.	No.

3.6 Estructura del repositorio

```
bibliometria-ia/
├── api.py                punto de entrada local (python api.py)
├── Procfile, render.yaml   arranque y despliegue en Render
├── requirements*.txt       dependencias (la app solo necesita 3)
├── src/                  toda la lógica (algoritmos, API, gráficas)
├── scripts/              tareas fuera del servidor y generación de evidencias
├── data/processed/         corpus, texto limpio, calidad y embeddings (JSON)
├── static/               interfaz web: styles.css, app.js y 5 páginas
├── modules/                README de cada módulo (propósito y evidencias)
├── tests/                94 pruebas automáticas (pytest)
```

```
└── docs/                documentación, figuras y evidencias
```

El capítulo 9 recorre cada una de estas carpetas, archivo por archivo y función por función.

3.7 Cómo se relacionan los módulos entre sí

Los módulos están organizados en niveles: los de abajo no conocen a los de arriba. Así se evitan dependencias circulares y cada pieza se puede probar por separado.

Tabla 9. Dependencias entre módulos

Módulo	Usa a	Lo usan
estructuras_datos	(nada)	Casi todos los demás módulos
preprocesamiento	estructuras_datos	corpus, patrones (solo a_minusculas), api
similitud_clasica	estructuras_datos	comparacion, clustering, corpus
similitud_embeddings	estructuras_datos, paths	comparacion, corpus
corpus	paths, preprocesamiento, similitud_clasica, similitud_embeddings	api
comparacion	similitud_clasica, similitud_embeddings, estructuras_datos	api
clustering	estructuras_datos, similitud_clasica	api, visualizacion
patrones	estructuras_datos, preprocesamiento	benchmark_patrones, api
benchmark_patrones	patrones, estructuras_datos	api
visualizacion	clustering	api
docs_algoritmos	(nada)	api
api	todos los anteriores	api.py (raíz), gunicorn

4. Corpus y preprocesamiento

4.1 Qué se pide y cómo se cumple

El enunciado entrega 20 artículos en PDF y pide trabajar sobre el abstract de cada uno, usando título y autores para identificarlos. Esto exige dos cosas: sacar esa información de los PDFs de forma confiable y limpiar el texto para que los algoritmos comparen lo importante. Esta sección explica ambas, y la de preprocesamiento a mano también responde a la regla de no usar librerías de procesamiento de lenguaje.

4.2 Extracción desde los PDFs

La extracción la hace scripts/extraer_corpus.py, que se ejecuta una sola vez en el computador de los autores (la aplicación web no lee PDFs). Es una herramienta de preparación de datos, por eso puede usar métodos como find(); los algoritmos evaluados, en cambio, están en src/ y no los usan. El script sigue estos pasos:

1. Lee los metadatos verificados de data/metadata_verificada.json. Los títulos y autores se comprobaron a mano contra la primera página de cada PDF, porque los metadatos internos de los archivos son incompletos (a veces solo traen al primer autor).
2. Lee el texto de las dos primeras páginas con pypdf (leer_texto) y corrige las ligaduras tipográficas: el PDF guarda «fi» como un solo carácter especial, y si no se corrige, «artificial» quedaría «artificial» (normalizar).
3. Arma un vocabulario global con las palabras completas de los 20 PDFs (vocabulario_global). Sirve para decidir cómo reparar palabras cortadas al final de una línea.
4. Une las palabras partidas por un guion de fin de línea solo si la palabra resultante existe en ese vocabulario (unir_guiones). Así «edu-cation» pasa a «education», pero «well-known» conserva su guion.
5. Localiza el abstract (localizar_abstract): busca marcas de inicio como ABSTRACT o Abstract: y marcas de fin como Keywords o 1. Introduction. Para los dos artículos que no tienen marcas limpias se usa un ancla manual: el #7 (inicio) y el #9 (fin).
6. Limpia y corrige (limpiar_abstract, reparar_guiones_en_linea, aplicar_correccion, aplicar_reemplazos): une líneas, quita espacios dobles y aplica las correcciones documentadas que se resumen en la sección 4.3.
7. Evalúa la calidad de cada abstract (evaluar_calidad) y escribe data/processed/corpus.json y reporte_calidad.json.

El script hace lo mismo que haría una persona con paciencia: abre cada PDF, busca el resumen, arregla las palabras cortadas y revisa que el resultado esté completo. La diferencia es que lo hace de forma repetible y deja constancia de cada decisión.

4.3 Reporte de calidad

Para no confiar a ciegas en la extracción, cada abstract pasa por controles automáticos: que tenga entre 80 y 300 palabras, que empiece con mayúscula y termine en punto, que no contenga residuos (palabras clave, «Received», citas entre paréntesis), que el título aparezca textual en la primera página y que coincida (Jaccard ≥ 0,9) con una extracción independiente hecha con otra herramienta (Java) en un trabajo anterior del curso. Resultado: 20 de 20 abstracts validados.

Tabla 10. Los 20 artículos del corpus y su validación

ID	Autores	Palabras	Caracteres	Estado	Título
1	4	187	1.147	OK	Artificial intelligence literacy in higher and adult education: A scoping literature review
2	5	127	904	OK	Systematic literature review on opportunities, challenges, and future research recommendations of artificial intelligence in education
3	4	233	1.545	OK	Myths, mis- and preconceptions of artificial intelligence: A review of the literature
4	5	207	1.410	OK	Opportunities of artificial intelligence for supporting complex problem-solving: Findings from a scoping review
5	4	244	1.816	OK	Application and theory gaps during the rise of Artificial Intelligence in Education
6	3	172	1.190	OK	A multi-perspective study on Artificial Intelligence in Education: grants, conferences, journals, software tools, institutions, and researchers
7	7	232	1.603	OK	Human-centred learning analytics and AI in education: A systematic literature review

ID	Autores	Palabras	Caracteres	Estado	Título
8	3	209	1.435	OK	A systematic review of artificial intelligence techniques for collaborative learning over the past two decades
9	2	161	1.245	OK	AI technologies for education: Recent research & future directions
10	2	200	1.441	OK	Educational applications of artificial intelligence in simulation-based learning: A systematic mapping review
11	5	185	1.379	OK	A scoping review on how generative artificial intelligence transforms assessment in higher education
12	9	250	1.793	OK	A meta systematic review of artificial intelligence in higher education: a call for increased ethics, collaboration, and rigour
13	2	246	1.668	OK	Artificial intelligence for education and its emphasis on assessment and adversity quotient: a review
14	3	131	979	OK	A comprehensive exploration of artificial intelligence competence frameworks for educators: A critical review
15	5	218	1.476	OK	A Systematic Review of Generative AI for Teaching and Learning Practice
16	2	153	1.088	OK	Generative Artificial Intelligence in Tertiary Education: Assessment Redesign Principles and Considerations
17	3	204	1.506	OK	A Systematic Review of AI-Driven Educational Assessment in STEM Education

ID	Autores	Palabras	Caracteres	Estado	Título
18	4	206	1.572	OK	Transforming Education: A Comprehensive Review of Generative Artificial Intelligence in Educational Settings through Bibliometric and Content Analysis
19	1	210	1.425	OK	What Is the Impact of ChatGPT on Education? A Rapid Review of the Literature
20	4	207	1.456	OK	Systematic review of research on artificial intelligence applications in higher education – where are the educators?

Tabla 11. Casos especiales de extracción y su tratamiento

Artículo	Qué pasaba	Qué se hizo
#6	El PDF pierde el espacio antes de una ligadura: «researchfield».	Reemplazo documentado por «research field».
#7	Diseño de dos columnas: no hay marca de inicio limpia.	Ancla manual de inicio («The rapid expansion of Learning Analytics»).
#9	El abstract no termina con «Keywords» ni «Introduction».	Ancla manual de fin («…longitudinal research and development efforts.»).
#14	El PDF publicado trae una referencia bibliográfica completa pegada dentro del abstract.	Se conserva el original en abstract_original y se elimina la cita en abstract.
#17	Guion de fin de línea que el vocabulario no pudo resolver: «holisti-cally».	Reemplazo documentado por «holistically».

Todas las correcciones quedan registradas en el propio corpus.json (campo correccion_manual), de modo que cualquier persona pueda ver qué se tocó y por qué.

4.4 Preprocesamiento escrito a mano

Una vez extraído el abstract, hay que prepararlo para compararlo. El preprocesamiento vive en src/preprocesamiento.py y se compone de seis pasos pequeños, cada uno en su propia función. Se

escribieron con bucles sobre caracteres, sin expresiones regulares ni funciones como split() o lower(), para que cada paso sea transparente.

Tabla 12. Los seis pasos del preprocesamiento

Paso	Función	Qué hace	Para qué sirve
1. Minúsculas	a_minusculas	Convierte A-Z y las letras latinas acentuadas mayúsculas a minúscula, sin cambiar la longitud del texto.	Education y education pasan a ser el mismo término.
2. Sin tildes	quitar_tildes	Reemplaza letras con acento por su versión simple.	Unifica variantes en español (educación, educacion).
3. Sin signos	limpiar_signos	Convierte en espacio todo lo que no sea una letra a-z: dígitos, guiones, apóstrofes y puntuación.	AI-driven (2023) queda como ai driven.
4. Tokenizar	tokenizar	Parte el texto por espacios con un bucle.	Obtiene la unidad mínima de análisis: la palabra.
5. Stopwords	quitar_stopwords	Descarta palabras vacías en inglés y español y las de menos de 2 letras.	Deja las palabras que sí distinguen un texto de otro. ai se conserva.
6. Stemming simple	stem_simple	Recorta sufijos frecuentes con reglas (-ation, -ing, -ies, -ed, -es, -s…).	Une variantes: learning, learner, learn.

Un ejemplo completo, ejecutado con las funciones reales. Frase original: «The Learners' Education: AI-driven assessments (2023) were studied.»

Tabla 13. Evolución de una frase a lo largo del preprocesamiento

Etapa	Resultado
Minúsculas	the learners' education: ai-driven assessments (2023) were studied.
Sin signos	the learners education ai driven assessments were studied
Tokens	the | learners | education | ai | driven | assessments | were | studied

Etapa	Resultado
Sin stopwords	learners | education | ai | driven | assessments | studied
Con stemming	learner | educate | ai | driven | assessment | study

Sobre las stopwords. Las listas viven en el propio archivo: 159 palabras en inglés y 107 en español, guardadas en una tabla hash propia para consultarlas rápido. Incluyen términos como based o using, muy frecuentes en los artículos científicos. Sobre el stemming. Es deliberadamente simple: la primera regla que coincide se aplica, y no toca palabras cortas ni las que terminan en «ss», «us» o «is» (para no estropear analysis o class). Las reglas, en orden, son:

Tabla 14. Reglas del stemming simple

Sufijo	Se reemplaza por	Raíz mínima	Ejemplo
-ational	-ate	3	educational → educate
-izations, -ization	-ize	3	organization → organize
-ations, -ation	-ate	3	education → educate
-ies	-y	2	studies → study
-sses	-ss	2	classes → class
-ings, -ing	(nada)	3	learning → learn
-ied	-y	2	applied → apply
-ed	(nada)	3	assessed → assess
-ly	(nada)	3	mainly → main
-es, -s	(nada)	3	models → model

Un matiz importante**.** El stemming simple puede equivocarse (por ejemplo, series pasa a sery). Lo que importa para los algoritmos es que sea consistente: la misma palabra siempre se reduce de la misma forma, de modo que dos abstracts que la usen coincidan en ese término.

4.5 Qué texto recibe cada algoritmo

No todos los algoritmos necesitan el mismo nivel de limpieza. La siguiente tabla indica qué versión del texto usa cada uno y por qué.

Tabla 15. Texto de entrada de cada algoritmo

Algoritmo	Texto que recibe	Razón
Levenshtein y Needleman-Wunsch	Texto limpio (minúsculas, sin signos, sin stopwords), sin stemming, como cadena de caracteres.	Comparan letras; el stemming alteraría las palabras.
Coseno TF-IDF, Jaccard y clustering	Tokens sin stopwords y con stemming.	Comparan vocabulario; conviene unir variantes de una misma palabra.
Embeddings	Tokens sin stopwords y sin stemming, procesados fuera del servidor.	Los modelos conocen palabras completas, no raíces recortadas.
Búsqueda de patrones	Abstract original, sin preprocesar.	Las posiciones y el resaltado deben corresponder al texto real.

Efecto medido del stemming. El corpus tiene 3.982 palabras en total y 2.431 tras quitar stopwords. El vocabulario pasa de 973 a 833 términos distintos con stemming, es decir, un 14,4 % menos. Con un vocabulario más compacto, dos abstracts que hablan de lo mismo con palabras ligeramente distintas coinciden más. El texto limpio de cada abstract queda guardado en data/processed/corpus_limpio.json.

4.6 Relación entre cada concepto y el código

Tabla 16. Conceptos y su implementación (los números de línea corresponden a la versión entregada)

Concepto	Archivo	Función	Línea
Extraer el abstract de un PDF	scripts/extraer_corpus.py	localizar_abstract	L104
Reparar palabras cortadas	scripts/extraer_corpus.py	unir_guiones	L73
Controles de calidad	scripts/extraer_corpus.py	evaluar_calidad	L203
Pasar a minúsculas	src/preprocesamiento.py	a_minusculas	L65
Quitar signos	src/preprocesamiento.py	limpiar_signos	L117
Quitar stopwords	src/preprocesamiento.py	quitar_stopwords	L137
Stemming	src/preprocesamiento.py	stem_simple	L167

Concepto	Archivo	Función	Línea
Todo el pipeline con pasos intermedios	src/preprocesamiento.py	preprocesar	L207
Cargar el corpus y construir vectores	src/corpus.py	Corpus.__init__	L14

En el código. El código del preprocesamiento está pensado para leerse como una receta. preprocesar llama, en orden, a a_minusculas, quitar_tildes, limpiar_signos, tokenizar, quitar_stopwords y aplicar_stemming, y devuelve cada resultado intermedio. Por eso la página Corpus de la aplicación puede mostrar el texto en cada etapa.

5. Requerimiento 1: similitud textual

5.1 Qué pide el enunciado y cómo se cumple

El enunciado pide que la aplicación permita elegir dos o más artículos del corpus y comparar sus abstracts con seis algoritmos: cuatro clásicos y dos basados en inteligencia artificial. Además exige una documentación con un caso concreto: dos abstracts específicos, la demostración paso a paso de cómo funciona cada algoritmo y un análisis crítico que contraste los resultados clásicos con los de IA.

Tabla 17. Requerimiento 1: exigencias y cumplimiento

Lo que pide el enunciado	Cómo lo cumplimos	Dónde
Seleccionar dinámicamente 2 o más artículos	La página «Similitud» permite marcar de 2 a 6 artículos y compara todos los pares.	static/modules/similarity/, src/comparacion.py
4 algoritmos clásicos desde cero (programación dinámica y espacio vectorial)	Levenshtein, Needleman-Wunsch, coseno con TF-IDF y Jaccard, con listas y bucles.	src/similitud_clasica.py
2 algoritmos con IA basados en embeddings	Dos modelos pre entrenados (GloVe y fastText) con coseno y distancia euclidiana calculados a mano.	src/similitud_embeddings.py, scripts/generar_embeddings.py
Caso con dos abstracts concretos	Artículos #2 y #6, elegidos de forma objetiva entre los 190 pares posibles.	Sección 5.6
Demostración paso a paso	Matrices de programación dinámica, vocabulario, TF, IDF, pesos, producto punto, normas, ángulo, intersección y unión, y obtención del embedding.	Sección 5.6
Análisis crítico clásicos frente a IA	Tablas comparativas, correlaciones entre algoritmos y explicación de por qué difieren.	Sección 5.7
Complejidad de cada algoritmo	Tabla de complejidad y tiempos medidos.	Secciones 5.2 y 5.8

5.2 Panorama: seis algoritmos y qué mira cada uno

Antes de entrar al detalle, conviene tener claro que cada algoritmo mira algo distinto de los textos. Esa diferencia es la que explica sus resultados en el análisis crítico.

Tabla 18. Los seis algoritmos de similitud

Algoritmo	Familia	Qué compara	Complejidad por par
Levenshtein	Clásico · programación dinámica	Cuántas ediciones (insertar, borrar, sustituir) separan dos cadenas de letras.	O(m·n) tiempo; O(mín(m,n)) espacio
Needleman-Wunsch	Clásico · programación dinámica	Qué tan bien se pueden alinear dos cadenas, premiando coincidencias y castigando diferencias y huecos.	O(m·n)
Coseno con TF-IDF	Clásico · espacio vectorial	El ángulo entre los vectores de palabras ponderadas por su rareza.	O(V) por par, con V términos de vocabulario
Jaccard	Clásico · conjuntos	Qué proporción del vocabulario total comparten los dos textos.	O(m + n) promedio
GloVe	IA · embeddings	Cercanía del significado aproximado, con vectores de 300 números.	O(d), d = 300
fastText	IA · embeddings	Igual que el anterior, con otro modelo entrenado de otra manera.	O(d), d = 300

Qué ocurre cuando se pulsa «Comparar» en la aplicación

1. El navegador envía los números de los artículos a POST /api/similarity (src/api.py, función api_similarity).
2. comparacion.comparar valida la lista: entre 2 y 6 artículos distintos que existan en el corpus; si no, responde con un error claro.
3. Para cada par llama a comparacion.similitud_par, que ejecuta los seis algoritmos y mide el tiempo de cada uno.
4. La API devuelve las similitudes y las matrices; static/app.js las dibuja como tablas, barras y mapa de calor.

5.3 Algoritmos clásicos: idea, matemática y código

5.3.1 Levenshtein (distancia de edición)

La idea. Imagine que debe convertir la palabra gato en pato modificando una letra a la vez. Puede insertar una letra, borrar una o sustituir una por otra. La distancia de Levenshtein es el menor número de operaciones que lo logra. Para gato y pato basta una sustitución (la g por la p), así que la distancia es 1.

La matemática. Se construye una tabla D en la que la casilla $D[i][j]$ guarda la distancia entre los primeros i caracteres de la primera cadena y los primeros j de la segunda. La primera fila y la primera columna son los costos de construir una cadena a partir de la vacía, y el resto se llena con una regla local:

$$D[i][0] = i \qquad D[0][j] = j$$

Casos base: pasar de/hacia la cadena vacía cuesta tantas operaciones como letras tenga la otra

$$D[i][j] = \text{mín}(D[i-1][j] + 1,\ D[i][j-1] + 1,\ D[i-1][j-1] + \text{costo})$$

costo = 0 si a[i] = b[j]; costo = 1 si son distintos

Las tres opciones del mínimo corresponden a **borrar** (venir de la casilla de arriba), **insertar** (venir de la izquierda) y **sustituir o conservar** (venir de la diagonal). La esquina inferior derecha, $D[m][n]$, es la distancia total. Para llevarla a una similitud entre 0 y 1 se divide entre la longitud de la cadena más larga:

$$\text{similitud} = 1 - D[m][n] / \text{máx}(m, n)$$

Ejemplo con gato y pato, calculado con la función real. Cada casilla se obtiene del mínimo de sus tres vecinas (arriba +1, izquierda +1, diagonal +0 o +1):

Tabla 19. Tabla de Levenshtein para gato → pato: la esquina inferior derecha vale 1

	—	p	a	t	o
—	0	1	2	3	4
g	1	1	2	3	4
a	2	2	1	2	3
t	3	3	2	1	2
o	4	4	3	2	1

La distancia es **1** y la similitud es 1 − 1/4 = **0,75**.

El código. Así se escribe la tabla en src/similitud_clasica.py, función levenshtein_matriz (línea 23). Compárelo con la fórmula: cada línea borrar, insertar y sustituir es una de las tres opciones del mínimo.

Fragmento de similitud_clasica.py

```
def levenshtein_matriz(a, b):
    la, lb = len(a), len(b)
    D = crear_matriz(la + 1, lb + 1, 0)
    i = 0
    while i <= la:
        D[i][0] = i
        i += 1
    j = 0
    while j <= lb:
        D[0][j] = j
        j += 1
    i = 1
    while i <= la:
        j = 1
        while j <= lb:
            costo = 0 if a[i - 1] == b[j - 1] else 1
            borrar = D[i - 1][j] + 1
            insertar = D[i][j - 1] + 1
            sustituir = D[i - 1][j - 1] + costo
            mejor = borrar
            if insertar < mejor:
                mejor = insertar
            if sustituir < mejor:
                mejor = sustituir
            D[i][j] = mejor
            j += 1
        i += 1
    return D
```

- Las dos primeras repeticiones (while i <= la, while j <= lb) llenan la primera columna y la primera fila: los casos base.
- El doble bucle recorre cada casilla en orden, de modo que cuando se calcula D[i][j] sus tres vecinas ya tienen valor.
- El costo es 0 si los caracteres coinciden y 1 si no. Las tres variables borrar, insertar y sustituir son los tres candidatos; después se escoge el menor con comparaciones simples, sin usar min().

Una versión más ligera. Para comparar los textos completos no hace falta guardar toda la tabla: cada fila solo depende de la anterior. La función levenshtein_distancia (línea 54) conserva únicamente dos filas, así que el espacio baja de O(m·n) a O(mín(m, n)) sin cambiar el resultado. La aplicación usa esa versión; la tabla completa se usa solo para mostrar el llenado paso a paso.

Complejidad. Hay que calcular m·n casillas y cada una cuesta un tiempo constante, así que el tiempo es O(m·n). Con abstracts de cientos de caracteres son cientos de miles de casillas por par, y por eso Levenshtein es de los algoritmos más lentos del requerimiento (unos 185 ms por par).

5.3.2 Needleman-Wunsch (alineamiento global)

La idea. Nace en la biología (para comparar secuencias de ADN) y se parece mucho a Levenshtein, pero en vez de contar errores suma puntos: da un punto por cada coincidencia y resta puntos por cada diferencia o por cada hueco que haya que dejar para alinear las cadenas. Además de un número, permite recuperar el alineamiento concreto, es decir, cómo encajan las dos cadenas.

La matemática. Se usa la misma tabla, con otra regla. Los puntajes que usamos son +1 por coincidencia, −1 por diferencia y −1 por hueco:

$$F[i][0] = i \cdot hueco \qquad F[0][j] = j \cdot hueco$$

$$F[i][j] = máx(\, F[i-1][j-1] + s(a[i], b[j]),\ F[i-1][j] + hueco,\ F[i][j-1] + hueco \,)$$

$$s = +1 \text{ si } a[i] = b[j];\ s = -1 \text{ si no; } hueco = -1$$

Aquí se toma el máximo (no el mínimo): se busca el mejor puntaje posible. Como el resultado puede ser negativo, hay que normalizar para obtener una similitud entre 0 y 1. Para eso se usan las cotas teóricas: el mejor puntaje posible y el peor.

$$máximo = mín(m, n) - |m - n| \qquad mínimo = -(m + n)$$

mejor caso: todo coincide y solo sobran huecos; peor caso: todo son huecos

$$similitud = (puntaje - mínimo) \,/\, (máximo - mínimo)$$

Ejemplo con gato y pato: el puntaje óptimo es 2 (tres coincidencias y una diferencia: 3 − 1) y el alineamiento recuperado con el *traceback* (el camino de vuelta desde la esquina final) es:

```
gato
pato
```

El código. La tabla se llena en needleman_wunsch_matriz (línea 96) y el camino de vuelta, en needleman_wunsch_alineamiento (línea 146). La normalización está en needleman_wunsch_normalizado (línea 171).

Núcleo de needleman_wunsch_matriz: el máximo de tres candidatos

```
   while j <= lb:
        F[0][j] = j * hueco
        j += 1
    i = 1
    while i <= la:
        j = 1
        while j <= lb:
            diagonal = F[i - 1][j - 1] + (coincide if a[i - 1] == b[j - 1]
else difiere)
            arriba = F[i - 1][j] + hueco
            izquierda = F[i][j - 1] + hueco
            F[i][j] = mayor_de_tres(diagonal, arriba, izquierda)
            j += 1
        i += 1
    return F
```

- diagonal es venir de la casilla diagonal sumando +1 o −1 según los caracteres coincidan; arriba e izquierda son los dos huecos.
- mayor_de_tres (de estructuras_datos.py) escoge el máximo: reemplaza a la función max() de Python.
- El *traceback* parte de F[m][n] y retrocede: si el valor vino de la diagonal, alinea las dos letras; si vino de arriba o de la izquierda, inserta un guion (hueco) en una de las cadenas.

Complejidad. Igual que Levenshtein: O(m·n) en tiempo, y O(n) de espacio si solo se necesita el puntaje (needleman_wunsch_puntaje, que guarda dos filas) u O(m·n) si se quiere el alineamiento.

Levenshtein pregunta “¿cuántos errores hay entre las dos cadenas?” y Needleman-Wunsch “¿qué tan bien encajan una sobre otra?”. Son dos formas de medir lo mismo, la forma de las letras, que no tiene nada que ver con el tema del texto. Más adelante veremos las consecuencias.

5.3.3 Coseno con TF-IDF

La idea. Aquí se deja de mirar letra por letra y se mira qué palabras usa cada texto. Cada abstract se convierte en un vector con un número por cada palabra del vocabulario. Ese número no es un simple conteo: es un peso que refleja qué tan importante es la palabra *en ese texto* y qué tan *especial* es en el corpus. Después se mide el ángulo entre los vectores: si apuntan en la misma dirección, los textos usan vocabulario parecido.

La matemática. El peso se llama TF-IDF y combina dos factores:

$$TF(t, d) = \text{veces que aparece } t \text{ en } d \,/\, \text{total de palabras de } d$$

Frecuencia del término: qué tanto se menciona en el texto

$$IDF(t) = \ln((1 + N) / (1 + df(t))) + 1$$

N = 20 abstracts; df(t) = en cuántos abstracts aparece t

$$peso(t, d) = TF(t, d) \cdot IDF(t)$$

Una palabra como educate**,** que aparece en 18 de los 20 abstracts, tiene un IDF bajo porque no distingue a nadie. Una como aied**,** que aparece en 7, tiene un IDF más alto: si dos abstracts la comparten, eso cuenta más. Se usa la versión «suavizada» (el +1 dentro del logaritmo y fuera de él) para que ninguna palabra tenga peso cero, ni siquiera las que están en todos los textos.

Con los dos vectores u y v ya construidos, la similitud es el coseno del ángulo que forman:

$$coseno(u, v) = (u \cdot v) / (|u| \cdot |v|)$$

$u \cdot v$ = suma de los productos término a término; $|u|$ = raíz de la suma de los cuadrados

$$\text{ángulo} = \arccos(\text{coseno})$$

Un coseno de 1 significa vectores idénticos en dirección (ángulo de 0°); un coseno de 0, vectores perpendiculares (90°), es decir, textos sin ninguna palabra en común. Como los pesos TF-IDF nunca son negativos, el coseno siempre queda entre 0 y 1.

El código. Cuatro funciones de src/similitud_clasica.py hacen todo el trabajo:

Tabla 20. Funciones del coseno con TF-IDF

Función	Línea	Qué hace	Fórmula que implementa
construir_vocabulario	L198	Reúne las palabras distintas de los 20 abstracts, las ordena y cuenta en cuántos documentos aparece cada una (df).	Vocabulario y df(t)
calcular_idf	L222	Calcula el IDF de cada palabra.	IDF(t) = ln((1+N)/(1+df)) + 1
vector_tf y vector_tfidf	L241, L254	Arman el vector del abstract: TF y luego TF × IDF.	peso(t, d) = TF · IDF
coseno_vectores	L276	Producto punto entre las normas.	coseno = (u·v)/(|u||v|)

calcular_idf y coseno_vectores, tal como están en el código

```
def calcular_idf(vocab):
    N = vocab["n_docs"]
    idf = [0.0] * len(vocab["df"])
    k = 0
    while k < len(idf):
        idf[k] = math.log((1 + N) / (1 + vocab["df"][k])) + 1
        k += 1
    return idf

def coseno_vectores(u, v):
    nu, nv = norma(u), norma(v)
    if nu == 0.0 or nv == 0.0:
        return 0.0
    c = producto_punto(u, v) / (nu * nv)
    if c > 1.0:
        c = 1.0
```

```
    if c < -1.0:
        c = -1.0
    return c
```

Fíjese en la correspondencia casi literal con las fórmulas: math.log((1 + N) / (1 + df)) + 1 es el IDF; producto_punto(u, v) / (nu * nv) es el coseno. Hay además dos protecciones: si algún vector es cero se devuelve 0 (evita dividir entre cero) y el resultado se recorta a [−1, 1] (evita errores de redondeo antes de acos).

Complejidad. Construir el vocabulario cuesta una pasada por todos los textos. Una vez hecho, cada par se compara en O(V), con V = 833 términos, es decir, en una fracción de milisegundo.

5.3.4 Coeficiente de Jaccard

La idea. Es la más sencilla: se ignora cuántas veces aparece cada palabra y se pregunta qué fracción del vocabulario total comparten los dos textos. Si A y B son los conjuntos de palabras de cada abstract:

$$J(A, B) = |A \cap B| / |A \cup B|$$

$A \cap B$: palabras en ambos textos; $A \cup B$: palabras que aparecen en al menos uno

Vale 1 si los dos conjuntos son idénticos y 0 si no comparten nada. Ejemplo pequeño: si A = {ai, education, learning} y B = {education, learning, assessment, chatbot}, comparten 2 palabras y entre los dos suman 5 distintas, así que J = 2/5 = 0,40.

Función jaccard (similitud_clasica.py, línea 295)

```
def jaccard(tokens_a, tokens_b):
    A = ConjuntoHash()
    B = ConjuntoHash()
    for t in tokens_a:
        A.agregar(t)
    for t in tokens_b:
        B.agregar(t)
    interseccion = ListaDinamica()
    union = ConjuntoHash()
    for t in A.elementos():
        union.agregar(t)
```

```
        if B.contiene(t):
            interseccion.agregar(t)
    for t in B.elementos():
        union.agregar(t)
    n_union = union.tamano()
    similitud = interseccion.tamano() / n_union if n_union > 0 else 1.0
    return {
        "similitud": similitud,
        "interseccion": ordenar_por_mezcla(interseccion.a_lista()),
        "union": ordenar_por_mezcla(union.elementos()),
        "tamano_a": A.tamano(),
        "tamano_b": B.tamano(),
}
```

En el código, ConjuntoHash (nuestra propia estructura, basada en una tabla hash) permite saber en tiempo casi constante si una palabra ya está en un conjunto. El primer bucle recorre A, agrega cada palabra a la unión y, si también está en B, la agrega a la intersección; el segundo completa la unión con las palabras de B. Complejidad: O(m + n) en promedio.

5.4 Algoritmos con IA: embeddings

La idea**.** Los cuatro algoritmos anteriores solo ven letras o palabras exactas. Para ellos, teacher y educator no tienen nada que ver. Los embeddings intentan resolver eso: cada palabra se representa con un vector de números aprendido a partir de millones de textos, y palabras con significados parecidos quedan con vectores cercanos. Con eso se puede comparar el *significado aproximado* de dos abstracts.

Cómo se obtienen los vectores. Usamos dos modelos preentrenados, que son diccionarios que ya traen un vector de 300 números por palabra. El script scripts/generar_embeddings.py se ejecuta una sola vez en el computador de los autores y, para cada abstract:

1. toma sus palabras sin stopwords y sin stemming (los modelos conocen palabras completas);
2. busca en el modelo el vector de cada palabra (leer_vectores), conservando sólo las palabras que necesita el corpus para no cargar todo el modelo;
3. promedia los vectores encontrados (promedio_normalizado) y divide el resultado entre su norma, para que el vector final tenga longitud 1;

4. guarda un vector por abstract en data/processed/embeddings_<modelo>.json, junto con la cobertura (qué fracción de las palabras tenía vector en el modelo).

Tabla 21. Los dos modelos de embeddings utilizados

Modelo	Entrenamiento	Dimensión	Cobertura media de palabras
GloVe (glove-wiki-gigaword-300)	Wikipedia 2014 + Gigaword 5 (6 000 millones de palabras). Aprende a partir de la co-ocurrencia global de las palabras.	300	97,2 %
fastText (fasttext-wiki-news-subwords-300)	Wikipedia 2017 + UMBC + noticias (16 000 millones de palabras). Modelo tipo word2vec que además aprovecha información de subpalabras.	300	97,2 %

La matemática. Con los vectores de dos abstracts u y v se calculan, con funciones propias, dos medidas:

$$coseno(u, v) = (u \cdot v) / (|u| \cdot |v|)$$

$$d(u, v) = \sqrt{(\Sigma (u_i - v_i)^2)} \qquad similitud\ euclidiana = 1 / (1 + d)$$

La similitud 1/(1+d) vale 1 cuando d = 0 y tiende a 0 cuando la distancia crece

Como los vectores tienen longitud 1, ambas medidas están ligadas: $d^2 = 2 - 2 \cdot coseno$. Por eso ordenan los pares exactamente igual; la aplicación muestra el coseno como similitud principal y la euclidiana como dato adicional.

similitud_embeddings.py: coseno (L24), distancia_euclidiana (L37) y similitud_euclidiana_01 (L53)

```
def coseno(u, v):
    nu = math.sqrt(producto_punto(u, u))
    nv = math.sqrt(producto_punto(v, v))
    if nu == 0.0 or nv == 0.0:
        return 0.0
    c = producto_punto(u, v) / (nu * nv)
    if c > 1.0:
        c = 1.0
    if c < -1.0:
```

```
        c = -1.0
    return c

def distancia_euclidiana(u, v):
    total = 0.0
    k = 0
    while k < len(u):
        d = u[k] - v[k]
        total += d * d
        k += 1
    return math.sqrt(total)

def similitud_euclidiana_01(u, v):
    return 1.0 / (1.0 + distancia_euclidiana(u, v))
```

La IA no «lee» el abstract: promedia el significado aproximado de sus palabras y compara esos promedios. Es una idea poderosa pero tosca: al promediar se pierde el orden de las palabras y el contexto. Más adelante veremos qué efecto tiene eso en los resultados.

Complejidad. Cada comparación recorre los 300 números del vector, es decir O(d) con d = 300: una fracción de milisegundo. El costo real de los embeddings está en calcularlos (cargar modelos de cientos de megabytes), y por eso se hace fuera del servidor.

5.5 Relación entre las fórmulas y el código

Esta tabla reúne en un solo lugar lo que se explicó antes. Sirve como índice para pasar de una expresión matemática al lugar exacto del código donde se calcula.

Tabla 22. De la fórmula al código (Requerimiento 1)

Expresión matemática	Significado	Función	Archivo y línea
D[i][j] = mín(arriba+1, izq.+1, diag.+costo)	Distancia de edición entre prefijos	levenshtein_matriz	similitud_clasica.py L23
sim = 1 − D[m][n] / máx(m, n)	Normalizar la distancia a [0, 1]	levenshtein_similitud	similitud_clasica.py L86

Expresión matemática	Significado	Función	Archivo y línea
F[i][j] = máx(diag+s, arriba+hueco, izq.+hueco)	Mejor puntaje de alineamiento	needleman_wunsch_matriz	similitud_clasica.py L96
(puntaje − mínimo)/(máximo − mínimo)	Normalizar con cotas teóricas	needleman_wunsch_normalizado	similitud_clasica.py L171
IDF = ln((1+N)/(1+df)) + 1	Rareza de una palabra en el corpus	calcular_idf	similitud_clasica.py L222
TF = conteo / total de palabras	Frecuencia de una palabra en un texto	vector_tf	similitud_clasica.py L241
coseno = (u·v)/(|u||v|)	Parecido de dirección entre vectores	coseno_vectores	similitud_clasica.py L276
ángulo = arccos(coseno)	El mismo dato, en grados	angulo_grados	similitud_clasica.py L288
J = |A ∩ B| / |A ∪ B|	Fracción de vocabulario compartido	jaccard	similitud_clasica.py L295
$d = \sqrt{\Sigma(u_i - v_i)^2}$	Distancia entre embeddings	distancia_euclidiana	similitud_embeddings.py L37
sim = 1/(1 + d)	Convertir distancia en similitud	similitud_euclidiana_01	similitud_embeddings.py L53
Reunir los seis algoritmos para un par	Resultado completo de la comparación	similitud_par	comparacion.py L45

5.6 Caso de estudio: abstracts #2 y #6

5.6.1 Cómo se eligió el caso

En lugar de escoger dos artículos a gusto, usamos un criterio objetivo: se calculó el coseno TF-IDF de los 190 pares posibles entre los 20 abstracts y se tomó el más parecido: los artículos #2 y #6. Para tener un punto de contraste se tomó también el menos parecido: #14 y #19. Así el caso de estudio muestra cómo reacciona cada algoritmo ante dos textos muy cercanos y ante dos muy lejanos.

5.6.2 Los dos abstracts

Abstract #2. *Systematic literature review on opportunities, challenges, and future research recommendations of artificial intelligence in education*

> *Applications of artificial intelligence in education (AIEd) are emerging and are new to researchers and practitioners alike. Reviews of the relevant literature have not examined how AI technologies have been integrated into each of the four key educational domains of learning, teaching, assessment, and administration. The relationships between the technologies and learning outcomes for students and teachers have also been neglected. This systematic review study aims to understand the opportunities and challenges of AIEd by examining the literature from the last 10 years (2012 – 2021) using matrix coding and content analysis approaches. The results present the current focus of AIEd research by identifying 13 roles of AI technologies in the key educational domains, 7 learning outcomes of AIEd, and 10 major challenges. The review also provides suggestions for future directions of AIEd research.*

Abstract #6. *A multi-perspective study on Artificial Intelligence in Education: grants, conferences, journals, software tools, institutions, and researchers*

> *With the rapid development of artificial intelligence (AI) technologies and a continuously growing interest in their application in educational contexts, there has been significant growth in the scientific literature in relation to the application of AI in education (AIEd). This study aims to present multiple perspectives on the development of AIEd in terms of relevant grants, conferences, journals, software tools, article trends, top issues, institutions, and researchers to provide an overview of AIEd for its further development and implementation. With this study, we contribute to the research field by enabling educators and scholars to understand the status and development of relevant grants and publications concerning AIEd. Also,findings concerning active actors can help educators and scholars identify the active researchers and institutions in the research on AIEd. Furthermore, researchers and educators are able to identify relevant journals and be more aware of major issues in AIEd studies. In addition, we also highlight the significance and necessity of the launch of the new Elsevier journal AIEd-related journal named Computers & Education: Artificial Intelligence.*

Tras el preprocesamiento, el abstract #2 queda con 54 términos distintos y el #6 con 68 (sin repetir palabras, y con stemming). Entre los dos usan 104 términos distintos, de un vocabulario total de 833 en los 20 abstracts.

5.6.3 Levenshtein paso a paso

Llenar una tabla de cientos × cientos de casillas no cabe en un documento, así que se muestra completa sobre un fragmento corto: los primeros 10 caracteres del texto limpio de cada abstract, applicatio y rapid deve. La mecánica es idéntica a la del ejemplo gato/pato; solo cambian los textos.

Tabla 23. Tabla de Levenshtein sobre el fragmento (la esquina inferior derecha vale 9)

	—	r	a	p	i	d		d	e	v	e
—	0	1	2	3	4	5	6	7	8	9	10
a	1	1	1	2	3	4	5	6	7	8	9
p	2	2	2	1	2	3	4	5	6	7	8
p	3	3	3	2	2	3	4	5	6	7	8
l	4	4	4	3	3	3	4	5	6	7	8
i	5	5	5	4	3	4	4	5	6	7	8
c	6	6	6	5	4	4	5	5	6	7	8
a	7	7	6	6	5	5	5	6	6	7	8
t	8	8	7	7	6	6	6	6	7	7	8
i	9	9	8	8	7	7	7	7	7	8	8
o	10	10	9	9	8	8	8	8	8	8	9

Sobre los textos limpios completos (652 y 903 caracteres) la tabla tiene 653 × 904 casillas. La distancia es 608 y la similitud es 1 − 608/903 = 0,3267. En otras palabras: para transformar un texto en el otro habría que hacer unas 608 ediciones, y como el texto más largo mide 903, la similitud es baja.

5.6.4 Needleman-Wunsch paso a paso

Con los mismos fragmentos y los puntajes +1 (coincidencia), −1 (diferencia) y −1 (hueco), la primera fila y la primera columna valen 0, −1, −2…, y el resto sale del máximo de tres candidatos:

Tabla 24. Tabla de Needleman-Wunsch sobre el fragmento (la esquina inferior derecha vale -6)

	—	r	a	p	i	d		d	e	v	e
—	0	-1	-2	-3	-4	-5	-6	-7	-8	-9	-10
a	-1	-1	0	-1	-2	-3	-4	-5	-6	-7	-8

	—	r	a	p	i	d		d	e	v	e
p	-2	-2	-1	1	0	-1	-2	-3	-4	-5	-6
p	-3	-3	-2	0	0	-1	-2	-3	-4	-5	-6
l	-4	-4	-3	-1	-1	-1	-2	-3	-4	-5	-6
i	-5	-5	-4	-2	0	-1	-2	-3	-4	-5	-6
c	-6	-6	-5	-3	-1	-1	-2	-3	-4	-5	-6
a	-7	-7	-5	-4	-2	-2	-2	-3	-4	-5	-6
t	-8	-8	-6	-5	-3	-3	-3	-3	-4	-5	-6
i	-9	-9	-7	-6	-4	-4	-4	-4	-4	-5	-6
o	-10	-10	-8	-7	-5	-5	-5	-5	-5	-5	-6

El puntaje del fragmento es -6. Siguiendo el camino de vuelta desde la esquina se recupera el alineamiento (el guion marca un hueco):

> -appli-catio
> ra-p-id deve

Sobre los textos completos el puntaje es -268. Las cotas teóricas son: máximo = mín(652, 903) − |652 − 903| = 401 y mínimo = −(652 + 903) = -1555. Aplicando la normalización: (-268 − (-1555)) / (401 − (-1555)) = 0,6580.

5.6.5 Coseno con TF-IDF paso a paso

1. **Vocabulario.** Los 20 abstracts forman un vocabulario de 833 términos (con stemming). Estos dos usan 104 en conjunto (la unión).
2. **IDF.** Para cada término se cuenta en cuántos de los 20 abstracts aparece (df) y se calcula IDF = ln(21/(1+df)) + 1. Ejemplos reales en la tabla siguiente.
3. **TF y peso.** Para cada abstract, TF = veces que aparece el término / total de palabras del abstract; el peso es TF × IDF.
4. **Producto punto.** Se multiplican los pesos de los dos abstracts término a término y se suman. Solo cuentan los términos que están en ambos.
5. **Normas.** Para cada abstract, raíz de la suma de los cuadrados de sus pesos.

6. **Coseno y ángulo.** Producto punto dividido entre el producto de las normas; el ángulo es su arcocoseno.

Tabla 25. Ejemplos reales de IDF: cuanto más raro el término, mayor su peso

Término (stem)	Aparece en (df)	IDF = ln(21/(1+df)) + 1
aied	7	1,9651
educate	18	1,1001
ai	18	1,1001
researcher	5	2,2528

La tabla siguiente muestra los 15 términos que más aportan al producto punto. Los 104 términos completos están en el Anexo A.

Tabla 26. Los 15 términos con mayor aporte al producto punto

Término	df	IDF	TF #2	TF #6	Peso #2	Peso #6	Producto
aied	7	1,965	0,0676	0,0686	0,1328	0,1349	0,01791
researcher	5	2,253	0,0135	0,0294	0,0304	0,0663	0,00202
relevant	7	1,965	0,0135	0,0294	0,0266	0,0578	0,00153
educate	18	1,100	0,0405	0,0294	0,0446	0,0324	0,00144
major	2	2,946	0,0135	0,0098	0,0398	0,0289	0,00115
new	3	2,658	0,0135	0,0098	0,0359	0,0261	0,00094
technology	12	1,480	0,0405	0,0098	0,0600	0,0145	0,00087
aim	4	2,435	0,0135	0,0098	0,0329	0,0239	0,00079
research	16	1,211	0,0270	0,0196	0,0327	0,0238	0,00078
identify	10	1,647	0,0135	0,0196	0,0223	0,0323	0,00072
study	14	1,336	0,0135	0,0294	0,0181	0,0393	0,00071
applicate	11	1,560	0,0135	0,0196	0,0211	0,0306	0,00064
literature	11	1,560	0,0270	0,0098	0,0422	0,0153	0,00064
ai	18	1,100	0,0270	0,0196	0,0297	0,0216	0,00064

Término	df	IDF	TF #2	TF #6	Peso #2	Peso #6	Producto
present	6	2,099	0,0135	0,0098	0,0284	0,0206	0,00058

Resultados. Sumando todos los productos se obtiene el producto punto $u \cdot v = 0{,}032251$. Las normas son |u| = 0,306488 y |v| = 0,320173. Entonces:

$$coseno = 0{,}032251 / (0{,}306488 \times 0{,}320173) = 0{,}3287$$

$$ángulo = arccos(0{,}3287) = 70{,}8°$$

Un ángulo de unos 71° indica que los dos vectores apuntan en direcciones distintas, pero no opuestas: comparten una parte del vocabulario, no la mayoría. Aun así, es el par más parecido de todo el corpus según esta medida, lo que dice mucho sobre lo diverso que es el vocabulario de los 20 abstracts.

5.6.6 Jaccard paso a paso

Con stemming, el abstract #2 tiene |A| = 54 términos distintos y el #6 tiene |B| = 68. Los términos compartidos son 18 y la unión tiene 104 términos.

Intersección (18 términos): ai, aied, aim, applicate, artificial, educate, identify, intelligence, literature, major, new, present, relevant, research, researcher, study, technology, understand.

$$J = 18 / 104 = 0{,}1731$$

5.6.7 Embeddings paso a paso

Para cada modelo, el vector del abstract se obtuvo promediando los vectores de sus palabras. La tabla muestra qué tan bien cubrió el modelo el vocabulario de cada abstract, las primeras seis componentes de cada vector (de 300) y los resultados:

Tabla 27. Embeddings del caso de estudio

Modelo	Palabras con vector #2	Palabras con vector #6	Coseno	Distancia euclidiana	1/(1+d)
glove-wiki-gigaword-300	69 de 74 (93,2 %)	95 de 102 (93,1 %)	0,9509	0,3135	0,7613
fasttext-wiki-news-subwords-300	69 de 74 (93,2 %)	95 de 102 (93,1 %)	0,9737	0,2296	0,8133

Primeras seis componentes (glove-wiki-gigaword-300): vector #2 = [-0,061; -0,004; 0,027; -0,053; 0,003; -0,011, …]; vector #6 = [-0,049; -0,005; 0,008; -0,033; 0,002; 0,004, …].

Primeras seis componentes (fasttext-wiki-news-subwords-300): vector #2 = [0,007; -0,075; 0,000; 0,016; -0,032; -0,034, …]; vector #6 = [-0,000; -0,078; 0,022; -0,007; -0,048; -0,033, …].

Como los vectores tienen longitud 1, se cumple $d^2 = 2 - 2 \cdot coseno$; por ejemplo, para GloVe: $2 - 2 \times 0{,}9509 = 0{,}0983 = d^2$, y $\sqrt{0{,}0983} = 0{,}3135$.

5.7 Análisis crítico: algoritmos clásicos frente a IA

Ahora comparamos los resultados de los seis algoritmos para el mismo caso, y también para el par de contraste.

Tabla 28. Similitud del caso de estudio y del contraste

Algoritmo	Tipo	Caso #2 y #6	Contraste #14 y #19	Puesto del caso entre 190 pares
Levenshtein	Clásico	0,3267	0,2822	3 de 190
Needleman-Wunsch	Clásico	0,6580	0,5997	31 de 190
Coseno TF-IDF	Clásico	0,3287	0,0215	1 de 190
Jaccard	Clásico	0,1731	0,0510	7 de 190
GloVe (coseno)	IA	0,9509	0,8786	18 de 190
fastText (coseno)	IA	0,9737	0,9162	16 de 190

Para saber si estos números son típicos, se calcularon los 190 pares y se resumió cada algoritmo:

Tabla 29. Comportamiento de cada algoritmo en los 190 pares

Algoritmo	Mínimo	Máximo	Promedio	Rango	Tiempo medio por par (ms)
Levenshtein	0,226	0,332	0,282	0,106	184,66
Needleman-Wunsch	0,539	0,764	0,610	0,225	187,17
Coseno TF-IDF	0,022	0,329	0,124	0,307	0,17
Jaccard	0,027	0,192	0,106	0,166	1,54
GloVe (coseno)	0,838	0,964	0,914	0,125	0,15

Algoritmo	Mínimo	Máximo	Promedio	Rango	Tiempo medio por par (ms)
fastText (coseno)	0,910	0,983	0,954	0,072	0,14

Y se midió cuánto **coinciden en el orden** de los pares (coeficiente de correlación de Spearman: 1 = mismo orden, 0 = sin relación):

Tabla 30. Correlación de Spearman entre algoritmos

	Levenshtein	Needleman-Wunsch	Coseno TF-IDF	Jaccard	GloVe	fastText
Levenshtein	1,00	0,89	0,32	0,18	0,23	0,16
Needleman-Wunsch	0,89	1,00	0,25	0,10	0,16	0,06
Coseno TF-IDF	0,32	0,25	1,00	0,57	0,64	0,54
Jaccard	0,18	0,10	0,57	1,00	0,45	0,29
GloVe	0,23	0,16	0,64	0,45	1,00	0,69
fastText	0,16	0,06	0,54	0,29	0,69	1,00

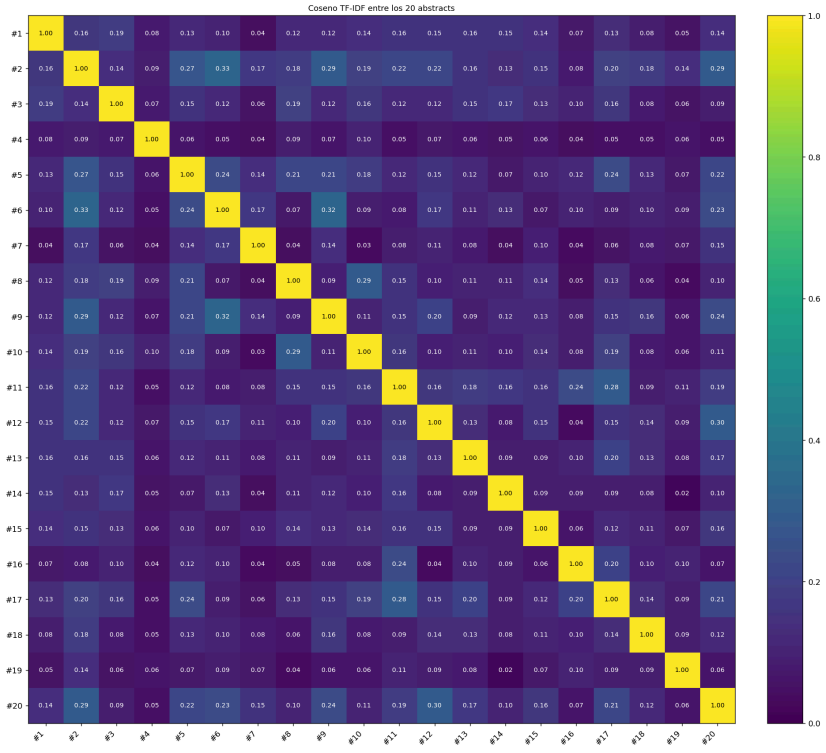


Figura 3. *Coseno TF-IDF entre los 20 abstracts: el caso de estudio es la celda más clara fuera de la diagonal (#2 con #6).*

Qué nos dicen estos datos.

1. **Cada familia mide algo distinto.** Levenshtein y Needleman-Wunsch coinciden mucho entre sí (Spearman 0,89), pero poco con TF-IDF (0,32 y 0,25): miran la forma de las letras, no el vocabulario. TF-IDF y Jaccard sí se parecen (0,57), porque ambos miran palabras compartidas. Los embeddings se acercan a TF-IDF de forma moderada (0,64 y 0,54).
2. **Levenshtein y Needleman-Wunsch discriminan poco, y dependen de la longitud.** Todos los pares caen en rangos estrechos (Levenshtein de 0,226 a 0,332; Needleman-Wunsch de 0,539 a 0,764). Medimos su relación con la razón de longitudes de los textos: la correlación de Spearman es -0,80 (Levenshtein) y -0,97 (Needleman-Wunsch), frente a -0,14 de TF-IDF. Es decir, ordenan los pares sobre todo según cuánto difieren en longitud, no según el tema. El signo negativo (más diferencia de longitud, más similitud normalizada) es coherente con un efecto de la normalización; no investigamos su mecanismo exacto.
3. **TF-IDF y Jaccard separan mejor.** TF-IDF va de 0,022 a 0,329. Ponderar por rareza hace que palabras poco comunes como aied (en 7 abstracts) pesen más que otras muy comunes como educate (en 18). Su límite es que no ven sinónimos: learner y student son términos distintos.

4. **Los embeddings dan valores altos para casi todos los pares** (GloVe de 0,838 a 0,964; fastText de 0,910 a 0,983). Lo comprobamos: el coseno de cada abstract con el vector medio de los 20 abstracts es de 0,958 (GloVe) y 0,978 (fastText) en promedio. O sea, todos los vectores comparten una dirección dominante, lo esperable cuando se promedian palabras de un mismo tema. Si se resta esa media (como diagnóstico, no en la aplicación), los cosenos entre pares pasan a abarcar de -0,48 a 0,45 en GloVe y de -0,57 a 0,62 en fastText: la información fina existe, pero el valor crudo la esconde. Por eso el par más parecido según TF-IDF ocupa solo el puesto 18 (GloVe) y 16 (fastText), y el par de contraste conserva valores de 0,879 y 0,916.
5. **El costo es muy distinto.** Levenshtein y Needleman-Wunsch cuestan unos 185 ms por par; TF-IDF, Jaccard y embeddings, entre 0,15 y 1,54 ms. Los embeddings son baratos de consultar, pero su costo real está en calcularlos, y por eso se hizo fuera del servidor.

Conclusión. Los valores no son comparables entre algoritmos; solo tiene sentido comparar el orden de los pares dentro de cada uno. Para encontrar artículos que comparten terminología conviene TF-IDF; para ver cercanía temática general, los embeddings aportan otra mirada; y las medidas de edición, aunque son el ejemplo clásico de programación dinámica, no sirven como medida de similitud temática en abstracts. Los modelos de oraciones tipo Transformer (script opcional generar_embeddings.py sbert) podrían comportarse distinto, pero no se ejecutaron en este trabajo y no afirmamos que mejoren.

5.8 Complejidad y tiempos

Tabla 31. Complejidad teórica y tiempo medido (promedio sobre los 190 pares)

Algoritmo	Tiempo	Espacio	Tiempo medido por par
Levenshtein	O(m·n)	O(mín(m,n)) con dos filas	184,7 ms
Needleman-Wunsch	O(m·n)	O(n) solo puntaje; O(m·n) con alineamiento	187,2 ms
Coseno TF-IDF	O(V) por par	O(V) por documento	0,17 ms
Jaccard	O(m+n) promedio	O(m+n)	1,54 ms
GloVe / fastText	O(d)	O(d) por documento	0,15 ms

Esto explica una decisión de la aplicación: se limita la comparación a 6 artículos como máximo (15 pares), porque Levenshtein y Needleman-Wunsch son los que dominan el tiempo total, y en el servidor gratuito de Render pueden tardar varios segundos.

5.9 Para qué sirve

- **Detectar textos casi duplicados o muy cercanos** (por ejemplo, versiones de un mismo trabajo): Jaccard y TF-IDF son buenos para eso.
- **Encontrar artículos relacionados** con uno dado, ordenándolos por similitud TF-IDF o por embeddings.
- **Entender qué mide cada técnica**: comparar los seis algoritmos sobre el mismo par enseña que parecido no significa una sola cosa.
- **Alimentar el clustering** del requerimiento 2, que usa justamente la similitud TF-IDF como base.

5.10 Cumplimiento del requerimiento 1

Tabla 32. Cumplimiento del Requerimiento 1

Exigencia	Evidencia	Estado
Selección dinámica de 2 o más artículos	Página /ui/similarity; comparacion.comparar valida 2 a 6.	Cumple
Cuatro algoritmos clásicos, desde cero	similitud_clasica.py; pruebas con casos conocidos (por ejemplo, Levenshtein(kitten, sitting) = 3).	Cumple
Dos modelos de IA con embeddings	GloVe y fastText; coseno y euclidiana propios en similitud_embeddings.py.	Cumple
Caso de estudio con dos abstracts concretos	Sección 5.6 (#2 y #6).	Cumple
Demostración matemática paso a paso	Matrices DP, vocabulario, TF, IDF, pesos, producto punto, normas, ángulo, intersección y unión, y embeddings.	Cumple
Análisis crítico clásicos frente a IA	Sección 5.7, con correlaciones y verificaciones medidas.	Cumple
Complejidad de cada algoritmo	Secciones 5.2 y 5.8.	Cumple

6. Requerimiento 2: clustering jerárquico

6.1 Qué pide el enunciado y cómo se cumple

El enunciado pide tres algoritmos de agrupamiento jerárquico implementados desde cero, que construyan un árbol (dendrograma) con la similitud entre los 20 abstracts. El flujo es: preprocesar el texto, calcular la similitud, aplicar el clustering y dibujar el dendrograma. Además exige evaluar cuál de los tres agrupa mejor con una métrica matemática interna y documentar las matrices, los pasos de fusión y la conclusión.

Tabla 33. Requerimiento 2: exigencias y cumplimiento

Lo que pide el enunciado	Cómo lo cumplimos	Dónde
Tres criterios de enlace desde cero	Single, Complete y Average Linkage (UPGMA), con listas y bucles.	src/clustering.py
Preprocesar, calcular similitud y agrupar	Texto limpio → vectores TF-IDF → matriz de distancias 1 − coseno → fusiones.	Secciones 6.2 y 6.3
Dendrograma de cada algoritmo	Se dibujan con matplotlib a partir de nuestro historial de fusiones (sin scipy).	Sección 6.6
Métrica interna que determine el mejor	Correlación cofenética (principal) y coeficiente de silueta (apoyo), ambas escritas a mano.	Sección 6.4
Documentar matriz de distancias, pasos de fusión y conclusión	Anexos B y C, sección 6.5 y sección 6.7.	Anexos y 6.7
Verificación opcional contra scipy	Solo en pruebas y en el script de evidencias, nunca en la aplicación.	Sección 6.8

6.2 Cómo funciona, paso a paso

1. **Vectores.** Cada uno de los 20 abstracts ya tiene su vector TF-IDF (capítulo 5).
2. **Matriz de distancias.** Para cada par de abstracts se calcula $d = 1 - coseno$. El resultado es una tabla de 20 × 20 simétrica, con ceros en la diagonal (cada texto está a distancia 0 de sí mismo). Está completa en el Anexo B (matriz_distancias).
3. **Empezar con 20 grupos.** Al principio cada artículo es su propio grupo.

4. **Fusionar.** Se buscan los dos grupos más cercanos y se unen en uno nuevo. Se anota la fusión: qué grupos se unieron, a qué distancia y cuántos artículos tiene el nuevo grupo (agrupar). Se repite 19 veces, hasta que queda un solo grupo.
5. **Dibujar.** El registro de las 19 fusiones —el *historial*— es todo lo que hace falta para dibujar el dendrograma (coordenadas_dendrograma y png_dendrograma).
6. **Evaluar.** Se miden la correlación cofenética y la silueta para decidir cuál de los tres criterios produce el árbol más coherente.

Es como armar equipos eligiendo siempre a los dos que más se parecen entre sí: primero se emparejan los más afines, luego esas parejas se juntan con otras parejas, y así hasta formar un solo grupo. El dendrograma es el registro gráfico de todas esas decisiones.

6.3 Los tres criterios de enlace

Cuando los grupos ya tienen varios artículos, hay que definir qué significa que dos grupos estén cerca. Los tres criterios responden de formas distintas. Sea $d(a, b)$ la distancia entre dos artículos, y A y B dos grupos:

Tabla 34. Los tres criterios de enlace

Criterio	Distancia entre los grupos A y B	Efecto práctico
Single (enlace simple)	$d(A, B) = \min d(a, b)$, con a en A y b en B	Basta con que **un** par de artículos esté cerca para unir los grupos. Tiende al «encadenamiento»: forma un grupo enorme que va absorbiendo a los demás.
Complete (enlace completo)	$d(A, B) = \max d(a, b)$	Los grupos se unen solo si **todos** sus artículos están razonablemente cerca. Forma grupos compactos y de tamaño parecido.
Average (enlace promedio, UPGMA)	$d(A, B) = (1 / \lvert A \rvert \lvert B \rvert) \cdot \Sigma\, d(a, b)$	Usa el **promedio** de todas las distancias entre pares. Es un punto medio entre los dos anteriores.

El código. La definición directa de cada criterio está en _distancia_clusters (src/clustering.py, línea 43); la repetición de fusiones, en agrupar (línea 68).

Función _distancia_clusters: una rama por criterio

```
def _distancia_clusters(D, miembros_a, miembros_b, criterio):
    if criterio == "single":
        mejor = None
        for i in miembros_a:
            for j in miembros_b:
                if mejor is None or D[i][j] < mejor:
                    mejor = D[i][j]
        return mejor
    if criterio == "complete":
        mejor = None
        for i in miembros_a:
            for j in miembros_b:
                if mejor is None or D[i][j] > mejor:
                    mejor = D[i][j]
        return mejor
    if criterio == "average":
        total = 0.0
        for i in miembros_a:
            for j in miembros_b:
                total += D[i][j]
        return total / (len(miembros_a) * len(miembros_b))
    raise ValueError("Criterio de enlace desconocido: %r (use single, complete o average)" % (criterio,))
```

Cada rama es la fórmula de la tabla escrita con bucles: single guarda la menor distancia encontrada, complete la mayor y average suma todas y divide entre |A|·|B|. Ahora el corazón del algoritmo:

Función agrupar: el ciclo de fusiones

```
def agrupar(D, criterio):
    n = len(D)
    if n < 2:
```

```
        raise ValueError("Se necesitan al menos 2 documentos para agrupar.")
    activos = ListaDinamica()          # ids de clusters activos
    miembros = {}                      # id cluster -> lista de documentos
    i = 0
    while i < n:
        activos.agregar(i)
        miembros[i] = [i]
        i += 1
    historial = []
    siguiente = n
    while activos.tamano() > 1:
        mejor_d, mejor_a, mejor_b = None, -1, -1
        p = 0
        while p < activos.tamano():
            q = p + 1
            while q < activos.tamano():
                a, b = activos[p], activos[q]
                d = _distancia_clusters(D, miembros[a], miembros[b], criterio)
                if mejor_d is None or d < mejor_d - 1e-15:
                    mejor_d, mejor_a, mejor_b = d, a, b
                q += 1
            p += 1
        nuevos = miembros[mejor_a] + miembros[mejor_b]
        historial.append({"a": mejor_a, "b": mejor_b, "distancia": mejor_d, "tamano": len(nuevos),
"nuevo": siguiente})
        miembros[siguiente] = nuevos
        nueva_lista = ListaDinamica()
        for c in activos:
            if c != mejor_a and c != mejor_b:
                nueva_lista.agregar(c)
        nueva_lista.agregar(siguiente)
        activos = nueva_lista
        siguiente += 1
    return historial
```

- activos es la lista de grupos vigentes; al empezar hay 20 (identificados del 0 al 19) y al terminar 1.
- Los dos bucles anidados p y q recorren todos los pares de grupos activos y se queda con el de menor distancia. Si hay empate se conserva el primero que se encontró, de modo que el resultado es siempre el mismo (determinista).
- Cada fusión se guarda como un diccionario {a, b, distancia, tamano, nuevo}: es una fila del historial. El grupo nuevo recibe un identificador consecutivo (20, 21, …).
- Los grupos fusionados salen de activos y entra el nuevo. Tras 19 vueltas queda uno solo.

Ejemplo pequeño para verlo con claridad. Tomemos cuatro puntos sobre una recta: 0, 1, 5 y 7, y como distancia la diferencia entre ellos. Los tres criterios hacen primero las mismas dos fusiones (0 con 1 a distancia 1, y 5 con 7 a distancia 2); la diferencia aparece al unir los dos grupos que quedan:

Tabla 35. Mismos datos, tres criterios: cambia la altura de la última fusión

Criterio	Fusión 1	Fusión 2	Fusión 3	Cómo sale la última
Single	1,00	2,00	4,00	mín(5, 7, 4, 6) = 4
Complete	1,00	2,00	7,00	máx(5, 7, 4, 6) = 7
Average	1,00	2,00	5,50	(5 + 7 + 4 + 6) / 4 = 5,5

Complejidad. En cada una de las 19 fusiones se revisan todos los pares de grupos y, para cada par, hasta |A|·|B| distancias. En total, el tiempo es **O(n³)** y el espacio **O(n²)** (la matriz). Con n = 20 es instantáneo. Existen versiones más rápidas que actualizan las distancias en lugar de recalcularlas, pero esta mantiene la definición de cada criterio a la vista, que es lo importante para entenderlo y explicarlo.

6.4 Métricas internas: cómo saber cuál agrupa mejor

Una métrica interna juzga la calidad de un agrupamiento usando solo los datos y el propio árbol, sin respuestas correctas externas. Usamos dos, implementadas a mano.

6.4.1 Correlación cofenética (la métrica principal)

La idea. Un dendrograma dice una distancia entre cada par de artículos: la altura a la que quedan unidos por primera vez (distancia cofenética). Si el árbol es fiel a los datos, esas alturas deberían parecerse a las distancias originales de la matriz. La correlación cofenética mide cuánto se parecen: es el coeficiente de correlación de Pearson entre las 190 distancias originales y las 190 distancias del árbol.

$$CCC = \Sigma\,(x_k - \bar{x})(y_k - \bar{y}) \,/\, \sqrt{(\,\Sigma\,(x_k - \bar{x})^2 \cdot \Sigma\,(y_k - \bar{y})^2\,)}$$

x_k: distancia original del par k; y_k: distancia cofenética del mismo par

Vale entre −1 y 1; **cuanto más cerca de 1, más fiel es el árbol a las distancias originales**. En el código, distancias_cofeneticas (línea 122) construye la tabla de alturas recorriendo el historial, pearson (línea 139) calcula la correlación y correlacion_cofenetica (línea 155) las une.

6.4.2 Coeficiente de silueta (la métrica de apoyo)

La idea. Se corta el árbol en *k* grupos y se le pregunta a cada artículo: «¿estás más cerca de tu propio grupo que de los demás?». Para el artículo *i*, sea *a* su distancia media a los demás de su grupo y *b* su distancia media al grupo vecino más cercano:

$$s(i) = (\,b - a\,) \,/\, máx(\,a,\, b\,)$$

s cerca de 1: bien agrupado; cerca de 0: en la frontera; negativo: probablemente en el grupo equivocado

La silueta del agrupamiento es el promedio de *s(i)* sobre todos los artículos (los grupos de un solo artículo aportan 0). Está en silueta (línea 203) y se evalúa para varios valores de *k* con siluetas_por_k (línea 243); cortar_en_k (línea 171) produce los grupos al deshacer las últimas fusiones.

6.4.3 De la fórmula al código

Tabla 36. Del concepto al código (Requerimiento 2)

Expresión	Función	Archivo y línea
d = 1 − coseno(u, v)	matriz_distancias	clustering.py L25
mín / máx / promedio de d(a, b)	_distancia_clusters	clustering.py L43
Altura de fusión de cada par (distancia cofenética)	distancias_cofeneticas	clustering.py L122
Correlación de Pearson	pearson	clustering.py L139
CCC entre distancias originales y cofenéticas	correlacion_cofenetica	clustering.py L155
s(i) = (b − a)/máx(a, b)	silueta	clustering.py L203
Elegir el mejor criterio (mayor CCC; desempate por silueta)	comparar_criterios	clustering.py L266

Expresión	Función	Archivo y línea
Posición (x, y) de cada línea del dendrograma	coordenadas_dendrograma	clustering.py L295

6.5 Matriz de distancias y pasos de fusión del corpus

La matriz de distancias completa de los 20 abstracts está en el Anexo B, en dos bloques de diez columnas. Un valor cercano a 0 indica abstracts con vocabulario muy parecido; uno cercano a 1, abstracts casi sin palabras en común. El valor más bajo de toda la matriz es 0,6713 (el par #2 y #6, el mismo del caso de estudio del capítulo 5) y el más alto, 0,9785.

El historial de fusiones de cada criterio está completo en el Anexo C. Aquí se muestran, a manera de lectura guiada, las primeras fusiones de Average Linkage. Cada fila dice: en el paso *k*, se unieron el grupo A y el grupo B a la distancia indicada, formando un grupo nuevo con ese tamaño. #n es el artículo *n*; Cn, un grupo creado por una fusión.

Tabla 37. Primeras 8 fusiones de Average Linkage (de 19)

Paso	Une	con	Distancia	Tamaño	Grupo nuevo
1	#2	#6	0,6713	2	C20
2	#9	C20	0,6932	3	C21
3	#12	#20	0,7017	2	C22
4	#8	#10	0,7117	2	C23
5	#11	#17	0,7163	2	C24
6	#5	C21	0,7580	4	C25
7	#16	C24	0,7809	3	C26
8	C22	C25	0,7820	6	C27

Las primeras fusiones unen artículos muy cercanos; las últimas, grupos enteros a distancias grandes. Las alturas de las fusiones finales, por criterio, son: Single 0,901, Complete 0,978 y Average 0,938; el rango de alturas va de 0,671 a 0,978 en los tres.

6.6 Los dendrogramas

Los dendrogramas se **dibujan a partir de nuestro propio historial**, sin usar scipy. La función coordenadas_dendrograma (línea 295) convierte las 19 fusiones en segmentos de línea, y png_dendrograma (visualizacion.py) los traza con matplotlib. El procedimiento es:

1. Se calcula el orden de las hojas (orden_hojas): se recorre el árbol desde la última fusión, de modo que los grupos que se unieron queden uno junto al otro y las líneas no se crucen.
2. Cada hoja (artículo) se coloca en una posición horizontal 0, 1, 2… y a altura 0.
3. Para cada fusión se dibujan dos líneas verticales que suben desde cada grupo hasta la altura de la fusión, y una línea horizontal que las une. El grupo nuevo se coloca en el punto medio horizontal de los dos que se unieron: $x = (x_a + x_b) / 2$, a la altura de la fusión.

La línea roja punteada de cada figura es un corte en $k = 4$ grupos, y las etiquetas están coloreadas según el grupo al que pertenecen. Cuanto más abajo se unen dos artículos, más parecidos son.

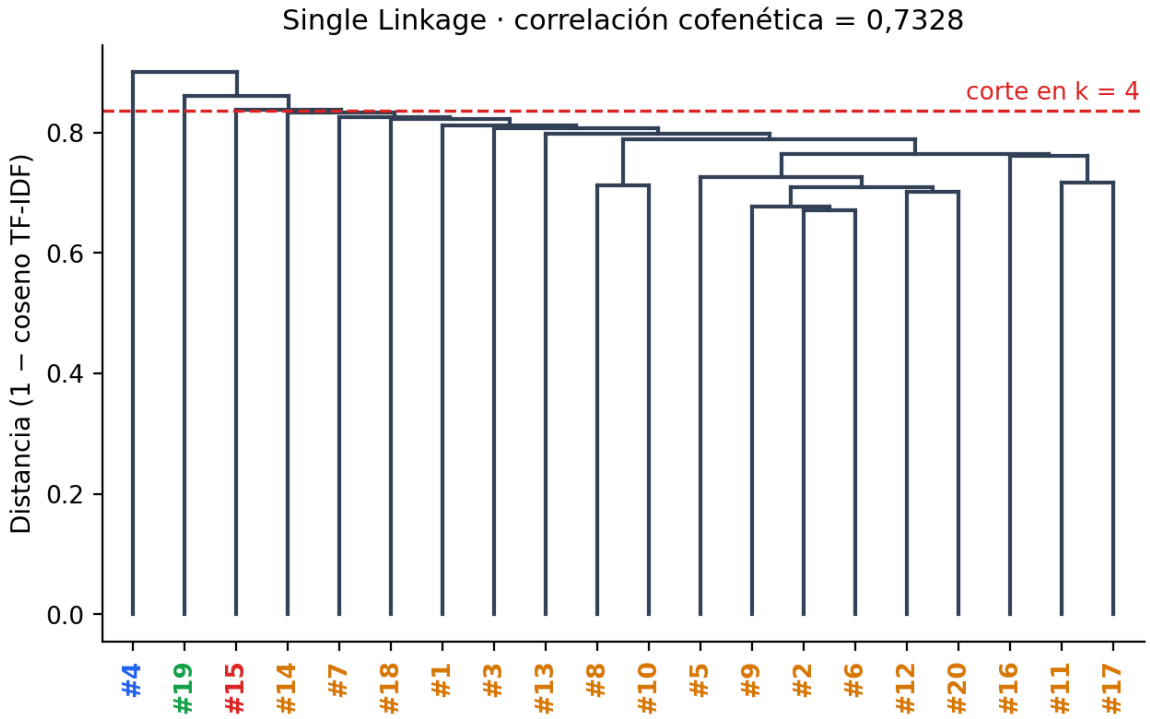


Figura 4. *Dendrograma de Single Linkage: el árbol «en escalera» típico del encadenamiento.*

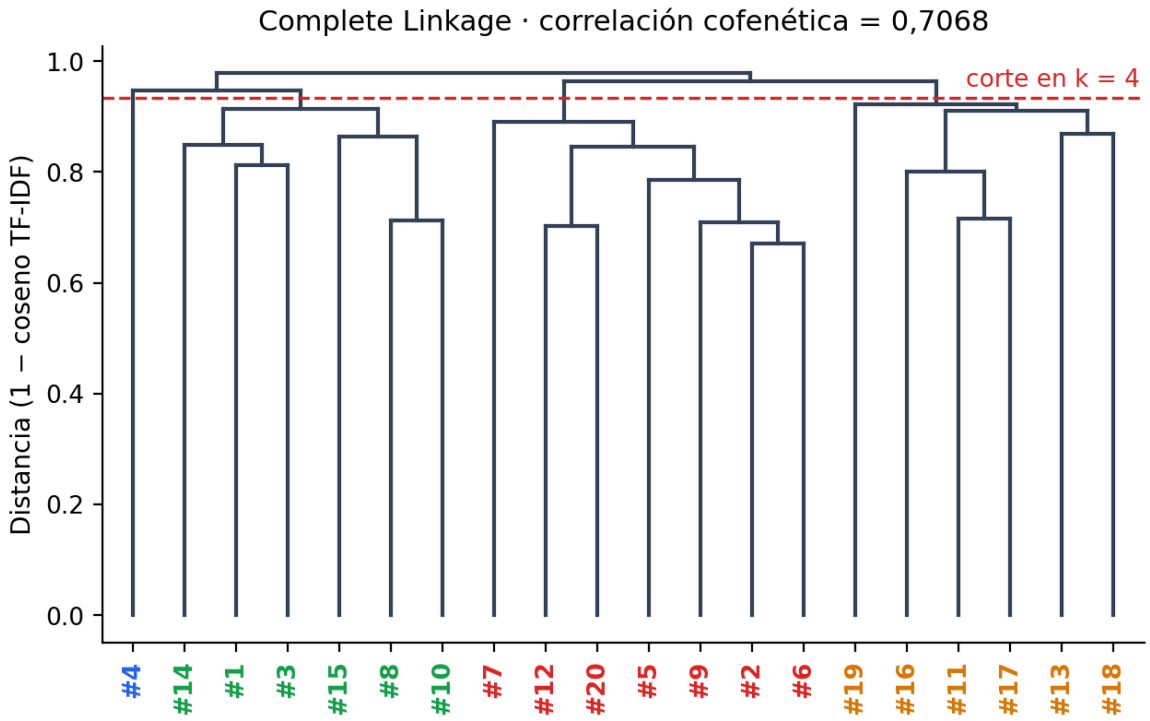


Figura 5. *Dendrograma de Complete Linkage: grupos más equilibrados y alturas altas.*

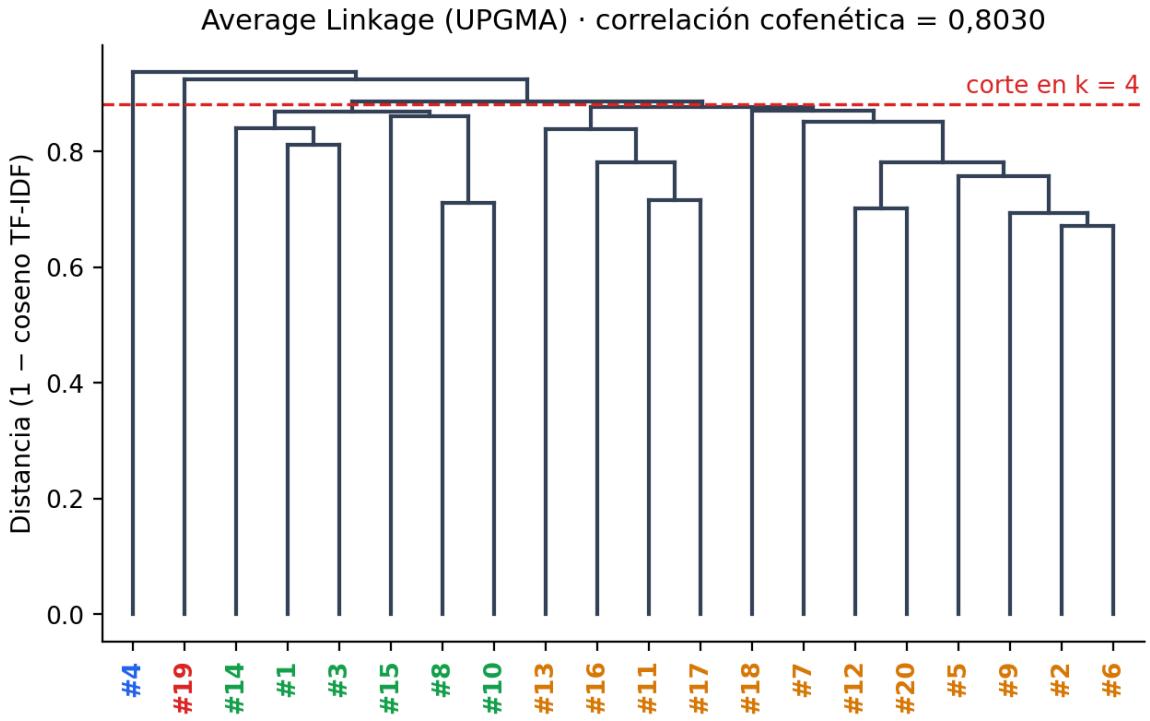


Figura 6. *Dendrograma de Average Linkage: el criterio con mayor correlación cofenética.*

6.7 Evaluación y conclusión

Tabla 38. Evaluación de los tres criterios

Criterio	Correlación cofenética	Mejor k (silueta)	Silueta en ese k	Silueta con k = 4	Puesto por CCC
Single Linkage (enlace simple)	0,7328	2	0,0693	0,0170	2
Complete Linkage (enlace completo)	0,7068	7	0,0639	0,0592	3
Average Linkage (UPGMA)	**0,8030**	**2**	**0,0693**	**0,0424**	**1**

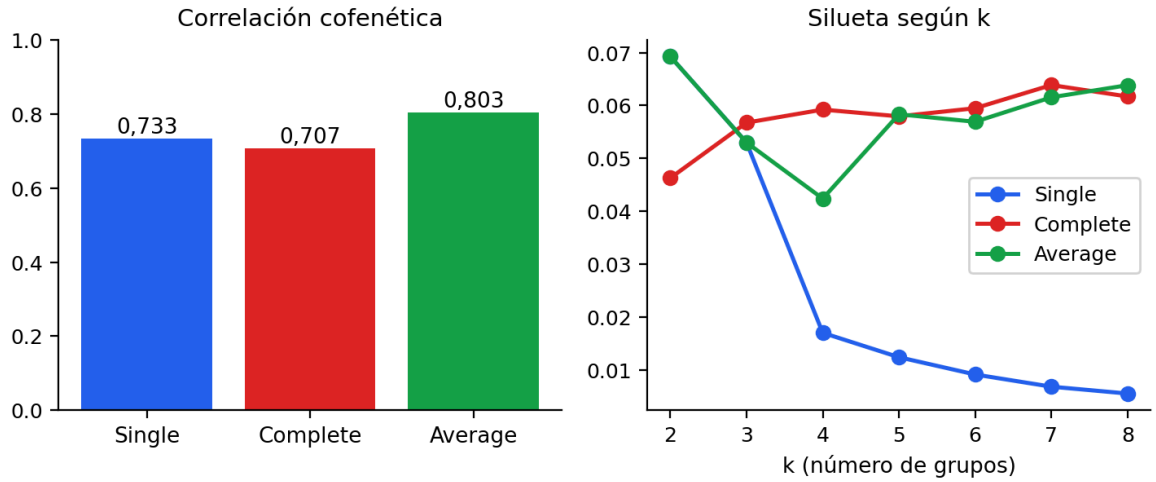


Figura 7. *Correlación cofenética (izquierda) y silueta según el número de grupos (derecha).*

Los grupos con k = 4, según cada criterio, ayudan a ver la diferencia de comportamiento:

Tabla 39. Agrupamiento con k = 4

Criterio	Grupos (artículos #)
Single Linkage (enlace simple)	{4} | {15} | {19} | {1, 2, 3, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 16, 17, 18, 20}
Complete Linkage (enlace completo)	{4} | {2, 5, 6, 7, 9, 12, 20} | {1, 3, 8, 10, 14, 15} | {11, 13, 16, 17, 18, 19}
Average Linkage (UPGMA)	{4} | {19} | {1, 3, 8, 10, 14, 15} | {2, 5, 6, 7, 9, 11, 12, 13, 16, 17, 18, 20}

Conclusión. El criterio más coherente es Average Linkage (UPGMA), porque tiene la mayor correlación cofenética (0,8030, frente a 0,7328 de Single y 0,7068 de Complete): su árbol reproduce mejor las distancias originales entre abstracts. Single muestra el encadenamiento esperado: con $k = 4$ deja tres artículos aislados y un grupo gigante de 17, con la silueta más baja (0,0170), por lo que informa poco. Complete forma grupos más equilibrados (tamaños 1, 7, 6, 6) y tiene la silueta más alta con $k = 4$ (0,0592), pero su árbol distorsiona más las distancias. Average combina razonablemente ambas virtudes.

Una salvedad. La silueta es baja en los tres criterios (el máximo es 0,069). Esto no es un error: indica que los 20 abstracts, al ser del mismo tema y de poca extensión, forman una nube sin grupos claramente separados. Por eso la decisión se apoya en la correlación cofenética y la silueta queda como información complementaria.

Como lectura temática, en el agrupamiento de Complete aparece un grupo con los artículos sobre IA generativa y evaluación (#11, #13, #16, #17, #18 y #19) y otro con las revisiones generales sobre IA en educación (#2, #5, #6, #7, #9, #12 y #20), lo cual es coherente con los títulos de esos artículos.

6.8 Verificación contra scipy

Para comprobar que nuestra implementación es correcta, la comparamos con la de scipy únicamente en las pruebas (tests/test_clustering.py) y en el script de evidencias; la aplicación nunca importa scipy. Sobre la matriz real de los 20 abstracts, la diferencia máxima entre las alturas de fusión de ambas implementaciones y entre sus correlaciones cofenéticas es prácticamente cero:

Tabla 40. Contraste con scipy (diferencias nulas o del orden de 10^{-16}: solo redondeo)

Criterio	Diferencia máxima de alturas	CCC propia	CCC de scipy	Diferencia de CCC
Single Linkage (enlace simple)	0.0e+00	0,732756	0,732756	0.0e+00
Complete Linkage (enlace completo)	0.0e+00	0,706790	0,706790	4.4e-16
Average Linkage (UPGMA)	3.3e-16	0,802978	0,802978	2.2e-16

6.9 Para qué sirve

- **Descubrir familias temáticas** en una colección de artículos sin leerlos todos: por ejemplo, separar los que tratan de evaluación con IA generativa de las revisiones generales.
- **Organizar una revisión de literatura**: el árbol sugiere cómo agrupar los artículos en secciones.

- **Detectar artículos atípicos**: los que se unen al final, muy arriba del árbol, como los #4 y #19 con $k = 4$.
- **Elegir el número de grupos**: el control de k de la aplicación permite probar cuántos grupos resultan razonables.

6.10 Cumplimiento del requerimiento 2

Tabla 41. Cumplimiento del Requerimiento 2

Exigencia	Evidencia	Estado
Tres criterios de enlace desde cero	agrupar y _distancia_clusters; verificados contra scipy en pruebas.	Cumple
Preprocesar, similitud, clustering y dendrograma	Capítulo 4, sección 6.2 y visualizacion.py.	Cumple
Dendrograma de cada algoritmo	Tres figuras de la sección 6.6, dibujadas desde el historial propio.	Cumple
Métrica interna para decidir el mejor	Correlación cofenética y silueta (sección 6.4).	Cumple
Documentar matriz, pasos de fusión y conclusión	Anexos B y C; sección 6.7.	Cumple

7. Requerimiento 3: búsqueda de patrones

7.1 Qué pide el enunciado y cómo se cumple

El enunciado pide que el usuario escriba un texto (el patrón) y la aplicación localice todas sus apariciones exactas en los abstracts de los 20 artículos. Debe hacerse con tres algoritmos clásicos de coincidencia de cadenas, implementados desde cero y sin funciones nativas como find() o indexOf(): Knuth-Morris-Pratt, Boyer-Moore y Rabin-Karp. También exige documentar un caso de estudio con las estructuras auxiliares de cada algoritmo y un análisis crítico de su complejidad frente a los tiempos reales.

Tabla 42. Requerimiento 3: exigencias y cumplimiento

Lo que pide el enunciado	Cómo lo cumplimos	Dónde
Ingresar un patrón y ver todas las ocurrencias	La página «Patrones» muestra documento, posición y fragmento resaltado.	static/modules/patterns/, src/api.py
KMP, Boyer-Moore y Rabin-Karp desde cero	Con índices y bucles; sin find, index, in, replace ni expresiones regulares.	src/patrones.py
Los tres devuelven las mismas posiciones	Comprobado con 2.500 casos aleatorios y sobre el corpus.	tests/test_patrones.py
Caso de estudio: patrón y párrafo reales	Patrón «perspectives» en el abstract #9.	Sección 7.7
Estructuras auxiliares paso a paso	Tabla LPS, tablas de Boyer-Moore, hash inicial y rodante con colisiones, y gráficas.	Sección 7.7
Big O frente a rendimiento observado	Tablas y gráficas de tiempos reales, escalabilidad y casos límite.	Sección 7.8

7.2 Cómo funciona la búsqueda en la aplicación

Cuando se pulsa «Buscar», ocurre lo siguiente (GET /api/patterns/search, función api_patterns_search en src/api.py):

1. **Validación** (patrones.validar_entradas): el patrón no puede estar vacío ni pasar de 200 caracteres, y el texto debe ser una cadena. Si algo falla, se lanza ErrorPatron y la API responde con un error 400 que la página muestra en rojo.

2. **Mayúsculas.** Por defecto no se distinguen: texto y patrón se pasan a minúsculas con a_minusculas, que conserva la longitud. Así las posiciones encontradas sirven para el texto original y el resaltado cae donde debe.
3. **Búsqueda en cada abstract** (patrones.buscar_en_corpus): para cada uno de los 20 abstracts se ejecuta el algoritmo elegido (patrones.ejecutar), que mide por separado el tiempo de preprocesamiento y el de búsqueda y devuelve las posiciones (base 0), incluidas las superpuestas (en aaaa, el patrón aa aparece en 0, 1 y 2).
4. **Fragmentos.** Para cada posición se recorta un fragmento de hasta 60 caracteres antes y después (patrones.fragmento) y se divide en tres partes —antes, coincidencia, después— para que el navegador resalte solo la del medio.
5. **Comparación (opcional).** Con la opción **Los tres**, comparar_algoritmos corre los tres algoritmos y verifica que devuelvan exactamente las mismas posiciones; la página muestra una etiqueta verde si es así.

Si el patrón es más largo que el texto, simplemente no hay coincidencias (no es un error). Los tres algoritmos dividen su trabajo en dos etapas, que la aplicación mide por separado: preprocesamiento (preparar tablas a partir del patrón, una vez) y búsqueda (recorrer el texto).

Tabla 43. Complejidad teórica (n: largo del texto; m: largo del patrón; s: tamaño del alfabeto)

Algoritmo	Preprocesamiento	Búsqueda: mejor caso	Búsqueda: promedio	Búsqueda: peor caso	Espacio
KMP	O(m)	O(n)	O(n)	O(n)	O(m)
Boyer-Moore	O(m + s)	O(n/m)	≈ O(n)	O(n·m)	O(m + s)
Rabin-Karp	O(m)	O(n + m)	O(n + m)	O(n·m)	O(1)

7.3 Knuth-Morris-Pratt (KMP)

La idea. El método ingenuo prueba el patrón en cada posición del texto y, cada vez que falla, empieza de nuevo desde el siguiente carácter, volviendo a mirar letras que ya había visto. KMP evita ese desperdicio: si ya se comprobó que varios caracteres coinciden y luego falla uno, esa información dice cuánto del patrón se puede aprovechar sin retroceder jamás en el texto.

La tabla LPS. La clave es una tabla que se calcula solo con el patrón. LPS[i] es la longitud del prefijo más largo del patrón que también es sufijo de lo que llevamos leído hasta la posición *i* (sin contar el patrón entero). Ejemplo con el patrón ABABCABAB, calculado con la función real:

Tabla 44. Tabla LPS de ABABCABAB

i	0	1	2	3	4	5	6	7	8
patrón	A	B	A	B	C	A	B	A	B
LPS	0	0	1	2	0	1	2	3	4

Por ejemplo, LPS[8] = 4: en ABABCABAB, el trozo ABAB del comienzo coincide con el trozo ABAB del final. Eso significa que si, después de leer los 9 caracteres, hay que continuar, no hace falta empezar de cero: ya se tienen 4 caracteres emparejados.

La búsqueda. Se mantienen dos posiciones: i en el texto y j en el patrón (cuántos caracteres ya coinciden). Si texto[i] == patron[j], ambas avanzan. Si no coincide y j > 0, **solo se retrocede j** (a LPS[j − 1]); i nunca baja. Si j llega a *m*, se encontró una ocurrencia y se sigue buscando con j = LPS[m − 1], lo que permite detectar coincidencias superpuestas.

patrones.py: kmp_tabla_lps (línea 40) y kmp_buscar (línea 59)

```
def kmp_tabla_lps(patron):
    m = len(patron)
    lps = [0] * m
    largo = 0
    i = 1
    while i < m:
        if patron[i] == patron[largo]:
            largo += 1
            lps[i] = largo
            i += 1
        elif largo > 0:
            largo = lps[largo - 1]      # retroceder en el patron, NUNCA en el texto
        else:
            lps[i] = 0
            i += 1
```

```
    return lps


def kmp_buscar(texto, patron, lps=None, traza=None):
    n, m = len(texto), len(patron)
    if lps is None:
        lps = kmp_tabla_lps(patron)
    posiciones = []
    comparaciones = 0
    i = 0   # indice en el texto
    j = 0   # caracteres del patron ya emparejados
    while i < n:
        comparaciones += 1
        if texto[i] == patron[j]:
            if traza is not None:
                traza.append({"i": i, "j": j, "ok": True, "s": i - j})
            i += 1
            j += 1
            if j == m:
                posiciones.append(i - m)
                j = lps[j - 1]          # permite ocurrencias superpuestas
        else:
            if traza is not None:
                traza.append({"i": i, "j": j, "ok": False, "s": i - j})
            if j > 0:
                j = lps[j - 1]
            else:
                i += 1
    return {"posiciones": posiciones, "comparaciones": comparaciones}
```

- En kmp_tabla_lps, largo es la longitud del prefijo-sufijo vigente. Si el carácter nuevo coincide con patron[largo], el borde crece; si no, se retrocede dentro del patrón usando la misma tabla (largo = lps[largo - 1]).
- En kmp_buscar, la línea j = lps[j - 1] es el «salto inteligente». El índice i solo se incrementa, nunca se reduce: ese es el corazón del algoritmo.

- traza (opcional) registra cada comparación para dibujar la gráfica de iteraciones del caso de estudio.

Complejidad. Calcular la tabla cuesta **O(m)** y la búsqueda **O(n)** en cualquier caso: i solo avanza, y cada retroceso de j se compensa con avances previos, de modo que en total hay a lo sumo 2n comparaciones.

7.4 Boyer-Moore

La idea. Boyer-Moore hace algo que parece contraintuitivo: compara el patrón de derecha a izquierda. La ventaja es que, cuando falla, el carácter del texto que causó la falla permite saltar varias posiciones de un solo golpe. En el mejor caso, mira solo uno de cada *m* caracteres del texto. Usa dos reglas y toma la que dé el salto mayor.

Regla del carácter malo. Si en la posición *j* del patrón falla la comparación con un carácter c del texto, se desplaza el patrón para alinear c con su última aparición dentro del patrón. Si c no está en el patrón, se salta el patrón completo. La tabla ultima[c] guarda esa última posición (o −1 si no existe):

$$desplazamiento\ por\ carácter\ malo = j - ultima[c]$$

Regla del sufijo bueno. Si ya coincidió un trozo final del patrón (el «sufijo bueno») y luego falla un carácter, se desplaza el patrón hasta la siguiente aparición de ese sufijo dentro del propio patrón, o hasta el mayor prefijo del patrón que coincida con el final de ese sufijo. Se construye con una tabla de «bordes», es decir, con la misma idea que la tabla LPS de KMP pero mirando el patrón desde el final. desplazamiento[k] es el avance cuando la falla ocurre en la posición $k - 1$ habiendo coincidido ya el sufijo desde la posición *k*.

$$avance = máx(\ j - ultima[c]\ ,\ \ desplazamiento_sufijo_bueno[j + 1]\ )$$

Se toma el mayor de los dos avances: ninguno puede hacer saltar una ocurrencia real

Ejemplo completo con el patrón example y el texto here is a simple example. Primero, las dos tablas que se calculan solo con el patrón:

Tabla 45. Tabla del carácter malo (un carácter que no aparece vale −1)

Carácter	a	e	l	m	p	x
última posición	2	6	5	3	4	1

Tabla 46. Tabla del sufijo bueno

k	0	1	2	3	4	5	6	7
desplazamiento[k]	6	6	6	6	6	6	6	1

Tabla 47. Recorrido de Boyer-Moore sobre here is a simple example: ocurrencia en la posición 17 con solo 15 comparaciones

Alineamiento s	Falla en j	Carácter del texto	Avance (malo)	Avance (bueno)	Regla usada	Avance
0	6	s	7	1	carácter malo	7
7	6	p	2	1	carácter malo	2
9	2	i	3	6	sufijo bueno	6
15	6	p	2	1	carácter malo	2
17	—	—	—	6	coincidencia completa (sufijo bueno)	6

Observe el primer paso: se comparó e (última letra del patrón) con s, que no está en example; al ser el carácter malo, el patrón saltó 7 posiciones de una vez. En total el algoritmo hizo 15 comparaciones sobre un texto de 24 caracteres.

patrones.py: bm_tabla_caracter_malo (línea 90) y bm_buscar (línea 133)

```
def bm_tabla_caracter_malo(patron):
    tabla = TablaHashSimple(64)
    i = 0
    while i < len(patron):
        tabla.poner(patron[i], i)
        i += 1
    return tabla

def bm_buscar(texto, patron, tablas=None, traza=None):
    n, m = len(texto), len(patron)
    if tablas is None:
        tablas = bm_preprocesar(patron)
    malo, bueno = tablas["malo"], tablas["bueno"]
    posiciones = []
    comparaciones = 0
```

```
    s = 0
    while s <= n - m:
        j = m - 1
        while j >= 0:
            comparaciones += 1
            if patron[j] != texto[s + j]:
                break
            j -= 1
        if j < 0:
            posiciones.append(s)
            desplazamiento = bueno[0]
            if traza is not None:
                traza.append({"s": s, "j": -1, "comparados": m, "desplazamiento": desplazamiento,
                              "regla": "coincidencia completa (sufijo bueno)", "malo": None, "bueno": bueno[0]})
            s += desplazamiento
        else:
            por_malo = j - malo.obtener(texto[s + j], -1)
            por_bueno = bueno[j + 1]
            desplazamiento = mayor_de_dos(por_malo, por_bueno)
            if traza is not None:
                traza.append({"s": s, "j": j, "comparados": m - j, "desplazamiento": desplazamiento,
                              "regla": "caracter malo" if por_malo >= por_bueno else "sufijo bueno",
                              "malo": por_malo, "bueno": por_bueno, "caracter_texto": texto[s + j]})
            s += desplazamiento
    return {"posiciones": posiciones, "comparaciones": comparaciones}
```

- El bucle interior (while j >= 0) compara de derecha a izquierda y se detiene en la primera diferencia.
- Si j < 0, todo el patrón coincidió: se registra la posición s y se avanza bueno[0], el desplazamiento que respeta posibles ocurrencias superpuestas.
- Si hubo una falla, por_malo = j − malo.obtener(texto[s + j], −1) es la regla del carácter malo y por_bueno = bueno[j + 1] es la del sufijo bueno; el avance es el mayor (mayor_de_dos).
- La tabla del sufijo bueno se construye en bm_tabla_sufijo_bueno (línea 100) mediante dos pasadas sobre el patrón: la primera calcula los bordes de cada sufijo (borde) y rellena los avances cuando el sufijo reaparece; la segunda completa los casos restantes con el borde más largo del patrón completo.

Complejidad. Preprocesar cuesta **O(m + s)**, con *s* el tamaño del alfabeto. La búsqueda, en el **mejor caso**, es O(n/m): cuando casi todos los caracteres del texto no están en el patrón y los saltos son de longitud *m*. En el caso promedio es cercano a O(n). En el peor caso, O(n·m): cuando hay muchísimas ocurrencias (por ejemplo, un patrón aaa…a sobre un texto aaa…a) y hay que comparar el patrón completo en cada posición. La versión que garantiza O(n) siempre usa además la regla de Galil, que no implementamos.

7.5 Rabin-Karp

La idea. En vez de comparar letra por letra, Rabin-Karp compara números. Convierte el patrón en un número (su *hash*) y hace lo mismo con cada ventana del texto del mismo tamaño. Comparar dos números es mucho más rápido que comparar *m* letras. Solo cuando los números coinciden se verifica carácter a carácter, porque dos cadenas distintas podrían dar el mismo número: eso se llama colisión.

El hash. Cada carácter se convierte en su código numérico (por ejemplo, p vale 112) y la cadena se trata como un número en base $B = 256$, con la aritmética hecha módulo un número primo q para que no crezca sin control:

$$h = (c_0 \cdot B^{(m-1)} + c_1 \cdot B^{(m-2)} + \ldots + c_{(m-1)}) \bmod q$$

c_i: código del carácter i; B = 256; q = 1.000.000.007

Se calcula con el método de Horner ($h = (h \cdot B + \text{código}) \bmod q$, letra por letra) en la función rk_hash. La parte inteligente es el hash rodante: para pasar a la ventana siguiente no se recalcula todo, sino que se quita el carácter que sale y se agrega el que entra:

$$h(s+1) = ((h(s) - T[s] \cdot B^{(m-1)}) \cdot B + T[s+m]) \bmod q$$

T[s]: carácter que sale; T[s+m]: carácter que entra; B^(m−1) mod q se calcula una vez

Ejemplo con un módulo pequeño ($q = 13$) para que se vean las colisiones. Patrón 415, texto 3141592653. El hash del patrón es 0.

Tabla 48. Rabin-Karp con q = 13: en la ventana 6 el hash coincide, pero el texto 265 no es el patrón

Ventana s	Texto de la ventana	Hash	¿Igual al del patrón?	Resultado
0	314	9	no	—
1	141	1	no	—
2	415	0	sí	ocurrencia
3	159	5	no	—
4	592	7	no	—

Ventana s	Texto de la ventana	Hash	¿Igual al del patrón?	Resultado
5	926	12	no	—
6	265	0	sí	colisión
7	653	1	no	—

En la ventana 2 el hash coincide y al verificar se confirma la ocurrencia. En la ventana 6 también coincide, pero al comparar las letras se ve que son distintas: es una colisión, y se descarta. Por eso el algoritmo siempre verifica cuando los hashes son iguales; así el resultado es correcto aunque haya colisiones. Con el módulo grande que usa la aplicación casi no ocurren.

patrones.py: rk_hash (línea 174) y rk_buscar (línea 190)

```
def rk_hash(cadena, base, modulo):
    h = 0
    i = 0
    while i < len(cadena):
        h = (h * base + ord(cadena[i])) % modulo
        i += 1
    return h


def rk_buscar(texto, patron, prep=None, traza=None):
    n, m = len(texto), len(patron)
    if prep is None:
        prep = rk_preprocesar(patron)
    base, modulo, hp, pot = prep["base"], prep["modulo"], prep["hash_patron"], prep["potencia"]
    posiciones = []
    comparaciones = 0
    colisiones = 0
    if m > n:
        return {"posiciones": posiciones, "comparaciones": 0, "colisiones": 0}
    h = rk_hash(texto[:m], base, modulo)
    s = 0
    while s <= n - m:
        if h == hp:
```

```
            # el hash coincide: puede ser coincidencia real o COLISION -> se verifica caracter por caracter
            k = 0
            while k < m:
                comparaciones += 1
                if texto[s + k] != patron[k]:
                    break
                k += 1
            real = (k == m)
            if real:
                posiciones.append(s)
            else:
                colisiones += 1
            if traza is not None:
                traza.append({"s": s, "hash": h, "hash_patron": hp, "igual": True, "real": real,
                              "colision": not real})
        elif traza is not None:
            traza.append({"s": s, "hash": h, "hash_patron": hp, "igual": False, "real": False, "colision":
False})
        if s < n - m:
            # hash rodante: quitar texto[s], desplazar una posicion y agregar texto[s+m]
            h = ((h - ord(texto[s]) * pot) * base + ord(texto[s + m])) % modulo
        s += 1
    return {"posiciones": posiciones, "comparaciones": comparaciones, "colisiones": colisiones}
```

- rk_hash es la fórmula del hash escrita con Horner: en cada vuelta multiplica por la base y suma el código del carácter, siempre módulo *q*.
- En rk_buscar, la línea h = ((h - ord(texto[s]) * pot) * base + ord(texto[s + m])) % modulo es literalmente la fórmula del hash rodante. pot es B^(m−1) mod q, que rk_preprocesar calcula una sola vez con exponenciación binaria (potencia_modular).
- Cuando h == hp se verifica carácter por carácter; si no coincide, se cuenta una colisión. Por eso hay tan pocas comparaciones de caracteres.

Complejidad. Preprocesar cuesta O(m). La búsqueda recorre las $n - m + 1$ ventanas haciendo O(1) por ventana, así que en promedio es **O(n + m)**. El peor caso es O(n·m): si muchas ventanas tienen el mismo

hash que el patrón (muchas colisiones, o muchas ocurrencias reales) hay que verificar *m* caracteres en cada una.

7.6 Relación entre las fórmulas y el código

Tabla 49. De la teoría al código (Requerimiento 3)

Concepto matemático	Función	Archivo y línea
Longitud del mayor prefijo que también es sufijo (LPS)	kmp_tabla_lps	patrones.py L40
Recorrido de KMP sin retroceder en el texto	kmp_buscar	patrones.py L59
ultima[c]: última posición de cada carácter	bm_tabla_caracter_malo	patrones.py L90
Desplazamientos del sufijo bueno y bordes	bm_tabla_sufijo_bueno	patrones.py L100
avance = máx(carácter malo, sufijo bueno)	bm_buscar	patrones.py L133
Hash polinomial de Horner módulo q	rk_hash	patrones.py L174
B^(m−1) mod q (exponenciación binaria)	potencia_modular	estructuras_datos.py L589
Hash rodante y verificación de colisiones	rk_buscar	patrones.py L190
Medir preprocesamiento y búsqueda por separado	ejecutar	patrones.py L238
Verificar que los tres algoritmos coinciden	comparar_algoritmos	patrones.py L305
Mediciones de escalabilidad y casos límite	medir_escalabilidad, medir_casos_sinteticos	benchmark_patrones.py L39, L52

7.7 Caso de estudio: el patrón «perspectives» en el abstract #9

Por qué este patrón. El enunciado sugiere un patrón concreto y un párrafo real de un abstract. Entre los términos del corpus buscamos uno cuya tabla LPS no fuera trivial (casi todas las palabras la tienen llena de ceros) y que apareciera varias veces. perspectives cumple ambas cosas: su tabla tiene un borde pe, y aparece 5 veces en el corpus, 2 de ellas en el abstract #9. El texto es ese abstract completo (1245 caracteres), titulado *AI technologies for education: Recent research & future directions*.

From unique educational perspectives, this article reports a comprehensive review of selected empirical studies on artificial intelligence in education (AIEd) published in 1993 – 2020, as collected in the Web of Sciences database and selected AIEd-specialized journals. A total of 40 empirical studies met all selection criteria, and were fully reviewed using multiple methods, including selected bibliometrics, content analysis and categorical meta-trends analysis. This article reports the current state of AIEd research, highlights selected AIEd technologies and applications, reviews their proven and potential benefits for education, bridges the gaps between AI technological innovations and their educational applications, and generates practical examples and inspirations for both technological experts that create AIEd technologies and educators who spearhead AI innovations in education. It also provides rich discussions on practical implications and future research directions from multiple perspectives. The advancement of AIEd calls for critical initiatives to address AI ethics and privacy concerns, and requires interdisciplinary and transdisciplinary collaborations in large-scaled, longitudinal research and development efforts.

Los tres algoritmos encuentran las ocurrencias en las posiciones 24 y 1002 (base 0).

7.7.1 KMP: tabla LPS

Tabla 50. Tabla LPS de «perspectives»

i	0	1	2	3	4	5	6	7	8	9	10	11
patrón	p	e	r	s	p	e	c	t	i	v	e	s
LPS	0	0	0	0	1	2	0	0	0	0	0	0

LPS[4] = 1 porque la segunda p (posición 4) coincide con la primera letra del patrón; LPS[5] = 2 porque pe (posiciones 4 y 5) coincide con las dos primeras letras. Todas las demás entradas son 0. Con esa tabla, KMP hizo 1273 comparaciones sobre un texto de 1245 caracteres.

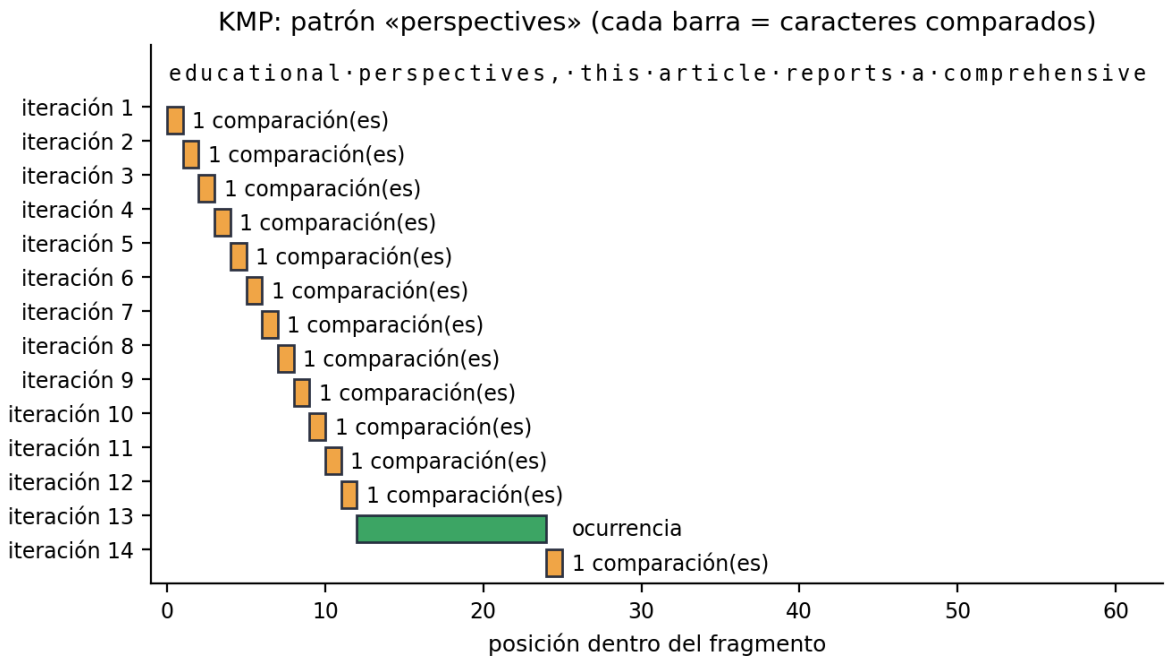


Figura 8. *KMP sobre un fragmento del abstract #9: cada barra muestra los caracteres comparados en una posición; casi siempre falla en la primera letra («p»), por eso las barras son tan cortas.*

7.7.2 Boyer-Moore: tablas heurísticas

Tabla 51. Tabla del carácter malo de «perspectives» (s vale 11 porque su última aparición es la posición 11)

Carácter	c	e	i	p	r	s	t	v
última posición	6	10	8	4	2	11	7	9

Tabla 52. Tabla del sufijo bueno y bordes de perspectives

k	0	1	2	3	4	5	6	7	8	9	10	11	12
desplazamiento[k]	12	12	12	12	12	12	12	12	12	12	12	8	1
borde[k]	12	12	12	11	12	12	12	12	12	12	12	12	13

Primeras iteraciones reales (cada fila es un alineamiento del patrón sobre el texto):

Tabla 53. Primeras 8 iteraciones de Boyer-Moore

Alineamientos	Falla en j	Carácter del texto	Avance (malo)	Avance (bueno)	Regla usada	Avance
0	11		12	1	carácter malo	12
12	11		12	1	carácter malo	12
24	—	—	—	12	coincidencia completa (sufijo bueno)	12
36	11	c	5	1	carácter malo	5
41	11	e	1	1	carácter malo	1
42	11	p	7	1	carácter malo	7
49	11		12	1	carácter malo	12
61	11	v	2	1	carácter malo	2

Boyer-Moore hizo 135 alineamientos y apenas 165 comparaciones de caracteres: salta la mayor parte del texto. Compárese con las 1273 de KMP.

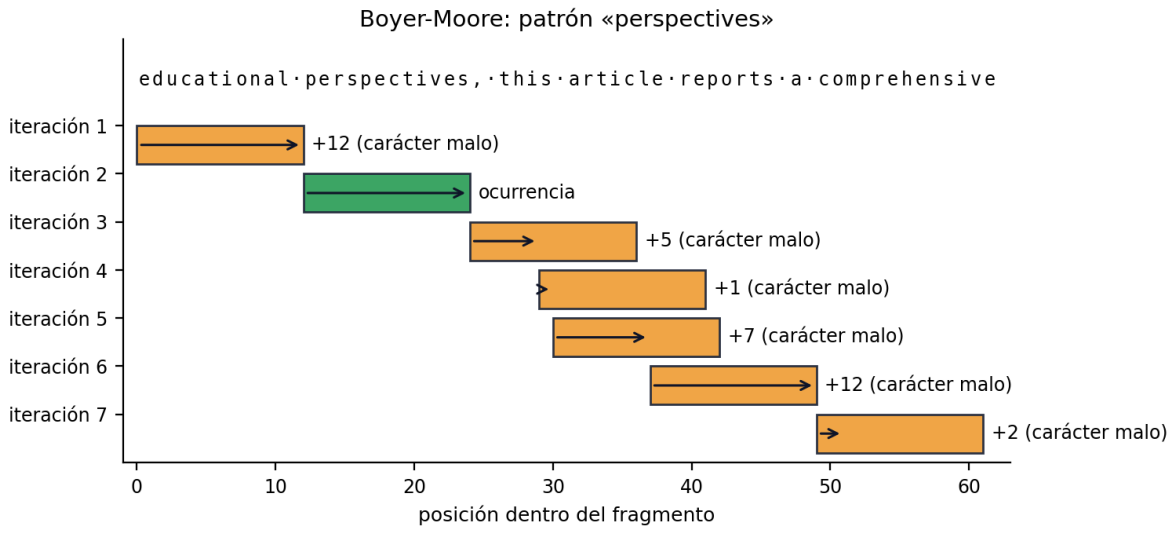


Figura 9. *Boyer-Moore sobre un fragmento del abstract #9: cada barra es un alineamiento del patrón y cada flecha, el desplazamiento hasta el siguiente intento. Verde: ocurrencia.*

7.7.3 Rabin-Karp: cálculo numérico del hash

Se usa base $B = 256$ y módulo $q = 1.000.000.007$. Los códigos de los caracteres del patrón son: p = 112, e = 101, r = 114, s = 115, p = 112, e = 101, c = 99, t = 116, i = 105, v = 118, e = 101, s = 115.

1. **Hash del patrón** (Horner, letra por letra): 791.898.702.
2. **Potencia B^(m−1) mod q** (con m = 12): 140.130.951.
3. **Hash de la primera ventana** (from unique , calculado con Horner): 543.860.792.
4. **Hash rodante.** Las ventanas siguientes se obtienen con la fórmula del hash rodante. Verificación numérica contra el cálculo directo:

Tabla 54. Hash rodante frente a hash directo: dan el mismo número

Ventana s	Hash de la ventana	Sale	Entra	Hash rodante de s+1	Hash directo de s+1	¿Coinciden?
0	543.860.792	f	e	128.994.981	128.994.981	sí
1	128.994.981	r	d	441.069.651	441.069.651	sí
2	441.069.651	o	u	952.754.247	952.754.247	sí

Colisiones. El módulo q controla cuántas colisiones aparecen. Sobre este mismo texto (1.234 ventanas) se contaron las colisiones reales para distintos módulos:

Tabla 55. Colisiones de Rabin-Karp según el módulo

Módulo q	Hash del patrón	Colisiones	Ocurrencias reales
11	1	98	2
31	8	41	2
101	63	13	2
211	183	7	2
1.009	215	0	2
10.007	8.341	0	2
1.000.000.007	791.898.702	0	2

Con $q = 101$, el hash del patrón es 63 y hay 13 colisiones; algunos ejemplos, en los que la ventana tiene el mismo hash que el patrón pero otras letras:

Tabla 56. Ejemplos de colisiones con q = 101

Posición s	Texto de la ventana	Hash	Hash del patrón	Resultado
166	ed in 1993 –	63	63	colisión: se descarta
175	3 – 2020, as	63	63	colisión: se descarta
183	, as collect	63	63	colisión: se descarta
215	ences databa	63	63	colisión: se descarta
293	studies met	63	63	colisión: se descarta

En todos los casos el resultado final fue correcto (posiciones 24 y 1002), porque cada coincidencia de hash se verifica. Con el módulo 1.000.000.007 no hubo ninguna colisión; Rabin-Karp hizo apenas 24 comparaciones de caracteres (solo las de verificación), pero recorrió 1.234 ventanas.

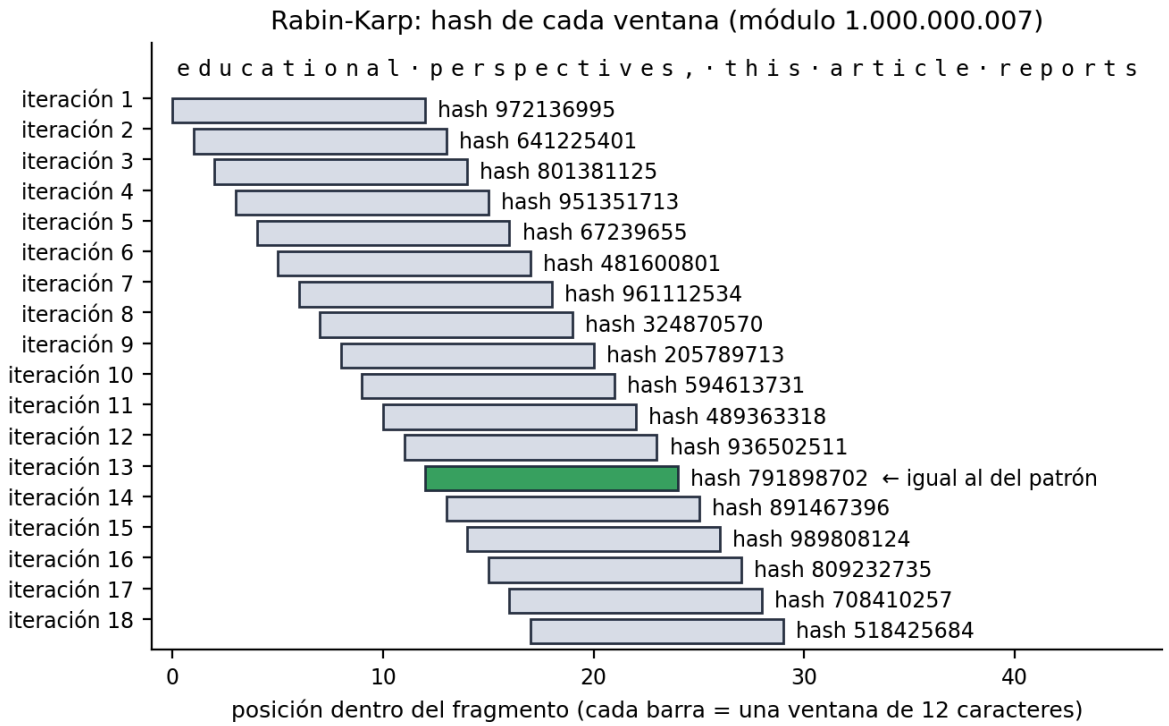


Figura 10. *Rabin-Karp sobre un fragmento: cada barra es una ventana de 12 caracteres con su hash; en verde, la que coincide con el hash del patrón y se confirma como ocurrencia.*

7.7.4 Resumen del caso

Tabla 57. Los tres algoritmos sobre el mismo texto

Algoritmo	Posiciones	Alineamientos o ventanas	Comparaciones de caracteres	Colisiones
KMP	24 y 1002	1.273	1.273	—
Boyer-Moore	24 y 1002	135	165	—
Rabin-Karp	24 y 1002	1.234	24	0

7.8 Análisis crítico: Big O frente a tiempos reales

La teoría dice cómo *debería* crecer el trabajo; los tiempos medidos dicen cómo crece *en realidad*. Contrastamos ambos con tres mediciones, todas generadas ejecutando el código (benchmark_patrones.py).

7.8.1 Sobre el corpus completo

Se concatenaron los 20 abstracts (28.097 caracteres) y se buscaron cinco patrones de distinto tamaño. Cada tiempo es la mediana de 15 corridas, y el preprocesamiento se mide aparte de la búsqueda.

Tabla 58. Tiempos reales sobre el corpus (mediana de 15 corridas)

Patrón	m	Algoritmo	Preproc. (ms)	Búsqueda (ms)	Comparaciones	Ocurrencias
«ai»	2	KMP	0,0013	2,65	29.954	170
		Boyer-Moore	0,0098	6,45	15.656	
		Rabin-Karp	0,0042	5,41	340	
«learning»	8	KMP	0,0019	2,28	29.026	40
		Boyer-Moore	0,0127	2,08	4.962	
		Rabin-Karp	0,0066	7,57	320	
«perspectives»	12	KMP	0,0023	2,24	28.598	5
		Boyer-Moore	0,0153	1,53	3.375	
		Rabin-Karp	0,0069	7,07	60	

Patrón	m	Algoritmo	Preproc. (ms)	Búsqueda (ms)	Comparaciones	Ocurrencias
«artificial intelligence in education»	36	KMP	0,0035	2,36	30.090	6
		Boyer-Moore	0,0249	0,86	2.604	
		Rabin-Karp	0,0088	7,82	216	
«systematic review of the literature»	35	KMP	0,0034	2,49	29.782	0
		Boyer-Moore	0,0248	0,76	1.992	
		Rabin-Karp	0,0061	7,10	0	

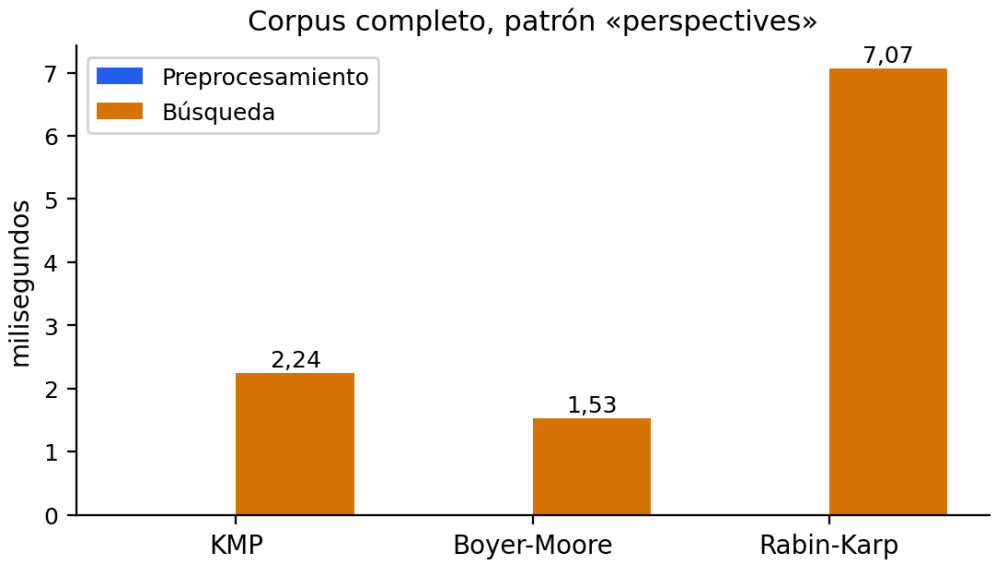


Figura 11. *Preprocesamiento y búsqueda por algoritmo para el patrón «perspectives» sobre todo el corpus.*

7.8.2 Escalabilidad

Repetimos el corpus 1, 2, 4… 32 veces para ver cómo crece el tiempo con el tamaño del texto. Si la búsqueda es lineal (O(n)), duplicar *n* debería duplicar el tiempo.

Tabla 59. Tiempo de búsqueda al aumentar el tamaño del texto

Repeticiones	n (caracteres)	KMP (ms)	Boyer-Moore (ms)	Rabin-Karp (ms)
1	28.097	2,2	1,3	6,9
2	56.195	4,3	2,7	13,8
4	112.391	8,6	5,3	27,7
8	224.783	17,3	10,7	55,5
16	449.567	34,4	21,3	110,8
32	899.135	69,0	42,5	220,5

Al multiplicar *n* por 32, el tiempo se multiplicó por 32,0 (KMP), 32,2 (Boyer-Moore) y 31,8 (Rabin-Karp): crecimiento lineal, tal como predice O(n).

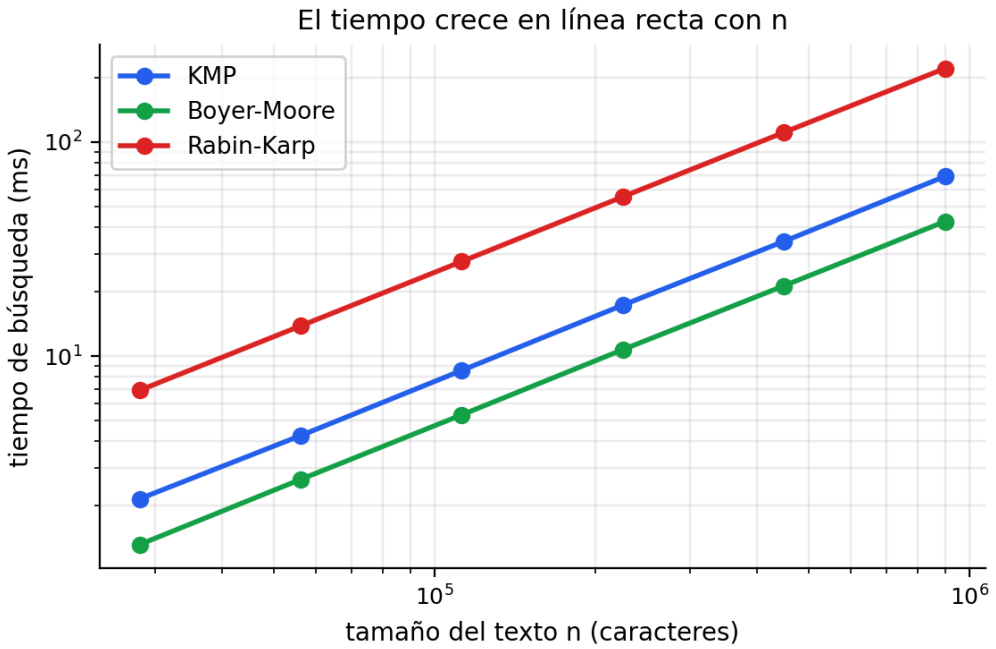


Figura 12. *Tiempo de búsqueda frente al tamaño del texto, en escala logarítmica: las tres curvas son rectas paralelas.*

7.8.3 Casos límite

Para observar el mejor y el peor caso hacen falta textos artificiales (n = 20.000, m = 40):

Tabla 60. Mejor y peor caso con textos sintéticos

Caso	KMP	Boyer-Moore	Rabin-Karp
Sin ocurrencias, alfabeto distinto (mejor caso de Boyer-Moore)	1,5 ms / 20.000 comp.	0,2 ms / 500 comp.	4,8 ms / 0 comp.
Casi coincide siempre: patron a^(m-1)b sobre a^n	3,0 ms / 39.961 comp.	8,2 ms / 19.961 comp.	4,9 ms / 0 comp.
Todas las posiciones coinciden: patron a^m sobre a^n (peor caso de BM y RK)	2,4 ms / 20.000 comp.	58,6 ms / 798.440 comp.	64,7 ms / 798.440 comp.

7.8.4 Lectura de los resultados

1. **El mejor caso de Boyer-Moore se observa.** Sin ocurrencias y con un alfabeto distinto, hizo 500 comparaciones, exactamente n/m = 500, frente a 20.000 de KMP. En el corpus real, con m = 12, hizo 3.375 frente a 28.598 de KMP: salta de a bloques.
2. **El peor caso O(n·m) también.** Con el patrón a…a sobre un texto a…a, Boyer-Moore y Rabin-Karp hicieron 798.440 comparaciones, que es exactamente (n − m + 1) × m = 798.440; KMP se mantuvo en 20.000 (lineal).
3. **Preprocesamiento frente a búsqueda.** El preprocesamiento es despreciable (0,0023 ms en KMP, 0,0153 en Boyer-Moore y 0,0069 en Rabin-Karp) frente a la búsqueda (2,24, 1,53 y 7,07 ms). Boyer-Moore paga el preprocesamiento más caro (dos tablas), pero lo recupera de sobra al saltar.
4. **Menos comparaciones no siempre es más rápido.** Rabin-Karp hace muy pocas comparaciones de caracteres (60), pero es el más lento en Python (7,07 ms frente a 2,24 ms de KMP) porque en cada ventana hace multiplicaciones y operaciones módulo. El número de comparaciones no es el único costo: también cuenta lo que cuesta cada operación. Y con patrones muy cortos (ai, m = 2), Boyer-Moore pierde su ventaja porque los saltos son mínimos.
5. **Los tiempos absolutos dependen del equipo** (aquí Python 3.12.3 en 1 CPU) y del lenguaje; lo que se contrasta con la teoría es la forma de crecimiento y el número de comparaciones, que no dependen de la máquina.

7.9 Para qué sirve

- **Localizar un término o una frase** en toda la colección de resúmenes, con el artículo, la posición y el contexto.
- **Contar la frecuencia** de un concepto (por ejemplo, cuántas veces aparece «assessment» y en qué artículos).
- **Comprobar citas o expresiones exactas** sin depender de un buscador externo.
- **Elegir el algoritmo adecuado según el caso:** KMP cuando importan las garantías, Boyer-Moore con patrones largos en textos grandes, Rabin-Karp para buscar varios patrones a la vez (idea para trabajo futuro).

7.10 Cumplimiento del requerimiento 3

Tabla 61. Cumplimiento del Requerimiento 3

Exigencia	Evidencia	Estado
Ingresar un patrón y mostrar ocurrencias exactas	Página /ui/patterns: documento, posición y fragmento resaltado.	Cumple
KMP, Boyer-Moore y Rabin-Karp desde cero, sin funciones nativas	patrones.py; sin find, index, in, replace ni regex.	Cumple
Las mismas posiciones en los tres	2.500 casos aleatorios y 8 patrones sobre el corpus (test_patrones.py).	Cumple
Caso con patrón y párrafo reales	«perspectives» en el abstract #9 (sección 7.7).	Cumple
Tabla LPS, tablas de Boyer-Moore, hash rodante, colisiones y gráficas	Sección 7.7: todas las tablas y las tres figuras de iteraciones.	Cumple
Big O teórica frente a tiempos reales	Sección 7.8: corpus, escalabilidad y casos límite.	Cumple

8. Requerimiento 4: despliegue y documentación

8.1 Qué pide el enunciado y cómo se cumple

El cuarto requerimiento pide que el proyecto esté desplegado y soportado con documentación técnica de cada requerimiento: un documento de diseño con la arquitectura, la explicación técnica con detalles de implementación, y la declaración explícita del uso de IA generativa como apoyo. También exige que no se usen funciones de alto nivel que implementen directamente los algoritmos.

Tabla 62. Requerimiento 4: exigencias y cumplimiento

Exigencia	Cómo se cumple	Dónde
Proyecto desplegado	Aplicación en Render desplegada desde GitHub, con enlace público: https://bibliometria-algoritmos-gp0j.onrender.com	Sección 8.3
Documento de diseño con arquitectura	Capítulo 3: diagramas de componentes y de flujo, decisiones de diseño.	Capítulo 3
Explicación técnica con detalles de implementación por requerimiento	Capítulos 5, 6 y 7, y la guía del código del capítulo 9.	Caps. 5 a 7 y 9
Casos de estudio, resultados, conclusiones y limitaciones	Secciones 5.6, 6.7 y 7.7; capítulo 10.	Caps. 5 a 7 y 10
Declaración de uso de IA generativa	Capítulo 12.	Capítulo 12
Sin funciones de alto nivel que implementen los algoritmos	Todo en src/ con listas y bucles; pruebas contra referencias solo en tests/.	Capítulo 9

8.2 La aplicación web y su API

La aplicación tiene cinco páginas, una por tema, que comparten el mismo estilo (static/styles.css) y la misma lógica de interfaz (static/app.js). Cada página llama a la API de Flask, que devuelve datos en JSON o gráficas en PNG.

Tabla 63. Páginas de la aplicación

Página	Ruta	Qué permite hacer
Corpus	/ui/corpus	Ver los 20 artículos, su calidad de extracción, el abstract y el preprocesamiento paso a paso; probar el preprocesamiento con un texto propio.
1. Similitud	/ui/similarity	Elegir de 2 a 6 artículos, compararlos con los seis algoritmos, ver el cálculo detallado y un mapa de calor.
2. Clustering	/ui/clustering	Ver las métricas de los tres criterios, el dendrograma de cada uno, cortar el árbol en k grupos y ver el historial de fusiones.
3. Patrones	/ui/patterns	Buscar un patrón con uno o los tres algoritmos, ver coincidencias resaltadas, las gráficas de iteraciones y medir tiempos.
Documentación	/ui/docs	Ver las fichas de complejidad de cada algoritmo y los endpoints disponibles.

Tabla 64. Endpoints de la API

Endpoint	Parámetros	Qué devuelve
GET /health	—	Estado del servicio: artículos cargados, tamaño del vocabulario y modelos de embeddings.
GET /api/corpus, /api/corpus/<id>, /api/corpus/calidad	id (1 a 20)	Lista de artículos; un abstract con su preprocesamiento; el reporte de calidad.
POST /api/preprocesar	JSON texto, stemming	Pasos del preprocesamiento de un texto libre (hasta 20.000 caracteres).
POST /api/similarity	JSON ids (2 a 6)	Similitud de cada par con los seis algoritmos y las matrices.
GET /api/similarity/detalle	a, b	Cálculo intermedio de TF-IDF y Jaccard de un par.
GET /plot/similarity.png	ids, algoritmo	Mapa de calor de similitud (PNG).

Endpoint	Parámetros	Qué devuelve
GET /api/clustering/comparar, /matriz, /grupos	criterio, k	Métricas e historial de los tres criterios; matriz de distancias; grupos al cortar en k.
GET /plot/dendrogram.png, /plot/clustering_metricas.png	criterio, k	Dendrograma y gráfica de métricas (PNG).
GET /api/patterns/search	patron, algoritmo, distinguir_mayusculas	Ocurrencias por artículo, tiempos y comparación de algoritmos.
GET /api/patterns/benchmark, /traza	patron, repeticiones, doc	Tiempos (corpus, escalabilidad, sintéticos); primeras iteraciones de un algoritmo.
GET /plot/patterns.png	patron, algoritmo, doc	Gráfica de iteraciones y desplazamientos (PNG).
GET /api/algorithm-docs	—	Fichas de complejidad de cada algoritmo.
GET /report/<formato>	docx, pdf	Descarga del documento correspondiente.

Manejo de errores. Todas las entradas inválidas se responden con un JSON {"detail": "…"} y el código adecuado: 400 para entradas inválidas (patrón vacío o de más de 200 caracteres, menos de 2 artículos, criterio desconocido), 404 para recursos que no existen (un artículo fuera de 1 a 20, una ruta inexistente) y 500 para errores inesperados. La interfaz muestra esos mensajes en rojo, en vez de fallar en silencio.

Caché. El servidor guarda en memoria (hasta 64 entradas) los resultados más costosos: el clustering, los dendrogramas, los mapas de calor, el cálculo detallado de un par y las mediciones de tiempo. Como los cálculos son deterministas, repetir una consulta responde de inmediato.

8.3 Despliegue en Render

La aplicación se despliega en Render, un servicio de alojamiento con plan gratuito, a partir del repositorio de GitHub (https://github.com/Marp2/bibliometria). El proceso es sencillo gracias a que la aplicación necesita muy poco para arrancar:

1. **Código en GitHub.** Se sube el proyecto al repositorio. Los PDFs y los modelos pesados de embeddings no viajan (están en .gitignore); sí viajan los JSON de data/processed/, que son pequeños.

2. **Servicio web en Render.** Se crea un *Web Service* conectado al repositorio, con lenguaje Python 3, comando de construcción pip install -r requirements.txt y plan *Free*. También se puede usar el archivo render.yaml, que describe lo mismo.
3. **Arranque.** Render ejecuta el comando del Procfile: gunicorn src.api:app --bind 0.0.0.0:$PORT --workers 1 --timeout 120. Es un solo proceso (ahorra memoria) con hasta 120 segundos por petición.
4. **Verificación.** Se abre /health; debe responder {"status": "ok", "articulos": 20, …}. Cada nuevo git push a la rama main vuelve a desplegar el servicio automáticamente.

Procfile, render.yaml y requirements.txt (archivos reales del repositorio)

```
web: gunicorn src.api:app --bind 0.0.0.0:$PORT --workers 1 --timeout 120


services:
  - type: web
    name: bibliometria-algoritmos
    runtime: python
    plan: free
    buildCommand: pip install -r requirements.txt
    startCommand: gunicorn src.api:app --bind 0.0.0.0:$PORT --workers 1 --timeout 120
    healthCheckPath: /health
    envVars:
      - key: PYTHON_VERSION
        value: 3.12.3

# requirements.txt
Flask
matplotlib
gunicorn
```

Las rutas de datos se pueden cambiar con las variables de entorno BIBLIO_DATA_DIR y BIBLIO_REPORTS_DIR (src/paths.py); si la carpeta indicada no se puede escribir, se usa una carpeta temporal del sistema. Limitaciones del plan gratuito: el servicio se duerme tras unos 15 minutos sin visitas (la primera consulta posterior tarda entre 30 y 60 segundos) y tiene poca capacidad de cálculo, por lo que comparar 6 artículos a la vez puede tardar varios segundos.

8.4 Documentación entregada

Tabla 65. Documentación del proyecto

Documento	Contenido	Ubicación
Documento técnico final	Este documento: diseño, implementación, casos de estudio, resultados y declaración de IA.	docs/DOCUMENTO_TECNICO_FINAL.*
Documentación técnica	Por componente: qué hace, dónde está, funciones clave y un ejemplo manual.	docs/DOCUMENTACION_TECNICA.md
README de cada módulo	Propósito, archivos, entradas, salidas y evidencias de cada módulo.	modules/<módulo>/README.md
README principal	Cómo ejecutar, probar, regenerar datos y desplegar.	README.md
Evidencias	Números reales que respaldan el documento, y las figuras.	docs/evidencias/, docs/figuras/

8.5 Pruebas y calidad

El proyecto incluye 94 pruebas automáticas con pytest (python -m pytest -q). No se limitan a comprobar que el código no falla: verifican resultados conocidos, comparan contra referencias independientes y ejercitan los casos de error que pide el enunciado.

Tabla 66. Pruebas automáticas por archivo (94 en total)

Archivo de pruebas	Pruebas	Qué verifican
tests/test_estructuras.py	8	Lista, pila, tabla hash (con redimensión), conjunto, merge sort estable, potencia modular, promedio y mediana.
tests/test_preprocesamiento.py	7	Cada paso de la limpieza, las reglas del stemming, el pipeline completo y los textos vacíos.
tests/test_similitud.py	14	Casos conocidos (Levenshtein(kitten, sitting) = 3; Needleman-Wunsch de GATTACA y GCATGCU = 0), TF-IDF calculado a mano, coseno, Jaccard, embeddings normalizados.
tests/test_clustering.py	11	Alturas de fusión de los tres criterios calculadas a mano, cofenética, silueta y **comparación contra scipy** sobre datos aleatorios.

Archivo de pruebas	Pruebas	Qué verifican
tests/test_patrones.py	29	Tabla LPS, tabla del sufijo bueno contra su definición, ocurrencias superpuestas, 2.500 casos aleatorios donde los tres algoritmos coinciden con una referencia independiente, mayúsculas, unicode, colisiones, patrón vacío o más largo que el texto.
tests/test_corpus_api.py	17	Corpus de 20 artículos completos, reporte de calidad, artículo inexistente, todos los endpoints y códigos de error.
tests/test_extraccion.py	8	Localización del abstract, unión de guiones, correcciones documentadas, detección de problemas y PDF sin abstract.

Sobre las pruebas. Las pruebas usan funciones prohibidas en la lógica como str.find() y scipy únicamente como referencia para comprobar que nuestros algoritmos dan la misma respuesta. Ese uso está permitido por el enunciado y es la forma estándar de validar una implementación propia.

Casos de error contemplados (los que pide el enunciado): PDF sin abstract (el script lo registra como error en el reporte de calidad), entradas vacías (patrón vacío, texto vacío, selección de menos de 2 artículos), patrón más largo que el texto (devuelve 0 ocurrencias sin fallar) y artículo inexistente (error 404 con el rango válido).

8.6 Uso de IA generativa

La declaración completa está en el capítulo 12. En resumen: la IA se usó como apoyo en código, pruebas y redacción, y las cifras del documento salen de ejecutar el código del repositorio.

8.7 Cumplimiento del requerimiento 4

Tabla 67. Cumplimiento del Requerimiento 4

Exigencia	Evidencia	Estado
Aplicación desplegada con enlace público	https://bibliometria-algoritmos-gp0j.onrender.com	Cumple
Documento de diseño con arquitectura	Capítulo 3	Cumple

Exigencia	Evidencia	Estado
Explicación técnica por requerimiento	Capítulos 5 a 7 y 9	Cumple
Declaración de IA generativa	Capítulo 12	Cumple
Algoritmos sin funciones de alto nivel	Código en src/; verificación en tests/	Cumple

9. Guía del código: carpeta por carpeta, archivo por archivo

Este capítulo es el mapa para recorrer el repositorio. Para cada carpeta se indica dónde está cada cosa, qué hace y con qué propósito; para cada archivo con lógica, una tabla con sus funciones. Los números de línea corresponden a la versión entregada. La regla general es sencilla: la lógica está en src/, las tareas previas en scripts/, los datos en data/, la interfaz en static/ y la verificación en tests/.

9.1 Mapa general del repositorio

Tabla 68. Mapa del repositorio

Ruta	Qué contiene	Propósito
/ (raíz)	api.py, Procfile, render.yaml, requirements*.txt, README.md, .gitignore	Arrancar la aplicación, desplegarla e instalar dependencias.
src/	14 archivos de Python (2.825 líneas)	Toda la lógica: estructuras, preprocesamiento, algoritmos, API y gráficas.
scripts/	extraer_corpus.py, generar_embeddings.py, generar_evidencias.py, generar_documentos.py y auxiliares	Preparar datos fuera del servidor y generar evidencias y documentos.
data/processed/	5 archivos JSON (corpus, texto limpio, calidad y 2 de embeddings)	Datos que lee la aplicación. Son el único «puente» entre el trabajo previo y el servidor.
data/ (otros)	metadata_verificada.json, referencia_java/	Títulos y autores verificados a mano; extracción independiente para validar.
static/	styles.css, app.js y modules/<página>/index.html (5 páginas)	Interfaz web.
modules/	Un README.md por módulo	Propósito, archivos, entradas, salidas y evidencias de cada parte.
tests/	7 archivos de pruebas	Verificar el comportamiento y los casos de error.

Ruta	Qué contiene	Propósito
docs/	Documentos, figuras/ y evidencias/	Documentación y soporte de los resultados.

9.2 Carpeta raíz

Tabla 69. Archivos de la carpeta raíz

Archivo	Qué hace	Por qué existe
api.py	Importa la aplicación Flask de src/api.py y la arranca en el puerto 8000 con python api.py.	Punto de entrada para trabajar en local.
Procfile	Una línea con el comando de gunicorn.	Indica a Render cómo arrancar la aplicación.
render.yaml	Describe el servicio: plan gratuito, comando de construcción, comando de arranque y chequeo de salud en /health.	Permite crear el servicio en Render con un clic («Blueprint»).
requirements.txt	Tres dependencias: Flask, matplotlib y gunicorn.	Lo mínimo para que la aplicación corra en el servidor.
requirements-dev.txt	Añade pypdf, pytest y scipy.	Lo necesario para extraer PDFs, correr pruebas y verificar contra scipy.
.gitignore	Excluye los PDFs, los modelos de embeddings, entornos virtuales y cachés.	Mantener el repositorio liviano.

9.3 Carpeta src/: la lógica del proyecto

Cada archivo de src/ tiene una responsabilidad. Se presentan en el orden en que conviene leerlos: de las piezas básicas hacia las que dependen de ellas.

9.3.1 src/paths.py (57 líneas): rutas y carpetas

Propósito: que ningún otro archivo tenga rutas escritas a mano. Define dónde están los datos, los documentos y la interfaz, permite cambiarlos con variables de entorno y, si una carpeta no se puede

escribir (algo habitual en servidores como Render), cae a una carpeta temporal. Lo usan: corpus.py, similitud_embeddings.py y api.py.

Tabla 70. Funciones de paths.py

Función (línea)	Qué hace	Para qué sirve / quién la usa
get_configured_processed_dir L14	Devuelve la carpeta de datos procesados: la de la variable BIBLIO_DATA_DIR si existe, o data/processed.	Permite mover los datos sin tocar el código.
get_configured_reports_dir L20	Igual, para la carpeta de reportes (BIBLIO_REPORTS_DIR).	Configuración por entorno.
get_fallback_processed_dir / get_fallback_reports_dir L25	Devuelven una carpeta temporal del sistema operativo.	Plan B cuando la carpeta preferida no es escribible.
ensure_writable_dir L33	Prueba escribir un archivo en la carpeta preferida; si falla, usa la de respaldo.	Evita que la aplicación falle en Render.
get_runtime_reports_dir L47	Combina las dos anteriores para los reportes.	Carpeta final de reportes.
find_data_file L51	Busca un archivo primero en la carpeta configurada y luego en data/processed del repositorio.	Lo usa Corpus para encontrar corpus.json y la API para el reporte de calidad.

9.3.2 src/estructuras_datos.py (649 líneas): estructuras de datos propias

Propósito: cumplir la regla de oro. En lugar de usar listas, diccionarios y funciones de alto nivel de Python dentro de los algoritmos, se escribieron estructuras propias, algunas heredadas del proyecto anterior del curso y otras ampliadas para este (ConjuntoHash, ordenar_por_mezcla, potencia_modular, promedio, mediana). Lo usan casi todos los módulos de src/.

Tabla 71. Estructuras y utilidades de estructuras_datos.py

Elemento (línea)	Qué es o hace	Dónde se usa
ListaDinamica L1	Arreglo que crece solo (duplica su capacidad cuando se llena): agregar, insertar, extraer, sublista, invertir_en_sitio, a_lista…	Tokens en el preprocesamiento; grupos activos en agrupar; intersección en jaccard.
Pila L137	Pila (último en entrar, primero en salir): apilar, desapilar.	Heredada del proyecto anterior; los algoritmos de este proyecto no la necesitan.
TablaHashSimple L154	Diccionario propio con direccionamiento abierto y redimensión automática: poner, obtener, contiene, claves.	Stopwords, vocabulario y conteo de frecuencias, tabla del carácter malo de Boyer-Moore.
ConjuntoManual L286	Conjunto simple basado en lista (búsqueda lineal).	Heredado; se sustituyó por ConjuntoHash en este proyecto.
ConjuntoHash L564	Conjunto basado en TablaHashSimple: agregar, contiene, tamano, elementos.	Jaccard (intersección y unión) y conteo de términos distintos.
crear_matriz L421	Crea una matriz de filas independientes con un valor inicial.	Tablas de Levenshtein y Needleman-Wunsch; matriz de distancias.
menor_de_dos, mayor_de_dos, menor_de_tres, mayor_de_tres, menor_de_varios, mayor_de_varios	Mínimos y máximos sin usar min() ni max().	mayor_de_tres en la recurrencia de Needleman-Wunsch y mayor_de_dos en el avance de Boyer-Moore; el resto son utilidades disponibles.
sumatoria L484, valor_absoluto L583	Suma de una lista y valor absoluto.	Estadísticos y utilidades numéricas.

Elemento (línea)	Qué es o hace	Dónde se usa
ordenar_por_mezcla L601	Merge sort estable O(n log n), con clave opcional y orden descendente.	Ordenar el vocabulario, los términos por aporte, los rankings y los criterios de clustering.
ordenar_por_seleccion L541	Ordenamiento por selección O(n²).	Heredado del proyecto anterior; no se usa en los algoritmos nuevos.
potencia_modular L589	Calcula (base^exponente) mod módulo por exponenciación binaria, O(log exponente).	Cálculo de B^(m−1) mod q en Rabin-Karp.
promedio L636, mediana L642	Media y mediana de una lista.	Correlación de Pearson, resúmenes y mediana de tiempos en las mediciones.
copiar_lista, sublista, invertir_lista, insertar_en_indice, extraer_en_indice, concatenar_iterables, crear_lista_rellena…	Utilidades pequeñas para copiar, cortar, invertir e insertar en listas.	Apoyo general; varias vienen del proyecto anterior.
obtener_valor, claves_diccionario, valores_diccionario, pares_diccionario, vaciar_diccionario…	Utilidades para diccionarios.	Heredadas; sirvieron de base de TablaHashSimple.

9.3.3 src/preprocesamiento.py (225 líneas): limpieza de texto

Propósito: convertir un abstract en la lista de palabras que los algoritmos van a comparar, con cada paso en una función pequeña. Lo usan**:** corpus.py (para preparar los 20 abstracts), patrones.py (solo a_minusculas) y api.py (el endpoint de preprocesar).

Tabla 72. Funciones de preprocesamiento.py

Función (línea)	Qué hace	Para qué sirve / quién la usa
a_minusculas L65	Pasa A-Z y letras latinas acentuadas mayúsculas a minúscula, carácter por carácter, sin cambiar la longitud.	Paso 1. También hace posible ignorar mayúsculas en la búsqueda sin mover posiciones.
quitar_tildes L94	Cambia letras con acento por su versión simple.	Paso 2. Unifica variantes del español.
limpiar_signos / es_letra_ascii L117	Convierte en espacio todo lo que no sea una letra a-z.	Paso 3. Elimina puntuación, dígitos, guiones y apóstrofes.
tokenizar L132	Parte el texto en palabras recorriéndolo con un bucle (sin split()).	Paso 4.
tabla_stopwords L53	Construye (una sola vez) la tabla hash con las stopwords en inglés y español.	Consulta rápida de palabras vacías.
quitar_stopwords L137	Descarta palabras vacías y las de menos de 2 letras.	Paso 5. Conserva ai.
stem_simple / _termina_en L167	Aplica la primera regla de sufijo que coincide; respeta palabras cortas y terminadas en ss, us, is.	Paso 6. Une variantes como learning y learn.
aplicar_stemming L181	Aplica stem_simple a una lista de palabras.	Paso 6, sobre todo el abstract.
texto_normalizado L189	Minúsculas + sin tildes + sin signos + espacios simples (conserva stopwords).	Variante usada en comparaciones de cadena.
preprocesar L207	Ejecuta todos los pasos y devuelve cada resultado intermedio (texto original, minúsculas, tokens, sin stopwords, con stemming, texto limpio).	Alimenta Corpus, el endpoint /api/preprocesar y la página «Corpus».

9.3.4 src/corpus.py (60 líneas): carga y cálculo base

Propósito: cargar los datos una sola vez y dejar listo todo lo que necesitan los tres requerimientos: abstracts limpios, vocabulario, IDF, vectores TF-IDF y embeddings. Lo usa: api.py, que obtiene el corpus con obtener_corpus().

Tabla 73. Funciones de corpus.py

Función (línea)	Qué hace	Para qué sirve / quién la usa
ErrorCorpus L10	Excepción propia para errores controlados (archivo faltante, artículo inexistente).	La API la convierte en un error 404 con mensaje claro.
Corpus.__init__ L14	Lee corpus.json, preprocesa los 20 abstracts, construye el vocabulario y el IDF, calcula el vector TF-IDF de cada abstract y carga los embeddings disponibles.	Es el «arranque en frío» de la aplicación: se ejecuta una sola vez.
Corpus.ids / obtener / resumen L14	Lista de ids; un artículo por id (valida que exista y esté entre 1 y 20); lista liviana para la interfaz.	Evitan que la API reciba ids inválidos.
obtener_corpus L56	Devuelve la única instancia de Corpus (la crea la primera vez).	Patrón «singleton»: todas las peticiones comparten los mismos datos en memoria.

9.3.5 src/similitud_clasica.py (319 líneas): los cuatro algoritmos clásicos

Propósito: implementar Levenshtein, Needleman-Wunsch, coseno con TF-IDF y Jaccard, con la matemática del capítulo 5. Lo usan: comparacion.py, clustering.py (vectores y coseno) y corpus.py (vocabulario e IDF).

Tabla 74. Funciones de similitud_clasica.py

Función (línea)	Qué hace	Para qué sirve / quién la usa
levenshtein_matriz L23	Construye la tabla completa de programación dinámica de Levenshtein.	Mostrar el llenado paso a paso (documentación y pruebas).
levenshtein_distancia L54	Misma recurrencia con solo dos filas.	Versión eficiente que usa la aplicación.

Función (línea)	Qué hace	Para qué sirve / quién la usa
levenshtein_similitud L86	1 − distancia / longitud mayor.	Normaliza a [0, 1].
needleman_wunsch_matriz L96	Tabla de puntajes de alineamiento global.	Base del alineamiento.
needleman_wunsch_puntaje L121	Solo el puntaje final, con dos filas.	Versión eficiente.
needleman_wunsch_alineamiento L146	Recupera el alineamiento óptimo (con huecos) retrocediendo desde la esquina final.	Documentación del caso de estudio.
needleman_wunsch_normalizado / needleman_wunsch_similitud L171	Normaliza el puntaje con las cotas teóricas.	Similitud entre 0 y 1.
construir_vocabulario L198	Reúne las palabras distintas de todo el corpus, las ordena y cuenta en cuántos documentos aparece cada una.	Base de TF-IDF.
calcular_idf L222	IDF suavizado de cada palabra.	Peso de rareza.
frecuencias L233	Cuenta las apariciones de cada palabra de un texto en una tabla hash.	Insumo del TF.
vector_tf / vector_tfidf L241	Vector de frecuencias y vector ponderado por IDF.	Representación de cada abstract.
producto_punto / norma L263	Suma de productos y raíz de la suma de cuadrados.	Piezas del coseno.
coseno_vectores / angulo_grados L276	Coseno entre dos vectores (con protección contra división por cero) y su ángulo.	Similitud TF-IDF y matriz de distancias del clustering.
jaccard L295	Intersección, unión y cociente entre dos conjuntos de palabras.	Similitud de Jaccard.

9.3.6 src/similitud_embeddings.py (75 líneas): similitud con embeddings

Propósito: operar sobre los vectores ya calculados fuera del servidor. No carga ningún modelo: solo lee los JSON. Lo usan: comparacion.py y corpus.py (para cargar los modelos).

Tabla 75. Funciones de similitud_embeddings.py

Función (línea)	Qué hace	Para qué sirve / quién la usa
producto_punto L15	Producto punto de dos vectores.	Base del coseno.
coseno L24	Coseno entre dos vectores, recortado a [−1, 1].	Similitud de embeddings.
distancia_euclidiana L37	Raíz de la suma de cuadrados de las diferencias.	Segunda medida pedida por el enunciado.
similitud_coseno_01 L47	Coseno recortado a [0, 1].	Similitud principal que se muestra.
similitud_euclidiana_01 L53	1 / (1 + distancia).	Convierte la distancia en similitud.
cargar_modelos L58	Lee todos los archivos embeddings_*.json disponibles.	Detecta los modelos sin tocar el código: añadir un JSON añade un modelo.

9.3.7 src/comparacion.py (131 líneas): comparar varios artículos

Propósito: orquestar el Requerimiento 1: recibir una lista de artículos, ejecutar los seis algoritmos sobre cada par y devolver todo ordenado para la interfaz. Lo usa: api.py.

Tabla 76. Funciones de comparacion.py

Función (línea)	Qué hace	Para qué sirve / quién la usa
ErrorComparacion L22	Excepción propia para selecciones inválidas.	La API la convierte en error 400.
validar_ids L26	Comprueba que los ids existan, no se repitan y sean entre 2 y 6.	Protege contra entradas inválidas.
similitud_par L45	Ejecuta los seis algoritmos sobre un par y mide el tiempo de cada uno.	Núcleo del requerimiento 1.

Función (línea)	Qué hace	Para qué sirve / quién la usa
algoritmos_disponibles L78	Lista los algoritmos disponibles, incluidos los modelos de embeddings encontrados.	La interfaz arma la tabla según esto.
comparar L86	Compara todos los pares de los artículos elegidos y arma las matrices de similitud.	Responde a POST /api/similarity.
detalle_par L107	Calcula los pasos intermedios de TF-IDF y Jaccard de un par.	Botón «Ver cálculo detallado».

9.3.8 src/clustering.py (313 líneas): clustering jerárquico

Propósito: el Requerimiento 2 completo: distancias, fusiones, métricas y coordenadas del dendrograma. Lo usan: visualizacion.py y api.py.

Tabla 77. Funciones de clustering.py

Función (línea)	Qué hace	Para qué sirve / quién la usa
matriz_distancias L25	Calcula d = 1 − coseno para todos los pares.	Paso previo al clustering.
_distancia_clusters L43	Distancia entre dos grupos según el criterio (mínimo, máximo o promedio).	Define Single, Complete y Average.
agrupar L68	Repite la fusión de los dos grupos más cercanos y guarda el historial.	Algoritmo principal.
miembros_por_cluster L107	Devuelve qué artículos pertenecen a cada grupo creado.	Apoyo para cortes y cofenética.
distancias_cofeneticas L122	Altura a la que cada par queda unido por primera vez.	Insumo de la correlación cofenética.
pearson / correlacion_cofenetica L139	Correlación de Pearson y su aplicación entre distancias originales y cofenéticas.	Métrica principal.
cortar_en_k L171	Deshace las últimas fusiones para obtener k grupos.	Control «k» de la interfaz y silueta.

Función (línea)	Qué hace	Para qué sirve / quién la usa
silueta / siluetas_por_k L203	Coeficiente de silueta para un corte y para varios k.	Métrica de apoyo.
evaluar / comparar_criterios L253	Evalúan un criterio, o los tres, y eligen el mejor (mayor CCC; desempate por silueta).	Responden a /api/clustering/comparar.
orden_hojas / coordenadas_dendrograma L277	Ordenan las hojas y convierten el historial en segmentos (x1, y1, x2, y2).	Dibujo de los dendrogramas sin scipy.

9.3.9 src/patrones.py (331 líneas): KMP, Boyer-Moore y Rabin-Karp

Propósito: el Requerimiento 3 completo, con la interfaz común ejecutar. Lo usan: benchmark_patrones.py y api.py.

Tabla 78. Funciones de patrones.py

Función (línea)	Qué hace	Para qué sirve / quién la usa
ErrorPatron / validar_entradas L26	Excepción propia y validación del patrón y el texto.	Entradas vacías, patrón de más de 200 caracteres, tipos incorrectos.
kmp_tabla_lps L40	Construye la tabla de prefijos-sufijos.	Preprocesamiento de KMP.
kmp_buscar L59	Busca sin retroceder en el texto y devuelve posiciones y comparaciones.	Búsqueda de KMP.
bm_tabla_caracter_malo L90	Última posición de cada carácter del patrón.	Regla del carácter malo.
bm_tabla_sufijo_bueno L100	Desplazamientos y bordes del sufijo bueno.	Regla del sufijo bueno.
bm_preprocesar / bm_buscar L128	Reúne las dos tablas y recorre el texto de derecha a izquierda con saltos.	Búsqueda de Boyer-Moore.
rk_hash L174	Hash polinomial (Horner) módulo q.	Hash del patrón y de la primera ventana.

Función (línea)	Qué hace	Para qué sirve / quién la usa
rk_preprocesar / rk_buscar L184	Calcula hash del patrón y B^(m−1) mod q, y recorre las ventanas con hash rodante verificando coincidencias.	Búsqueda de Rabin-Karp.
_normalizar L231	Pasa texto y patrón a minúsculas si se ignoran mayúsculas.	Conserva posiciones.
ejecutar L238	Interfaz única: ejecuta un algoritmo midiendo preprocesamiento y búsqueda por separado.	Todo lo demás la usa.
fragmento L273	Corta el contexto alrededor de una coincidencia en tres partes.	Resaltado en la interfaz.
buscar_en_corpus L285	Busca el patrón en los 20 abstracts y arma el resultado por artículo.	Responde a /api/patterns/search.
comparar_algoritmos L305	Corre los tres algoritmos y verifica que coincidan.	Etiqueta «los tres devuelven las mismas posiciones».

9.3.10 src/benchmark_patrones.py (62 líneas): mediciones de tiempo

Tabla 79. Funciones de benchmark_patrones.py

Función (línea)	Qué hace	Para qué sirve / quién la usa
texto_del_corpus L8	Une los 20 abstracts en un solo texto.	Texto base de las mediciones.
medir L21	Para cada algoritmo, mediana de varias corridas de preprocesamiento y búsqueda, más comparaciones.	Tablas de tiempos.
medir_escalabilidad L39	Repite el corpus k veces para ver cómo crece el tiempo.	Evidencia de crecimiento lineal.
medir_casos_sinteticos L52	Textos artificiales para mejor y peor caso.	Contrastar con Big O.

Propósito: contrastar la complejidad teórica con el comportamiento real. Usa time.perf_counter y la mediana de varias corridas para reducir el ruido.

9.3.11 src/visualizacion.py (188 líneas): gráficas

Tabla 80. Funciones de visualizacion.py

Función (línea)	Qué hace	Para qué sirve / quién la usa
png_dendrograma L22	Dibuja el dendrograma desde el historial, con corte en k y etiquetas coloreadas por grupo.	/plot/dendrogram.png.
png_metricas_clustering L51	Barras de correlación cofenética y curvas de silueta.	/plot/clustering_metricas.png.
png_heatmap L72	Mapa de calor de una matriz de similitud.	/plot/similarity.png.
png_alineaciones L90	Gráfica de iteraciones y desplazamientos de KMP, Boyer-Moore o Rabin-Karp.	/plot/patterns.png.
png_barras_tiempos / png_escalabilidad L161	Barras de tiempo y curva de escalabilidad.	Figuras de la sección 7.8.
_png L15	Convierte una figura de matplotlib en bytes PNG.	Función común a todas.

9.3.12 src/docs_algoritmos.py y src/api.py

docs_algoritmos.py (58 líneas) contiene la lista DOCS_ALGORITMOS: una ficha por algoritmo (idea, fórmula, complejidad, archivo y funciones) que alimenta la página Documentación. api.py (356 líneas) es la puerta de entrada: define las páginas, los endpoints y el manejo de errores; no contiene lógica de algoritmos, solo llama a los módulos anteriores.

Tabla 81. Rutas y manejadores de api.py

Función de api.py	Ruta	Qué hace
pagina	GET /ui/<módulo>	Entrega una de las cinco páginas HTML.
health	GET /health	Estado del servicio.
algorithm_docs	GET /api/algorithm-docs	Fichas de algoritmos.

Función de api.py	Ruta	Qué hace
api_corpus / api_doc / api_calidad	GET /api/corpus…	Lista, abstract con preprocesamiento y reporte de calidad.
api_preprocesar	POST /api/preprocesar	Preprocesa un texto libre.
api_similarity / api_similarity_detalle	POST /api/similarity, GET /api/similarity/detalle	Comparación de artículos y cálculo detallado.
plot_similarity	GET /plot/similarity.png	Mapa de calor.
api_clustering_comparar / _matriz / _grupos	GET /api/clustering/…	Métricas, matriz y grupos.
plot_dendrogram / plot_clustering_metricas	GET /plot/dendrogram.png, /plot/clustering_metricas.png	Gráficas del clustering.
api_patterns_search / _benchmark / _traza	GET /api/patterns/…	Búsqueda, tiempos e iteraciones.
plot_patterns	GET /plot/patterns.png	Gráfica de iteraciones.
informe	GET /report/<formato>	Descarga de documentos.
_error, _cache, _h_*	(interno)	Respuesta de error en JSON, caché de resultados y manejadores de errores 400, 404 y 500.

9.4 Carpeta scripts/: tareas fuera del servidor

Son herramientas que se ejecutan una vez, en el computador de los autores. No las usa la aplicación en producción. Pueden usar métodos de alto nivel (por ejemplo find()) porque no son parte de los algoritmos evaluados.

Tabla 82. Scripts del repositorio

Script	Qué hace	Funciones principales	Salidas
extraer_corpus.py (312 líneas)	Lee los 20 PDFs, extrae y valida los abstracts (sección 4.2).	leer_texto, unir_guiones, localizar_abstract, limpiar_abstract, aplicar_correccion, aplicar_reemplazos, evaluar_calidad, main	corpus.json, reporte_calidad.json
generar_embeddings.py (137 líneas)	Calcula el vector de cada abstract con GloVe y fastText (y, opcionalmente, un modelo de oraciones).	leer_vectores, promedio_normalizado, generar_estatico, generar_sbert	embeddings_glove.json, embeddings_fasttext.json
generar_evidencias.py (300 líneas)	Ejecuta los algoritmos reales, mide tiempos y guarda todos los números y figuras que usa este documento.	rangos, spearman, guardar_png, main	docs/evidencias/evidencias.json, docs/figuras/*.png, corpus_limpio.json
generar_documentos.py y auxiliares	Arma los diagramas y la documentación técnica a partir de las evidencias (sin escribir ninguna cifra a mano).	diagrama_componentes, diagrama_flujo, documentacion_tecnica	docs/DOCUMENTACION_TECNICA.md, docs/figuras/

9.5 Carpeta data/: los datos

Tabla 83. Archivos de datos

Archivo	Contenido	Quién lo genera y quién lo usa
processed/corpus.json	Para cada artículo: id, archivo, título, autores y abstract (y, si hubo corrección, abstract_original y correccion_manual).	Lo genera extraer_corpus.py; lo lee Corpus.

Archivo	Contenido	Quién lo genera y quién lo usa
processed/corpus_limpio.json	Texto normalizado, texto limpio y tokens finales de cada abstract.	Lo genera generar_evidencias.py; sirve de consulta.
processed/reporte_calidad.json	Palabras, caracteres, estado y problemas de cada abstract.	Lo genera extraer_corpus.py; lo muestra la página Corpus.
processed/embeddings_glove.json y **embeddings_fasttext.json**	Un vector normalizado de 300 números por abstract, y la cobertura de palabras.	Los genera generar_embeddings.py; los lee similitud_embeddings.py.
metadata_verificada.json	Título y autores de cada artículo, verificados a mano contra la primera página.	Lo lee extraer_corpus.py.
referencia_java/	Abstracts extraídos con otra herramienta en un trabajo anterior.	Se usa solo para validar la extracción.

9.6 Carpeta static/: la interfaz web

static/styles.css define el aspecto (con modo claro y oscuro) y static/app.js toda la lógica de la interfaz. Cada página vive en static/modules/<módulo>/index.html; todas incluyen el mismo app.js, que activa el código de la página según la etiqueta data-page. El código del navegador construye los elementos con **textContent**, nunca insertando texto como HTML, para evitar que un patrón de búsqueda malicioso inyecte código.

Tabla 84. Funciones principales de static/app.js

Función de app.js	Qué hace
h, api, estado, tabla, barra	Utilidades comunes: crear elementos de forma segura, llamar a la API, mostrar mensajes, construir tablas y barras.
paginaCorpus, verDoc, iniciarPreprocesar	Página «Corpus»: tabla de artículos, abstract con preprocesamiento paso a paso, caja de pruebas.

Función de app.js	Qué hace
paginaSimilitud, actualizarSeleccion, comparar, renderComparacion, verDetalle, cambiarMapa	Página Similitud: selección de 2 a 6 artículos, tarjetas por par, cálculo detallado y mapa de calor.
paginaClustering, elegirCriterio, dibujar	Página Clustering: métricas, conclusión, dendrograma, control de k, grupos e historial.
buscar, verIteraciones, benchmark, paginaPatrones	Página Patrones: búsqueda, fragmentos resaltados, gráficas de iteraciones y medición de tiempos.
paginaDocs	Página Documentación: fichas de algoritmos.

9.7 Carpeta tests/ y carpetas docs/ y modules/

tests/ contiene los siete archivos de pruebas descritos en la sección 8.5, más conftest.py, que prepara el corpus y el cliente de pruebas de la API. docs/ guarda los documentos, figuras/ (las gráficas que usa este documento) y evidencias/evidencias.json (los números reales). modules/ tiene un README.md por módulo con su propósito, archivos, entradas, salidas y sus evidencias.

9.8 Recorrido de una petición: qué funciones se ejecutan y en qué orden

Para ver cómo encajan las piezas, se sigue el camino completo de dos acciones de la aplicación.

Acción 1: buscar «perspectives» con «Los tres» (página Patrones).

1. static/modules/patterns/index.html tiene el formulario; al pulsar «Buscar», app.js ejecuta buscar() y arma la dirección /api/patterns/search?patron=perspectives&algoritmo=todos.
2. En src/api.py, la función api_patterns_search (línea 276) llama a _patron_arg para leer y validar el patrón y a _c() para obtener el corpus en memoria (corpus.obtener_corpus).
3. patrones.comparar_algoritmos (línea 305) recorre los 20 abstracts y, para cada algoritmo, llama a patrones.ejecutar (línea 238).
4. ejecutar valida con validar_entradas, pasa a minúsculas con _normalizar (que usa preprocesamiento.a_minusculas) y, según el algoritmo, llama a kmp_tabla_lps + kmp_buscar, a bm_preprocesar + bm_buscar o a rk_preprocesar + rk_buscar, midiendo el tiempo de cada etapa.
5. buscar_en_corpus arma, para cada posición, el fragmento con fragmento. La API responde en JSON y app.js dibuja cada coincidencia con la parte central dentro de <mark>.

Acción 2: comparar los artículos #2 y #6 (página Similitud).

1. app.js (comparar) envía POST /api/similarity con {"ids": [2, 6]}.
2. api_similarity (línea 159) llama a comparacion.comparar (línea 86), que valida con validar_ids.
3. Para el par, similitud_par (línea 45) llama a levenshtein_distancia, needleman_wunsch_puntaje, coseno_vectores (sobre los vectores TF-IDF que Corpus ya calculó), jaccard y, por cada modelo de embeddings, a coseno y distancia_euclidiana.
4. La respuesta incluye similitudes, tiempos y matrices; renderComparacion las dibuja y cambiarMapa pide /plot/similarity.png, que genera visualizacion.png_heatmap.

10. Resultados consolidados y conclusiones

10.1 Resumen de resultados

Tabla 85. Resultados consolidados

Requerimiento	Resultado principal	Dato medido
Corpus	20 de 20 abstracts extraídos y validados.	Vocabulario de 833 términos (con stemming) frente a 973 sin stemming.
1. Similitud	TF-IDF discrimina mejor; los embeddings promediados comprimen los valores; las medidas de edición dependen de la longitud.	Caso #2–#6: TF-IDF 0,329, Jaccard 0,173, GloVe 0,951, fastText 0,974.
2. Clustering	Average Linkage es el criterio más coherente.	Correlación cofenética: Average 0,803, Single 0,733, Complete 0,707. Silueta baja en los tres (máx. 0,069).
3. Patrones	Los tres algoritmos devuelven las mismas posiciones; el tiempo crece linealmente.	«perspectives»: posiciones 24 y 1002; comparaciones KMP 1.273, Boyer-Moore 165, Rabin-Karp 24.
4. Despliegue y calidad	Aplicación en Render, documentada y probada.	94 pruebas; resultados verificados contra scipy en pruebas.

10.2 Conclusiones

- **Similitud.** Ninguna medida es «la mejor» porque miden cosas distintas: la forma de las cadenas (Levenshtein y Needleman-Wunsch), el vocabulario compartido (TF-IDF y Jaccard) o el significado aproximado (embeddings). En este corpus TF-IDF separa mejor los pares; las medidas de edición dependen sobre todo de la longitud de los textos; y los embeddings promediados dan valores altos para casi todo, de modo que solo es útil comparar el orden de los pares.
- **Clustering.** Average Linkage reproduce mejor las distancias originales (correlación cofenética de 0,803). La silueta es baja en los tres criterios porque los abstracts son del mismo dominio y forman una nube sin grupos nítidos.
- **Búsqueda de patrones.** KMP, Boyer-Moore y Rabin-Karp devuelven exactamente las mismas posiciones. Los tiempos crecen linealmente con el tamaño del texto, y el mejor caso de Boyer-Moore

(n/m comparaciones) y el peor caso O(n·m) de Boyer-Moore y Rabin-Karp se observaron de forma directa en el número de comparaciones.

- **Proceso.** Separar lo pesado (PDFs y embeddings) de lo liviano (algoritmos) permitió una aplicación pequeña, rápida de desplegar y fácil de explicar. Escribir cada algoritmo desde cero obligó a entender sus fórmulas y a poder conectarlas con el código línea a línea.

10.3 Limitaciones

- Solo se procesan 20 abstracts en inglés. El stemming es simple y puede unir o separar formas incorrectamente (por ejemplo, series pasa a sery).
- Los embeddings son promedios de vectores de palabras estáticos: pierden el orden y el contexto. El script opcional con un modelo de oraciones tipo Transformer está escrito pero **no se ejecutó**, así que no se reportan resultados con él.
- La silueta es baja: el corpus es pequeño y homogéneo. No se probó el criterio de Ward.
- Boyer-Moore no incluye la regla de Galil, por lo que su peor caso con muchas ocurrencias es O(n·m).
- Los tiempos se midieron en una sola máquina; sirven para contrastar formas de crecimiento y conteos de comparaciones, no como valores absolutos.
- El plan gratuito de Render duerme el servicio tras un rato sin uso, y comparar 6 artículos a la vez puede tardar varios segundos.

10.4 Trabajo futuro

- Probar modelos de oraciones (tipo Transformer) y restar la componente común de los embeddings, como sugiere el diagnóstico de la sección 5.7.
- Ampliar el corpus y comparar el clustering con otras representaciones (por ejemplo, embeddings) y con el criterio de Ward.
- Implementar la regla de Galil y la búsqueda de varios patrones a la vez (Aho-Corasick); explorar búsqueda aproximada.
- Añadir un buscador por significado sobre los embeddings, y exportación de resultados.

11. Matriz final de cumplimiento del enunciado

Esta matriz resume, punto por punto, lo que exige el enunciado oficial y dónde se puede verificar que se cumplió.

Tabla 86. Matriz de cumplimiento del enunciado

Exigencia del enunciado	Cómo se cumple	Dónde verificarlo	Estado
Corpus de 20 PDFs; abstract como foco; título y autores como metadatos	Extracción y validación 20 de 20	Cap. 4; corpus.json; reporte_calidad.json	Cumple
R1: selección dinámica de dos o más artículos	Página de similitud (2 a 6)	5.2; /ui/similarity	Cumple
R1: 4 algoritmos clásicos (distancia de edición, TF-IDF + coseno, Jaccard)	Levenshtein, Needleman-Wunsch, coseno TF-IDF y Jaccard	5.3; similitud_clasica.py	Cumple
R1: 2 algoritmos con embeddings (coseno o euclidiana)	GloVe y fastText	5.4; similitud_embeddings.py	Cumple
R1: caso con dos abstracts, demostración paso a paso y análisis crítico	Abstracts #2 y #6	5.6 y 5.7	Cumple
R2: tres algoritmos de clustering jerárquico desde cero	Single, Complete y Average	6.3; clustering.py	Cumple
R2: dendrograma de cada uno	Tres dendrogramas desde el historial propio	6.6	Cumple
R2: métrica interna para el mejor	Correlación cofenética y silueta	6.4 y 6.7	Cumple
R3: KMP, Boyer-Moore y Rabin-Karp sin funciones nativas	patrones.py	7.3 a 7.5	Cumple
R3: caso con patrón y párrafo, estructuras auxiliares y gráficas	«perspectives» en el abstract #9	7.7	Cumple

Exigencia del enunciado	Cómo se cumple	Dónde verificarlo	Estado
R3: análisis Big O frente a tiempos reales	Corpus, escalabilidad y casos límite	7.8	Cumple
R4: despliegue con enlace público	https://bibliometria-algoritmos-gp0j.onrender.com	8.3	Cumple
R4: documento con arquitectura, implementación y resultados	Este documento	Caps. 3 a 10	Cumple
Declaración del uso de IA generativa	Capítulo 12	Cap. 12	Cumple
Sin funciones de alto nivel que implementen los algoritmos	Lógica en src/ con listas y bucles	Cap. 9; tests/	Cumple
Pruebas y manejo de errores	94 pruebas; errores controlados	8.5	Cumple

12. Declaración de uso de IA generativa

Se utilizó **Claude (Anthropic)** como asistente de desarrollo durante el proyecto. Su aporte fue de apoyo:

- Generó borradores de código (implementación de los algoritmos, API, interfaz web), pruebas unitarias y la redacción inicial de la documentación, a partir de los requerimientos y decisiones indicados por el grupo (por ejemplo, la elección de los modelos de embeddings, los criterios de enlace y la estructura del repositorio).
- Ejecutó el código en su entorno para producir las evidencias numéricas, las figuras y las tablas del documento; ninguna cifra se escribió a mano.
- Los modelos preentrenados de embeddings (GloVe y fastText) son modelos de terceros usados como insumo, descritos en la sección 5.4.

Lo que la IA no reemplaza. El grupo es responsable de comprender cada algoritmo (fórmulas, complejidad y casos de estudio), de revisar y ejecutar el código, de verificar los resultados (las pruebas y la comparación con scipy están en el repositorio) y de sustentarlos. Las referencias bibliográficas fueron sugeridas por el asistente a partir de su conocimiento general; sus datos de publicación deben contrastarse con las fuentes originales.

A continuación se presentan algunos ejemplos de los prompts empleados durante el desarrollo del proyecto. Dado que el lenguaje natural suele ser amplio o ambiguo para los modelos de IA, antes de enviar cada instrucción a nuestra sesión principal la refinamos con apoyo de otras IAs en sesiones independientes. Este paso previo nos permitió formular solicitudes más precisas, orientadas a tareas específicas y con resultados que podíamos verificar por nuestra cuenta, en lugar de depender de una generación automática sin control.

“Actúa como corrector de estilo académico en español. Te comparto un capítulo de un documento técnico universitario de Ingeniería de Sistemas. Corrige ortografía, gramática y puntuación, y mejora la organización y la claridad sin cambiar el contenido técnico ni las cifras. Aplica APA 7 en citas, tablas y figuras. Devuélveme el texto corregido y una lista breve de los cambios importantes para que yo los revise.”

“A partir de los temas y autores que menciona mi documento (Levenshtein, TF-IDF, KMP, clustering jerárquico, GloVe, fastText, etc.), propón referencias en formato APA 7 de fuentes primarias y libros de referencia. No inventes datos: si no estás seguro de un año, volumen, páginas o DOI, márcalo con «[verificar]». Ordénalas alfabéticamente.”

“Mi aplicación web ya funciona en local. Guíame paso a paso para desplegarla en Render: configuración del servicio, comando de build y de inicio, variables de entorno, manejo de archivos de datos y errores comunes. Para cada paso indica cómo comprobar que salió bien.”

“Revisa este módulo en Python. Señala errores, casos límite no cubiertos y problemas de complejidad o legibilidad. Para cada hallazgo indica archivo y línea, la gravedad (alta, media, baja), por qué es un problema y una corrección propuesta. Si es posible, ejecuta o propón pruebas que lo confirmen.”

“Explícame, a nivel de estudiante de pregrado, cómo funcionan GloVe y fastText, en qué se diferencian (por ejemplo, el manejo de subpalabras) y cómo se usan para comparar la similitud entre textos. Incluye ventajas y limitaciones, y recomiéndame fuentes de estudio (artículos originales, documentación oficial y cursos), indicando cuáles son primarias.”

“Esta es la interfaz de mi aplicación (HTML/CSS/JS). Propón un rediseño más moderno y legible, con jerarquía visual clara, paleta coherente, buen contraste y diseño adaptable a móvil. Conserva los ids y las clases que usa el JavaScript para no romper la funcionalidad. Entrégame el CSS y los cambios de HTML necesarios.”

“Explícame este algoritmo (Boyer-Moore) con un ejemplo paso a paso y dime qué preguntas podría hacerme un jurado sobre su complejidad.”

“Implementa clustering jerárquico aglomerativo sin librerías de clustering, con tres criterios de enlace: Single, Complete y Average Linkage. La entrada es una matriz de distancias de 20×20. Debe devolver el historial de fusiones (clústeres unidos, altura y tamaño). Calcula también la correlación cofenética y el coeficiente de silueta, implementados a mano. Explica cómo se actualiza la distancia en cada criterio y cómo comprobar el resultado contra scipy, usado solo como verificación y no como parte de la solución.”

“Genera un diagrama de componentes para una aplicación web con esta arquitectura: navegador → API (FastAPI) → tres módulos algorítmicos (similitud, clustering, patrones), con datos preprocesados fuera del servidor. Haz también un diagrama de flujo de datos que separe lo que se ejecuta una sola vez (extracción del PDF y embeddings) de lo que ocurre en cada petición. Usa colores sobrios, etiquetas legibles y estilo apto para un documento impreso. Entrégalo como [PNG / SVG / Mermaid].”

“Con la matriz de distancias de los 20 abstracts, genera con matplotlib un mapa de calor de similitud y un dendrograma por criterio de enlace (Single, Complete y Average). Etiqueta cada hoja con el número del artículo, usa la misma escala de color en las figuras comparables y agrega título y leyenda. Guarda los PNG a 200 dpi.”

“Escribe pruebas con pytest para estas funciones, con casos conocidos de la literatura (por ejemplo, la distancia entre «kitten» y «sitting» es 3). Incluye casos límite y una comparación contra una implementación de referencia, solo dentro de las pruebas.”

“Implementa KMP (tabla LPS), Boyer-Moore (heurísticas de mal carácter y buen sufijo) y Rabin-Karp (hash rodante) sin usar str.find, in ni expresiones regulares. Cada función debe devolver todas las posiciones de coincidencia y el número de comparaciones realizadas. Maneja los casos límite: patrón vacío, patrón más largo que el texto y ninguna coincidencia. Explica la complejidad de cada algoritmo en el mejor y el peor caso, con un ejemplo de traza.”

13. Referencias

Bibliografía de apoyo sobre los algoritmos y modelos utilizados.

[1] V. I. Levenshtein, «Binary codes capable of correcting deletions, insertions and reversals», *Soviet Physics Doklady*, 10(8), 707–710, 1966.

[2] S. B. Needleman y C. D. Wunsch, «A general method applicable to the search for similarities in the amino acid sequence of two proteins», *Journal of Molecular Biology*, 48(3), 443–453, 1970.

[3] P. Jaccard, «Distribution de la flore alpine dans le bassin des Dranses et dans quelques régions voisines», *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37, 241–272, 1901.

[4] G. Salton y C. Buckley, «Term-weighting approaches in automatic text retrieval», *Information Processing & Management*, 24(5), 513–523, 1988.

[5] D. E. Knuth, J. H. Morris y V. R. Pratt, «Fast pattern matching in strings», *SIAM Journal on Computing*, 6(2), 323–350, 1977.

[6] R. S. Boyer y J. S. Moore, «A fast string searching algorithm», *Communications of the ACM*, 20(10), 762–772, 1977.

[7] R. M. Karp y M. O. Rabin, «Efficient randomized pattern-matching algorithms», *IBM Journal of Research and Development*, 31(2), 249–260, 1987.

[8] R. R. Sokal y C. D. Michener, «A statistical method for evaluating systematic relationships», *University of Kansas Science Bulletin*, 38, 1409–1438, 1958.

[9] R. R. Sokal y F. J. Rohlf, «The comparison of dendrograms by objective methods», *Taxon*, 11(2), 33–40, 1962.

[10] P. J. Rousseeuw, «Silhouettes: a graphical aid to the interpretation and validation of cluster analysis», *Journal of Computational and Applied Mathematics*, 20, 53–65, 1987.

[11] J. Pennington, R. Socher y C. D. Manning, «GloVe: Global vectors for word representation», *Proc. EMNLP*, 1532–1543, 2014.

[12] T. Mikolov, E. Grave, P. Bojanowski, C. Puhrsch y A. Joulin, «Advances in pre-training distributed word representations», *Proc. LREC*, 2018.

[13] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, MIT Press (capítulos de programación dinámica y de coincidencia de cadenas).

Corpus analizado (20 artículos):

1. Matthias Carl Laupichler, Alexandra Aster, Jana Schirch, Tobias Raupach. *Artificial intelligence literacy in higher and adult education: A scoping literature review*.

2. Thomas K.F. Chiu, Qi Xia, Xinyan Zhou, Ching Sing Chai, Miaoting Cheng. *Systematic literature review on opportunities, challenges, and future research recommendations of artificial intelligence in education*.

3. Arne Bewersdorff, Xiaoming Zhai, Jessica Roberts, Claudia Nerdel. *Myths, mis- and preconceptions of artificial intelligence: A review of the literature*.

4. Srecko Joksimovic, Dirk Ifenthaler, Rebecca Marrone, Maarten De Laat, George Siemens. *Opportunities of artificial intelligence for supporting complex problem-solving: Findings from a scoping review*.

5. Xieling Chen, Haoran Xie, Di Zou, Gwo-Jen Hwang. *Application and theory gaps during the rise of Artificial Intelligence in Education*.

6. Xieling Chen, Haoran Xie, Gwo-Jen Hwang. *A multi-perspective study on Artificial Intelligence in Education: grants, conferences, journals, software tools, institutions, and researchers*.

7. Riordan Alfredo, Vanessa Echeverria, Yueqiao Jin, Lixiang Yan, Zachari Swiecki, Dragan Gašević, Roberto Martinez-Maldonado. *Human-centred learning analytics and AI in education: A systematic literature review*.

8. Seng Chee Tan, Alwyn Vwen Yen Lee, Min Lee. *A systematic review of artificial intelligence techniques for collaborative learning over the past two decades*.

9. Ke Zhang, Ayse Begum Aslan. *AI technologies for education: Recent research & future directions*.

10. Chih-Pu Dai, Fengfeng Ke. *Educational applications of artificial intelligence in simulation-based learning: A systematic mapping review*.

11. Qi Xia, Xiaojing Weng, Fan Ouyang, Tzung Jin Lin, Thomas K.F. Chiu. *A scoping review on how generative artificial intelligence transforms assessment in higher education*.

12. Melissa Bond, Hassan Khosravi, Maarten De Laat, Nina Bergdahl, Violeta Negrea, Emily Oxley, Phuong Pham, Sin Wang Chong, George Siemens. *A meta systematic review of artificial intelligence in higher education: a call for increased ethics, collaboration, and rigour*.

13. Jyoti Mudkanna Gavhane, Reena Pagare. *Artificial intelligence for education and its emphasis on assessment and adversity quotient: a review*.

14. Tamar Mikeladze, Paulien C. Meijer, Roald P. Verhoeff. *A comprehensive exploration of artificial intelligence competence frameworks for educators: A critical review*.

15. Bayode Ogunleye, Kudirat Ibilola Zakariyyah, Oluwaseun Ajao, Olakunle Olayinka, Hemlata Sharma. *A Systematic Review of Generative AI for Teaching and Learning Practice*.

16. Che Yee Lye, Lyndon Lim. *Generative Artificial Intelligence in Tertiary Education: Assessment Redesign Principles and Considerations*.

17. Fan Ouyang, Tuan Anh Dinh, Weiqi Xu. *A Systematic Review of AI-Driven Educational Assessment in STEM Education*.

18. Zied Bahroun, Chiraz Anane, Vian Ahmed, Andrew Zacca. *Transforming Education: A Comprehensive Review of Generative Artificial Intelligence in Educational Settings through Bibliometric and Content Analysis*.

19. Chung Kwan Lo. *What Is the Impact of ChatGPT on Education? A Rapid Review of the Literature*.

20. Olaf Zawacki-Richter, Victoria I. Marín, Melissa Bond, Franziska Gouverneur. *Systematic review of research on artificial intelligence applications in higher education – where are the educators?*.

Anexo A. Términos TF-IDF del caso de estudio (#2 y #6)

Los 104 términos que usan en conjunto los dos abstracts, ordenados por su aporte al producto punto. N = 20; IDF = ln(21/(1+df)) + 1. Los términos que aparecen en un solo abstract tienen producto 0.

Tabla 87. Tabla completa de términos del caso de estudio

Término	df	IDF	TF #2	TF #6	Peso #2	Peso #6	Producto
aied	7	1,965	0,0676	0,0686	0,1328	0,1349	0,01791
researcher	5	2,253	0,0135	0,0294	0,0304	0,0663	0,00202
relevant	7	1,965	0,0135	0,0294	0,0266	0,0578	0,00153
educate	18	1,100	0,0405	0,0294	0,0446	0,0324	0,00144
major	2	2,946	0,0135	0,0098	0,0398	0,0289	0,00115
new	3	2,658	0,0135	0,0098	0,0359	0,0261	0,00094
technology	12	1,480	0,0405	0,0098	0,0600	0,0145	0,00087
aim	4	2,435	0,0135	0,0098	0,0329	0,0239	0,00079
research	16	1,211	0,0270	0,0196	0,0327	0,0238	0,00078
identify	10	1,647	0,0135	0,0196	0,0223	0,0323	0,00072
study	14	1,336	0,0135	0,0294	0,0181	0,0393	0,00071
applicate	11	1,560	0,0135	0,0196	0,0211	0,0306	0,00064
literature	11	1,560	0,0270	0,0098	0,0422	0,0153	0,00064
ai	18	1,100	0,0270	0,0196	0,0297	0,0216	0,00064
present	6	2,099	0,0135	0,0098	0,0284	0,0206	0,00058
understand	10	1,647	0,0135	0,0098	0,0223	0,0161	0,00036
artificial	20	1,000	0,0135	0,0196	0,0135	0,0196	0,00026
intelligence	20	1,000	0,0135	0,0196	0,0135	0,0196	0,00026
able	2	2,946	0,0000	0,0098	0,0000	0,0289	0,00000
active	2	2,946	0,0000	0,0196	0,0000	0,0578	0,00000
actor	1	3,351	0,0000	0,0098	0,0000	0,0329	0,00000

Término	df	IDF	TF #2	TF #6	Peso #2	Peso #6	Producto
addition	2	2,946	0,0000	0,0098	0,0000	0,0289	0,00000
administrate	1	3,351	0,0135	0,0000	0,0453	0,0000	0,00000
alike	1	3,351	0,0135	0,0000	0,0453	0,0000	0,00000
analysis	7	1,965	0,0135	0,0000	0,0266	0,0000	0,00000
approach	8	1,847	0,0135	0,0000	0,0250	0,0000	0,00000
article	4	2,435	0,0000	0,0098	0,0000	0,0239	0,00000
assessment	10	1,647	0,0135	0,0000	0,0223	0,0000	0,00000
aware	1	3,351	0,0000	0,0098	0,0000	0,0329	0,00000
challeng	6	2,099	0,0270	0,0000	0,0567	0,0000	0,00000
cod	5	2,253	0,0135	0,0000	0,0304	0,0000	0,00000
computer	4	2,435	0,0000	0,0098	0,0000	0,0239	0,00000
concern	4	2,435	0,0000	0,0196	0,0000	0,0477	0,00000
conferenc	1	3,351	0,0000	0,0098	0,0000	0,0329	0,00000
content	6	2,099	0,0135	0,0000	0,0284	0,0000	0,00000
context	7	1,965	0,0000	0,0098	0,0000	0,0193	0,00000
continuous	1	3,351	0,0000	0,0098	0,0000	0,0329	0,00000
contribute	1	3,351	0,0000	0,0098	0,0000	0,0329	0,00000
current	9	1,742	0,0135	0,0000	0,0235	0,0000	0,00000
development	5	2,253	0,0000	0,0392	0,0000	0,0883	0,00000
direction	3	2,658	0,0135	0,0000	0,0359	0,0000	0,00000
domain	6	2,099	0,0270	0,0000	0,0567	0,0000	0,00000
educator	5	2,253	0,0000	0,0294	0,0000	0,0663	0,00000
elsevier	1	3,351	0,0000	0,0098	0,0000	0,0329	0,00000
emerg	5	2,253	0,0135	0,0000	0,0304	0,0000	0,00000
enabl	1	3,351	0,0000	0,0098	0,0000	0,0329	0,00000
examin	4	2,435	0,0270	0,0000	0,0658	0,0000	0,00000

Término	df	IDF	TF #2	TF #6	Peso #2	Peso #6	Producto
field	8	1,847	0,0000	0,0098	0,0000	0,0181	0,00000
find	11	1,560	0,0000	0,0098	0,0000	0,0153	0,00000
focus	6	2,099	0,0135	0,0000	0,0284	0,0000	0,00000
four	3	2,658	0,0135	0,0000	0,0359	0,0000	0,00000
furthermore	3	2,658	0,0000	0,0098	0,0000	0,0261	0,00000
future	11	1,560	0,0135	0,0000	0,0211	0,0000	0,00000
grant	1	3,351	0,0000	0,0196	0,0000	0,0657	0,00000
grow	2	2,946	0,0000	0,0098	0,0000	0,0289	0,00000
growth	2	2,946	0,0000	0,0098	0,0000	0,0289	0,00000
help	4	2,435	0,0000	0,0098	0,0000	0,0239	0,00000
highlight	4	2,435	0,0000	0,0098	0,0000	0,0239	0,00000
implementate	2	2,946	0,0000	0,0098	0,0000	0,0289	0,00000
institution	4	2,435	0,0000	0,0196	0,0000	0,0477	0,00000
integrat	2	2,946	0,0135	0,0000	0,0398	0,0000	0,00000
interest	3	2,658	0,0000	0,0098	0,0000	0,0261	0,00000
issu	4	2,435	0,0000	0,0196	0,0000	0,0477	0,00000
journal	6	2,099	0,0000	0,0392	0,0000	0,0823	0,00000
key	4	2,435	0,0270	0,0000	0,0658	0,0000	0,00000
last	3	2,658	0,0135	0,0000	0,0359	0,0000	0,00000
launch	2	2,946	0,0000	0,0098	0,0000	0,0289	0,00000
learn	12	1,480	0,0405	0,0000	0,0600	0,0000	0,00000
matrix	1	3,351	0,0135	0,0000	0,0453	0,0000	0,00000
multiple	2	2,946	0,0000	0,0098	0,0000	0,0289	0,00000
nam	1	3,351	0,0000	0,0098	0,0000	0,0329	0,00000
necessity	1	3,351	0,0000	0,0098	0,0000	0,0329	0,00000
neglect	1	3,351	0,0135	0,0000	0,0453	0,0000	0,00000

Término	df	IDF	TF #2	TF #6	Peso #2	Peso #6	Producto
opportunity	4	2,435	0,0135	0,0000	0,0329	0,0000	0,00000
outcom	3	2,658	0,0270	0,0000	0,0718	0,0000	0,00000
overview	5	2,253	0,0000	0,0098	0,0000	0,0221	0,00000
perspectiv	4	2,435	0,0000	0,0098	0,0000	0,0239	0,00000
practitioner	1	3,351	0,0135	0,0000	0,0453	0,0000	0,00000
provid	5	2,253	0,0135	0,0000	0,0304	0,0000	0,00000
provide	10	1,647	0,0000	0,0098	0,0000	0,0161	0,00000
publicate	5	2,253	0,0000	0,0098	0,0000	0,0221	0,00000
rapid	4	2,435	0,0000	0,0098	0,0000	0,0239	0,00000
relat	4	2,435	0,0000	0,0098	0,0000	0,0239	0,00000
relate	1	3,351	0,0000	0,0098	0,0000	0,0329	0,00000
relationship	4	2,435	0,0135	0,0000	0,0329	0,0000	0,00000
result	8	1,847	0,0135	0,0000	0,0250	0,0000	0,00000
review	18	1,100	0,0405	0,0000	0,0446	0,0000	0,00000
rol	1	3,351	0,0135	0,0000	0,0453	0,0000	0,00000
scholar	4	2,435	0,0000	0,0196	0,0000	0,0477	0,00000
scientific	1	3,351	0,0000	0,0098	0,0000	0,0329	0,00000
significance	1	3,351	0,0000	0,0098	0,0000	0,0329	0,00000
significant	4	2,435	0,0000	0,0098	0,0000	0,0239	0,00000
software	1	3,351	0,0000	0,0098	0,0000	0,0329	0,00000
status	2	2,946	0,0000	0,0098	0,0000	0,0289	0,00000
student	8	1,847	0,0135	0,0000	0,0250	0,0000	0,00000
suggestion	4	2,435	0,0135	0,0000	0,0329	0,0000	0,00000
systematic	10	1,647	0,0135	0,0000	0,0223	0,0000	0,00000
teach	6	2,099	0,0135	0,0000	0,0284	0,0000	0,00000
teacher	5	2,253	0,0135	0,0000	0,0304	0,0000	0,00000

Término	df	IDF	TF #2	TF #6	Peso #2	Peso #6	Producto
term	5	2,253	0,0000	0,0098	0,0000	0,0221	0,00000
tool	5	2,253	0,0000	0,0098	0,0000	0,0221	0,00000
top	1	3,351	0,0000	0,0098	0,0000	0,0329	0,00000
trend	6	2,099	0,0000	0,0098	0,0000	0,0206	0,00000
year	5	2,253	0,0135	0,0000	0,0304	0,0000	0,00000

Anexo B. Matriz de distancias entre los 20 abstracts

Distancia = 1 − coseno TF-IDF. Se muestra en dos bloques de diez columnas.

Tabla 88. Matriz de distancias: columnas #1 a #10

	#1	#2	#3	#4	#5	#6	#7	#8	#9	#10
#1	0,000	0,836	0,812	0,919	0,873	0,898	0,957	0,875	0,875	0,858
#2	0,836	0,000	0,856	0,915	0,727	0,671	0,826	0,821	0,709	0,806
#3	0,812	0,856	0,000	0,926	0,846	0,876	0,942	0,807	0,883	0,839
#4	0,919	0,915	0,926	0,000	0,941	0,950	0,960	0,914	0,934	0,901
#5	0,873	0,727	0,846	0,941	0,000	0,761	0,857	0,789	0,786	0,822
#6	0,898	0,671	0,876	0,950	0,761	0,000	0,829	0,931	0,678	0,915
#7	0,957	0,826	0,942	0,960	0,857	0,829	0,000	0,956	0,862	0,965
#8	0,875	0,821	0,807	0,914	0,789	0,931	0,956	0,000	0,905	0,712
#9	0,875	0,709	0,883	0,934	0,786	0,678	0,862	0,905	0,000	0,890
#10	0,858	0,806	0,839	0,901	0,822	0,915	0,965	0,712	0,890	0,000
#11	0,841	0,782	0,882	0,952	0,875	0,921	0,918	0,855	0,846	0,838
#12	0,849	0,777	0,877	0,931	0,846	0,826	0,890	0,905	0,795	0,905
#13	0,840	0,843	0,853	0,938	0,880	0,892	0,923	0,893	0,907	0,892
#14	0,848	0,868	0,833	0,947	0,928	0,869	0,956	0,894	0,880	0,903
#15	0,860	0,852	0,866	0,937	0,902	0,927	0,903	0,860	0,871	0,864
#16	0,932	0,921	0,899	0,960	0,878	0,902	0,963	0,953	0,915	0,918
#17	0,865	0,804	0,839	0,950	0,765	0,915	0,944	0,867	0,853	0,807
#18	0,918	0,823	0,923	0,951	0,874	0,896	0,921	0,939	0,836	0,919
#19	0,946	0,860	0,943	0,940	0,928	0,913	0,930	0,962	0,937	0,939
#20	0,859	0,709	0,913	0,950	0,778	0,765	0,846	0,899	0,760	0,892

Tabla 89. Matriz de distancias: columnas #11 a #20

	#11	#12	#13	#14	#15	#16	#17	#18	#19	#20
#1	0,841	0,849	0,840	0,848	0,860	0,932	0,865	0,918	0,946	0,859
#2	0,782	0,777	0,843	0,868	0,852	0,921	0,804	0,823	0,860	0,709
#3	0,882	0,877	0,853	0,833	0,866	0,899	0,839	0,923	0,943	0,913
#4	0,952	0,931	0,938	0,947	0,937	0,960	0,950	0,951	0,940	0,950
#5	0,875	0,846	0,880	0,928	0,902	0,878	0,765	0,874	0,928	0,778
#6	0,921	0,826	0,892	0,869	0,927	0,902	0,915	0,896	0,913	0,765
#7	0,918	0,890	0,923	0,956	0,903	0,963	0,944	0,921	0,930	0,846
#8	0,855	0,905	0,893	0,894	0,860	0,953	0,867	0,939	0,962	0,899
#9	0,846	0,795	0,907	0,880	0,871	0,915	0,853	0,836	0,937	0,760
#10	0,838	0,905	0,892	0,903	0,864	0,918	0,807	0,919	0,939	0,892
#11	0,000	0,836	0,823	0,836	0,845	0,762	0,716	0,910	0,893	0,809
#12	0,836	0,000	0,867	0,924	0,855	0,964	0,852	0,864	0,913	0,702
#13	0,823	0,867	0,000	0,908	0,910	0,897	0,799	0,868	0,922	0,830
#14	0,836	0,924	0,908	0,000	0,914	0,907	0,914	0,915	0,978	0,900
#15	0,845	0,855	0,910	0,914	0,000	0,940	0,884	0,888	0,933	0,839
#16	0,762	0,964	0,897	0,907	0,940	0,000	0,800	0,904	0,898	0,934
#17	0,716	0,852	0,799	0,914	0,884	0,800	0,000	0,865	0,912	0,786
#18	0,910	0,864	0,868	0,915	0,888	0,904	0,865	0,000	0,914	0,877
#19	0,893	0,913	0,922	0,978	0,933	0,898	0,912	0,914	0,000	0,940
#20	0,809	0,702	0,830	0,900	0,839	0,934	0,786	0,877	0,940	0,000

Anexo C. Historial completo de fusiones

Cada fila es una fusión. #n es el artículo n; Cn, el grupo creado por una fusión anterior (numerados desde C20).

Tabla 90. Historial de Single Linkage (enlace simple)

Paso	Une	con	Distancia	Tamaño	Grupo nuevo
1	#2	#6	0,6713	2	C20
2	#9	C20	0,6775	3	C21
3	#12	#20	0,7017	2	C22
4	C21	C22	0,7094	5	C23
5	#8	#10	0,7117	2	C24
6	#11	#17	0,7163	2	C25
7	#5	C23	0,7269	6	C26
8	#16	C25	0,7615	3	C27
9	C26	C27	0,7646	9	C28
10	C24	C28	0,7890	11	C29
11	#13	C29	0,7988	12	C30
12	#3	C30	0,8073	13	C31
13	#1	C31	0,8120	14	C32
14	#18	C32	0,8226	15	C33
15	#7	C33	0,8260	16	C34
16	#14	C34	0,8332	17	C35
17	#15	C35	0,8387	18	C36
18	#19	C36	0,8602	19	C37
19	#4	C37	0,9005	20	C38

Tabla 91. Historial de Complete Linkage (enlace completo)

Paso	Une	con	Distancia	Tamaño	Grupo nuevo
1	#2	#6	0,6713	2	C20
2	#12	#20	0,7017	2	C21
3	#9	C20	0,7088	3	C22
4	#8	#10	0,7117	2	C23
5	#11	#17	0,7163	2	C24
6	#5	C22	0,7860	4	C25
7	#16	C24	0,8002	3	C26
8	#1	#3	0,8120	2	C27
9	C21	C25	0,8456	6	C28
10	#14	C27	0,8484	3	C29
11	#15	C23	0,8638	3	C30
12	#13	#18	0,8681	2	C31
13	#7	C28	0,8899	7	C32
14	C26	C31	0,9101	5	C33
15	C29	C30	0,9143	6	C34
16	#19	C33	0,9223	6	C35
17	#4	C34	0,9470	7	C36
18	C32	C35	0,9638	13	C37
19	C36	C37	0,9785	20	C38

Tabla 92. Historial de Average Linkage (UPGMA)

Paso	Une	con	Distancia	Tamaño	Grupo nuevo
1	#2	#6	0,6713	2	C20
2	#9	C20	0,6932	3	C21
3	#12	#20	0,7017	2	C22

Paso	Une	con	Distancia	Tamaño	Grupo nuevo
4	#8	#10	0,7117	2	C23
5	#11	#17	0,7163	2	C24
6	#5	C21	0,7580	4	C25
7	#16	C24	0,7809	3	C26
8	C22	C25	0,7820	6	C27
9	#1	#3	0,8120	2	C28
10	#13	C26	0,8397	4	C29
11	#14	C28	0,8408	3	C30
12	#7	C27	0,8515	7	C31
13	#15	C23	0,8617	3	C32
14	C30	C32	0,8686	6	C33
15	#18	C31	0,8701	8	C34
16	C29	C34	0,8772	12	C35
17	C33	C35	0,8867	18	C36
18	#19	C36	0,9257	19	C37
19	#4	C37	0,9375	20	C38

Anexo D. Los 5 pares más parecidos por algoritmo

Tabla 93. Pares más parecidos según cada algoritmo

Algoritmo	1.º	2.º	3.º	4.º	5.º
Levenshtein	#2–#10 (0,332)	#5–#20 (0,330)	#2–#6 (0,327)	#1–#2 (0,322)	#2–#8 (0,321)
Needleman-Wunsch	#2–#5 (0,764)	#2–#18 (0,751)	#2–#12 (0,749)	#2–#13 (0,736)	#2–#7 (0,722)
Coseno TF-IDF	#2–#6 (0,329)	#6–#9 (0,322)	#12–#20 (0,298)	#2–#9 (0,291)	#2–#20 (0,291)
Jaccard	#2–#18 (0,192)	#15–#20 (0,181)	#2–#20 (0,180)	#9–#18 (0,176)	#2–#11 (0,176)
GloVe (coseno)	#5–#20 (0,964)	#12–#20 (0,961)	#17–#20 (0,961)	#15–#20 (0,959)	#19–#20 (0,959)
fastText (coseno)	#5–#20 (0,983)	#12–#20 (0,981)	#15–#20 (0,980)	#12–#15 (0,979)	#19–#20 (0,978)

Anexo E. Glosario

Tabla 94. Glosario

Término	Significado
Abstract	Resumen de un artículo científico; es el único texto que se analiza.
Corpus	Conjunto de documentos analizados (aquí, 20 artículos).
Token	Cada palabra resultante de separar un texto.
Stopword	Palabra muy común sin valor temático (the, of, de, la).
Stemming	Reducir una palabra a una raíz aproximada (learning → learn).
Vector	Lista ordenada de números que representa un texto.
TF-IDF	Peso de una palabra: frecuencia en el texto multiplicada por su rareza en el corpus.
Coseno	Medida del ángulo entre dos vectores; 1 significa la misma dirección.
Jaccard	Proporción de palabras compartidas sobre palabras totales.
Embedding	Vector de números que representa el significado aproximado de una palabra o un texto.
Programación dinámica	Resolver un problema llenando una tabla con subproblemas más pequeños.
Levenshtein	Mínimo de inserciones, borrados y sustituciones entre dos cadenas.
Alineamiento	Forma de hacer encajar dos cadenas, con huecos donde haga falta.
Dendrograma	Árbol que muestra en qué orden y a qué distancia se fusionan los grupos.
Correlación cofenética	Qué tan fiel es el árbol a las distancias originales (cerca de 1 = más fiel).
Silueta	Qué tan bien separados están los grupos (entre −1 y 1).
LPS	Tabla de KMP: prefijo más largo que también es sufijo.
Hash	Número que resume una cadena de texto.
Hash rodante	Hash que se actualiza en un paso al desplazar la ventana una posición.
Colisión	Dos cadenas distintas con el mismo hash.

Término	Significado
Big O	Forma de expresar cómo crece el trabajo de un algoritmo con el tamaño de la entrada.
Mediana	Valor central de una lista ordenada; resiste valores extremos.
Spearman	Correlación entre los órdenes de dos listas (1 = mismo orden).

Anexo F. Guía de ejecución

F.1 Ejecutar la aplicación en local (Windows)

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
python -m pytest -q
python api.py
```

Después se abre http://127.0.0.1:8000/. En macOS o Linux se usa python3 -m venv .venv y source .venv/bin/activate; el resto es igual. Para solo ejecutar la aplicación basta con pip install -r requirements.txt.

F.2 Regenerar los datos y los documentos (opcional)

```
# 1) Corpus (requiere los 20 PDFs en la carpeta pdfs/)
python scripts/extraer_corpus.py

# 2) Embeddings (descarga ~1,4 GB la primera vez)
python scripts/generar_embeddings.py

# 3) Evidencias reales y documentos
python scripts/generar_evidencias.py
python scripts/generar_documentos.py
```

F.3 Desplegar en Render

Los pasos están en la sección 8.3. En resumen: subir el repositorio a GitHub, crear un *Web Service* en Render conectado a él con el comando de construcción pip install -r requirements.txt y el comando de arranque del Procfile, y comprobar /health.